



UNIVERSITÀ DEGLI STUDI DI PALERMO
DIPARTIMENTO DI INGEGNERIA

CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

Modelli decisionali basati sul calcolo quantistico

TESI DI LAUREA DI
Giuseppe Tripiciano

RELATORE
Prof. Daniele Peri

CONTRORELATORE
Prof.ssa Liliana Lo Presti

CORRELATORE
Dott. Filippo Vella

ANNO ACCADEMICO 2025 – 2026

MAGISTRALE



Indice

Sommario	iii
Elenco delle figure	iv
Elenco dei codici	v
1 Introduzione	1
1.1 Il processo decisionale	1
1.2 Il paradigma quantistico	1
1.3 Contributo sperimentale	2
1.4 Organizzazione del lavoro	3
2 Elementi di calcolo quantistico	4
2.1 Rappresentazione dello stato	4
2.2 Logica circuitale	5
2.2.1 Porte a singolo qubit	6
2.2.2 Porte multi-qubit	8
2.2.3 Entanglement e phase kickback	10
2.3 L'algoritmo di Grover	11
2.3.1 Tassonomia di riferimento	11
2.3.2 Formulazione del problema	12
2.3.3 Procedimento	13
2.3.4 Analisi geometrica	14
2.3.5 Limiti e sviluppi successivi	16
3 Il framework QLPP	18
3.1 Fondamenti formali	18
3.2 La programmazione logica quantistica	20
3.3 Architettura di base	21
3.3.1 Definizione degli operatori	22
3.3.2 Astrazione dell'hardware	24
3.3.3 Compilazione	25
3.4 Estensione progettuale	33
3.4.1 Catena di porte Toffoli	34

3.4.2	Cascata di porte a due qubit	37
3.4.3	Funzionalità di supporto	38
3.4.4	Altri operatori	44
3.4.5	Interfaccia Python/Qiskit	50
4	Scenario applicativo: il problema delle n regine	55
4.1	Panoramica progettuale	55
4.2	Motore logico QLPP	57
4.2.1	Inizializzazione	57
4.2.2	Vincolo di colonna	62
4.2.3	Vincolo di diagonale	66
4.2.4	Test di soddisfacimento	73
4.2.5	Composizione dell'oracolo	75
4.3	Flusso operativo Python/Qiskit	78
4.3.1	Configurazione	78
4.3.2	Composizione del circuito	82
4.3.3	Esecuzione su simulatore	84
4.3.4	Filtro di ammissibilità	86
4.3.5	Valutazione con SNR	86
4.3.6	Ispezione degli output	87
4.4	Risultati	88
4.5	Possibili ottimizzazioni	94
5	Conclusione	95
	Bibliografia	97

Sommario

Il calcolo quantistico rappresenta un paradigma emergente per la risoluzione di problemi decisionali, ma la progettazione a livello delle porte circuitali costituisce una sfida sempre maggiore al crescere della complessità del problema; di conseguenza, la ricerca si è orientata verso strumenti con un maggiore livello di astrazione, in grado di derivare circuiti a partire da formulazioni logiche. Nel presente lavoro di tesi viene proposta una metodologia operativa che integra la programmazione logica con il calcolo quantistico, e in particolare con l'algoritmo di ricerca di Grover, per la progettazione di modelli decisionali basati su vincoli, riprendendo ed estendendo il framework QLPP (*Quantum Logic Programming in Prolog*), introdotto in [1], e verificandone l'efficacia e le potenzialità su uno scenario applicativo, il noto benchmark del problema delle n regine. La trattazione illustra dapprima i presupposti teorici di tale metodologia, per poi descrivere l'architettura di base e l'estensione del framework QLPP, comprensiva di un'interfaccia in ambiente Python/Qiskit per l'interazione con Prolog e per la traduzione del circuito allo standard OpenQASM. Infine, vengono presentati ed esaminati in dettaglio il motore logico QLPP e il flusso operativo Python/Qiskit adottati nello scenario applicativo considerato, con una discussione dei risultati ottenuti e delle prospettive future.

Repository del progetto: <https://github.com/giuseppe-tripiciano/tesi.git>

Elenco delle figure

1.1	Motore logico QLPP.	2
1.2	Flusso operativo in Python/Qiskit.	2
2.1	Sfera di Bloch.	5
2.2	Porte a singolo qubit.	7
2.3	Porte a due qubit.	9
2.4	Porte a tre qubit.	9
2.5	Azione di controllo.	10
2.6	Stati di Bell.	11
2.7	Algoritmi fondamentali e applicazioni.	12
2.8	Algoritmo di Grover.	13
2.9	Operatore di Grover.	14
2.10	Rotazione indotta dall'operatore di Grover.	15
3.1	Architettura di base del framework QLPP.	22
3.2	Diagramma Graphviz dell'albero sintattico.	44
3.3	Interfaccia Python/Qiskit - Circuito Qiskit.	54
4.1	Vincolo di colonna - Diagramma parziale Qiskit.	64
4.2	Vincolo di colonna - Circuito parziale Qiskit.	65
4.3	Vincolo di diagonale - Diagramma parziale Qiskit (1).	69
4.4	Vincolo di diagonale - Diagramma parziale Qiskit (2).	70
4.5	Vincolo di diagonale - Circuito parziale Qiskit (1).	70
4.6	Vincolo di diagonale - Circuito parziale Qiskit (2).	71
4.7	Test di soddisfacimento senza ancilla - Circuito Qiskit.	74
4.8	Test di soddisfacimento con ancilla - Circuito Qiskit.	75
4.9	Istogramma per N=4, con un'iterazione.	89
4.10	Istogramma per N=4, con iterazioni ottimali.	90
4.11	Istogramma per N=5, con tre iterazioni.	92

Elenco dei codici

3.1	Estensione del parser Prolog.	23
3.2	Validazione degli operatori.	23
3.3	Mapping tra operatori e porte fisiche.	23
3.4	Contatore di qubit.	24
3.5	Istanza del registro di qubit.	25
3.6	Caricamento nel registro.	25
3.7	Pipeline standard di compilazione.	26
3.8	Fase di sintesi logica.	27
3.9	Fase di layering.	29
3.10	Semplificazione post-processing.	29
3.11	Operatori speciali.	30
3.12	Creazione ed estensione di un circuito.	31
3.13	Creazione ed estensione di un circuito - Query.	32
3.14	Catena di porte Toffoli - Inizializzazione.	34
3.15	Catena di porte Toffoli - Concatenazione.	35
3.16	Catena di porte Toffoli - Query.	36
3.17	Cascata di porte a due qubit.	37
3.18	Cascata di porte a due qubit - Query.	38
3.19	Strumenti per l'usabilità.	39
3.20	Strumenti per l'usabilità - Query.	40
3.21	Log delle traduzioni.	41
3.22	Log delle traduzioni - Query.	42
3.23	Modulo di rendering dell'albero sintattico.	42
3.24	Operatore OR.	45
3.25	Generalizzazione dell'operatore OR.	45
3.26	Parity check.	46
3.27	Entanglement GHZ.	47
3.28	Incremento e decremento.	48
3.29	Misurazione multipla.	49
3.30	Modulo di traduzione da QLPP a QASM.	50
3.31	Interfaccia Python/Qiskit.	52
3.32	Interfaccia Python/Qiskit - Query.	53
3.33	Interfaccia Python/Qiskit - Circuito QASM.	54

4.1	Inizializzazione.	57
4.2	Dimensionamento del registro.	60
4.3	Partizionamento dello stato.	61
4.4	Moduli per la verifica e per l'offset.	61
4.5	Vincolo di colonna.	62
4.6	Vincolo di colonna - Circuito parziale QLPP.	65
4.7	Vincolo di diagonale.	66
4.8	Vincolo di diagonale - Circuito parziale QLPP.	71
4.9	Test di soddisfacimento.	73
4.10	Test di soddisfacimento - Circuito QLPP.	74
4.11	Composizione dell'oracolo.	76
4.12	Configurazione.	79
4.13	Composizione del circuito.	82
4.14	Esecuzione su simulatore.	85
4.15	Filtro di ammissibilità.	86
4.16	Valutazione con SNR.	87
4.17	Ispezione degli output.	87
4.18	Simulazione con AerSimulator per $N=4$	88
4.19	Scacchiere soluzione per $N=4$	90
4.20	Simulazione con DDSIM per $N=5$	90
4.21	Scacchiere soluzione per $N=5$	92

Capitolo 1

Introduzione

1.1 Il processo decisionale

Il processo decisionale (*decision making*) consiste nella selezione, all'interno di un insieme di possibilità, di una o più configurazioni che soddisfano determinati vincoli. In ambito informatico, i modelli decisionali sono tipicamente formulati come sistemi discreti, la cui logica è descritta attraverso rappresentazioni booleane. "Decidere" equivale a risolvere un problema di soddisfacibilità di vincoli (*Constraint Satisfaction Problem*, CSP): data una codifica binaria delle configurazioni possibili, il decisore identifica i vettori di bit che non violano i vincoli del problema. Tuttavia, per la crescita esponenziale dello spazio delle possibili configurazioni (2^n), nell'approccio classico l'elaborazione di modelli decisionali diventa onerosa, se non proibitiva, per i limiti architetturali dell'hardware, nonostante l'utilizzo di algoritmi altamente efficienti basati su tecniche come il *backtracking* e il *pruning*. Non è un caso che molti problemi CSP appartengano alla classe dei problemi NP (*Nondeterministic Polynomial time*), per i quali non è nota, ad oggi, alcuna strategia risolutiva efficiente nel caso generale. Da queste considerazioni nasce la necessità di un paradigma più potente, in grado di gestire l'esplosione combinatoria delle configurazioni, e, in tal senso, il calcolo quantistico rappresenta il candidato ideale.

1.2 Il paradigma quantistico

L'evoluzione delle tecnologie quantistiche ha condotto ad una crescente disponibilità di hardware per l'elaborazione dell'informazione. Il calcolo quantistico (*quantum computing*) offre un cambio di paradigma capace di superare le limitazioni intrinseche della computazione classica; l'informazione non è più veicolata da stati deterministici binari, ma da una sovrapposizione di tutte le configurazioni possibili, alle quali sono applicati principi come l'interferenza di fase e l'amplificazione dell'ampiezza di probabilità. In tale contesto, si inserisce l'algoritmo di ricerca di Grover che fornisce un guadagno quadratico rispetto agli algoritmi classici di ricerca su spazi non strutturati. In Grover il decisore (detto oracolo) è un operatore che racchiude la logica del modello decisionale ed è in grado di marcare gli stati

che soddisfano i vincoli del problema; tuttavia, l'implementazione circuitale dell'oracolo di Grover è una sfida progettuale che risulta complessa nell'approccio manuale standard degli ambienti di sviluppo.

1.3 Contributo sperimentale

La progettazione di circuiti quantistici necessita di strumenti che descrivano ad alto livello gli operatori del calcolo quantistico, in modo da poter effettuare una traduzione automatica da formule logiche a gate fisici e, più in generale, di funzioni complesse a circuiti quantistici. Il presente lavoro di tesi si propone di inserirsi in tale contesto di ricerca, estendendo un framework di programmazione logica quantistica in Prolog (*Quantum Logic Programming in Prolog*, QLPP), discusso in [1], ed impiegandolo come motore logico dell'oracolo dell'algoritmo di ricerca di Grover (Figura 1.1). Il flusso operativo si avvale di Python, linguaggio di alto livello ampiamente utilizzato dalla comunità scientifica, con la libreria Qiskit, ambiente open-source di IBM per la progettazione dei circuiti quantistici; in particolare, prevede la query all'oracolo QLPP, la traduzione dell'oracolo dal formato QLPP allo standard OpenQASM 3.0, l'integrazione con l'algoritmo di Grover e il testing (Figura 1.2). Le potenzialità di questa pipeline vengono mostrate con lo scenario applicativo del problema delle n regine, un problema classico di soddisfacibilità di vincoli, in cui devono essere disposte n regine su una scacchiera $n \times n$ in modo che nessuna regina ne attacchi un'altra (ovvero nessuna regina si trovi sulla stessa colonna o diagonale di un'altra).

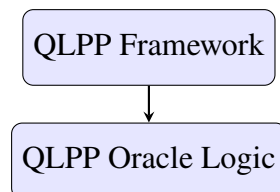


Figura 1.1: Motore logico QLPP.

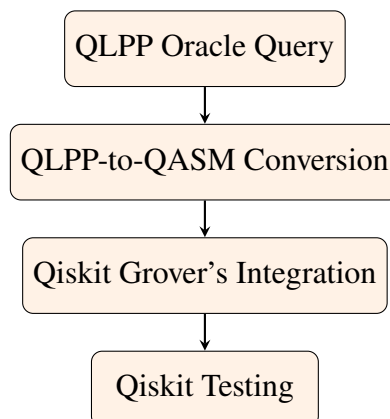


Figura 1.2: Flusso operativo in Python/Qiskit.

1.4 Organizzazione del lavoro

Il lavoro di tesi si articola partendo dai presupposti teorici fino alla verifica sperimentale del metodo proposto. L'iter è il seguente:

- **Capitolo 2** (Elementi di calcolo quantistico): illustra i fondamenti teorici del calcolo quantistico (dal libro di riferimento [2]) propedeutici al lavoro sperimentale, con enfasi sull'algoritmo di Grover;
- **Capitolo 3** (Il framework QLPP): presenta i fondamenti formali della logica quantistica (discussa in [3]) e la programmazione logica quantistica, e descrive l'architettura del framework QLPP e la relativa estensione apportata dal contributo sperimentale, che include l'interfaccia in ambiente Python/Qiskit per l'interrogazione verso Prolog e per la traduzione del circuito da QLPP allo standard OpenQASM;
- **Capitolo 4** (Scenario applicativo: il problema delle n regine): descrive il contributo sperimentale che comprende l'implementazione in QLPP dell'oracolo per il problema delle n regine e la sua integrazione con l'algoritmo di Grover in Qiskit, e infine analizza e discute i risultati e le possibili ottimizzazioni;
- **Capitolo 5** (Conclusione): racchiude le considerazioni conclusive, i limiti della ricerca e i possibili sviluppi futuri.

Capitolo 2

Elementi di calcolo quantistico

2.1 Rappresentazione dello stato

Analogamente al concetto di bit per il calcolo classico, il *quantum bit* (o qubit) è l'oggetto matematico, oltre che fisico, alla base del calcolo quantistico. Mentre lo stato di un bit può essere solo 0 o 1, lo stato di un qubit è in generale una combinazione lineare, detta sovrapposizione, dei due stati ed è rappresentato da un vettore in uno spazio di Hilbert bidimensionale.

Utilizzando la notazione di Dirac, lo stato di un qubit si formula nel modo seguente:

$$|\psi\rangle = \alpha |0\rangle + \beta |1\rangle \quad (2.1)$$

dove $|0\rangle$ e $|1\rangle$ sono detti stati della base computazionale e costituiscono una base ortonormale dello spazio vettoriale, α e β sono numeri complessi che rappresentano ampiezze di probabilità. È importante distinguere la differenza tra lo stato interno di un qubit e ciò che possiamo osservare con la misurazione: non è possibile misurare α e β , ma soltanto 0, con probabilità $|\alpha|^2$, e 1, con probabilità $|\beta|^2$ (con la misurazione il qubit "collassa" allo stato 0 o 1). Per la proprietà di normalizzazione della probabilità, vale che $|\alpha|^2 + |\beta|^2 = 1$ e di conseguenza il vettore di stato è unitario.

Per le caratteristiche di α e β è possibile riformulare l'equazione (2.1) nel modo seguente:

$$|\psi\rangle = \cos \frac{\theta}{2} |0\rangle + e^{i\phi} \sin \frac{\theta}{2} |1\rangle \quad (2.2)$$

in cui θ e ϕ sono coordinate angolari, con $\theta \in [0, \pi]$ e $\phi \in [0, 2\pi)$. Dalla formula è stato ommesso il fattore comune $e^{i\gamma}$, una fase globale che non ha alcun effetto osservabile sullo stato quantistico. In questa rappresentazione, $\alpha = \cos \frac{\theta}{2}$ è un numero reale non negativo e $\beta = e^{i\phi} \sin \frac{\theta}{2}$ è un numero complesso.

θ e ϕ identificano un punto su una sfera tridimensionale di raggio unitario, detta sfera di Bloch (Figura 2.1), in cui il polo nord corrisponde allo stato $|0\rangle$, il polo sud a $|1\rangle$ e l'equatore all'insieme di stati in cui la sovrapposizione è uniforme (la probabilità di $|0\rangle$ e $|1\rangle$ è $1/2$). In particolare, θ è l'angolo misurato rispetto all'asse Z positivo e definisce le ampiezze di

probabilità come rotazione attorno all'asse Y , e ϕ è l'angolo misurato rispetto all'asse X positivo e definisce la fase relativa (da cui dipende l'interferenza) come rotazione attorno all'asse Z .

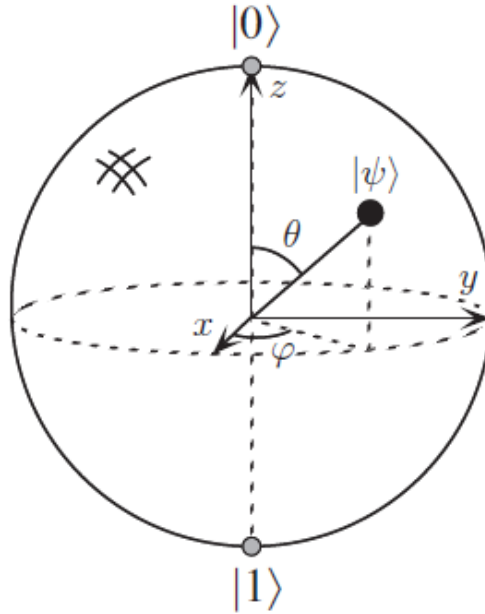


Figura 2.1: Sfera di Bloch.

Analogamente, lo stato che descrive un sistema di due qubit è:

$$|\psi\rangle = \alpha_{00} |00\rangle + \alpha_{01} |01\rangle + \alpha_{10} |10\rangle + \alpha_{11} |11\rangle$$

in cui $|00\rangle$, $|01\rangle$, $|10\rangle$ e $|11\rangle$ sono gli stati della base computazionale, ai quali sono associate le rispettive ampiezze di probabilità normalizzate.

Lo stato quantistico può essere generalizzato per un sistema di n qubit, in cui gli stati della base computazionale sono della forma $|x_1 x_2 \dots x_n\rangle$ (con $x_i \in \{0, 1\}$, $1 \leq i \leq n$), e il sistema è rappresentato da 2^n ampiezze di probabilità (ovvero da un vettore in uno spazio di Hilbert di dimensione 2^n).

2.2 Logica circuitale

La meccanica quantistica impone che gli operatori debbano essere lineari; questo significa che le relative porte logiche possono essere espresse come matrici. In particolare, per la conservazione della probabilità, una matrice rappresenta una porta logica quantistica valida se è unitaria ($U^\dagger U = I$). L'unitarietà delle porte logiche ne garantisce anche l'invertibilità, grazie alla quale le trasformazioni, a differenza di quelle classiche, sono sempre reversibili. L'unitarietà vincola i circuiti ad essere aciclici, ovvero ad avere un flusso temporale unidirezionale e quindi privo di meccanismi di feedback. Inoltre, per la linearità, si dimostra che è impossibile la copia di stati non noti (*no-cloning theorem*).

2.2.1 Porte a singolo qubit

Di seguito sono presentate le porte logiche standard a singolo qubit:

- **Hadamard** (o H): operazione di cambiamento di base rappresentata dalla matrice

$$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \quad (2.3)$$

che corrisponde a un mapping bidirezionale tra la base di Z (base computazionale) e la base di X (base della fase) e, in particolare, tra $|0\rangle$, $|1\rangle$ e, rispettivamente, gli stati di sovrapposizione uniforme $|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$, $|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$;
significato geometrico: rotazione di π attorno alla bisettrice del piano XZ , ovvero una rotazione di $\pi/2$ attorno all'asse Y seguita da una di π attorno all'asse X ;

- **X di Pauli**: operazione di bit-flip (analogica al *NOT* classico) rappresentata dalla matrice

$$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix} \quad (2.4)$$

che corrisponde a uno scambio delle ampiezze di probabilità e, in particolare, degli stati $|0\rangle$ e $|1\rangle$;

significato geometrico: rotazione di π attorno all'asse X ;

- **Z di Pauli**: operazione di phase-flip rappresentata dalla matrice

$$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad (2.5)$$

che corrisponde a un'inversione della fase relativa;

significato geometrico: rotazione di π attorno all'asse Z ;

- **Y di Pauli**: operazione sia di bit-flip che di phase-flip rappresentata dalla matrice

$$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix} \quad (2.6)$$

che corrisponde a uno scambio delle ampiezze di probabilità e a un'inversione della fase relativa, a meno di una fase globale i ;

significato geometrico: rotazione di π attorno all'asse Y .

Le porte H , X , Z e Y sono hermitiane ($U^\dagger = U$) e quindi auto-inverse ($U^2 = I$).

Valgono, inoltre, le seguenti trasformazioni delle porte di Pauli tramite coniugazione di H :

$$HXH = Z, \quad (2.7)$$

$$HZH = X, \quad (2.8)$$

$$HYH = -Y. \quad (2.9)$$

Alle precedenti porte si aggiungono le seguenti, necessarie all'elaborazione di fasi più piccole:

- S (o $Phase$): operazione di phase-shift rappresentata dalla matrice

$$S = \begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix} \quad (2.10)$$

che corrisponde a uno sfasamento relativo di $\pi/2$;
significato geometrico: rotazione di $\pi/2$ attorno all'asse Z ;

- T (o $\pi/8$): operazione di phase-shift rappresentata dalla matrice

$$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix} \quad (2.11)$$

che corrisponde a uno sfasamento relativo di $\pi/4$;
significato geometrico: rotazione di $\pi/4$ attorno all'asse Z .

La porta T storicamente è chiamata $\pi/8$ perché può essere trasformata estraendo un fattore di fase globale $e^{i\pi/8}$. Le porte S e T non sono hermitiane ($S^2 = Z$ e $T^2 = S$); gli operatori inversi $S^\dagger$ e $T^\dagger$ prendono il nome rispettivamente di S -dagger (o SDG) e T -dagger (o TDG). Le porte a singolo qubit sono riassunte in Figura 2.2.

Hadamard	$\text{---} \boxed{H} \text{---}$	$\frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$
Pauli- X	$\text{---} \boxed{X} \text{---}$	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$
Pauli- Y	$\text{---} \boxed{Y} \text{---}$	$\begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$
Pauli- Z	$\text{---} \boxed{Z} \text{---}$	$\begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$
Phase	$\text{---} \boxed{S} \text{---}$	$\begin{bmatrix} 1 & 0 \\ 0 & i \end{bmatrix}$
$\pi/8$	$\text{---} \boxed{T} \text{---}$	$\begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$

Figura 2.2: Porte a singolo qubit.

Qualsiasi porta a singolo qubit può essere espressa, a meno di una fase globale, con al più tre rotazioni attorno a due assi ortogonali alternati (tipicamente Z - Y - Z o Z - X - Z) mediante i seguenti operatori, definiti a partire dalle matrici di Pauli:

$$R_x(\theta_x) = e^{-i\theta_x X/2} = \cos(\theta_x/2)I - i \sin(\theta_x/2)X = \begin{bmatrix} \cos(\theta_x/2) & -i \sin(\theta_x/2) \\ -i \sin(\theta_x/2) & \cos(\theta_x/2) \end{bmatrix}, \quad (2.12)$$

$$R_y(\theta_y) = e^{-i\theta_y Y/2} = \cos(\theta_y/2)I - i \sin(\theta_y/2)Y = \begin{bmatrix} \cos(\theta_y/2) & -\sin(\theta_y/2) \\ \sin(\theta_y/2) & \cos(\theta_y/2) \end{bmatrix}, \quad (2.13)$$

$$R_z(\theta_z) = e^{-i\theta_z Z/2} = \cos(\theta_z/2)I - i \sin(\theta_z/2)Z = \begin{bmatrix} e^{-i\theta_z/2} & 0 \\ 0 & e^{i\theta_z/2} \end{bmatrix}. \quad (2.14)$$

2.2.2 Porte multi-qubit

Di seguito sono presentate le porte logiche standard a due qubit:

- **Controlled-NOT** (o *CNOT*, *CX*): operazione di bit-flip controllato rappresentata dalla matrice

$$CNOT = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix} \quad (2.15)$$

che scambia le ampiezze di probabilità degli stati in cui il qubit di controllo è in $|1\rangle$, ovvero $|10\rangle$ e $|11\rangle$;

- **SWAP**: operazione di swap rappresentata dalla matrice

$$SWAP = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2.16)$$

che scambia le ampiezze di probabilità degli stati in cui i due qubit sono diversi, ovvero $|01\rangle$ e $|10\rangle$;

- **Controlled-Z** (o *CZ*): operazione di phase-flip controllato rappresentata dalla matrice

$$CZ = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix} \quad (2.17)$$

che inverte la fase relativa dello stato $|11\rangle$ (i due qubit sono sia controlli sia target).

Le porte *CNOT*, *SWAP* e *CZ* sono hermitiane e quindi auto-inverse.

La *CNOT* si può considerare una generalizzazione dello *XOR* classico, in quanto può essere scritta nel modo seguente:

$$|c, t\rangle \mapsto |c, c \oplus t\rangle \quad (2.18)$$

dove c è il qubit di controllo, t il qubit target e " $\oplus$ " indica l'addizione modulo 2 (o *XOR*).

La porta *SWAP* è tipicamente realizzata come sequenza di tre porte *CNOT*, in cui si alternano i qubit per controllo e target. Le porte a due qubit sono riassunte in Figura 2.3.

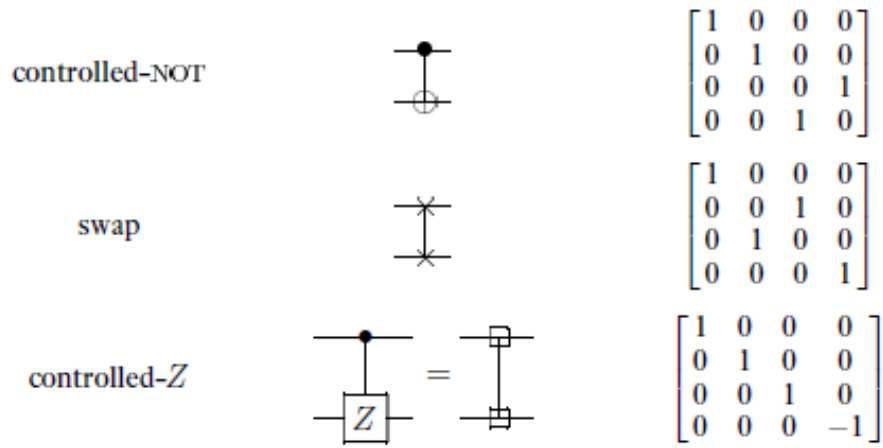


Figura 2.3: Porte a due qubit.

Alle precedenti porte, si aggiungono le seguenti a tre qubit:

- **Toffoli** (o *TOF*, *CCNOT*, *CCX*): estensione controllata della *CNOT*;
- **Fredkin** (o *CSWAP*): estensione controllata della *SWAP*.

La Toffoli si può considerare una generalizzazione di *AND* e *NAND* classici, in quanto può essere scritta nel modo seguente:

$$|c_1, c_2, t\rangle \mapsto |c_1, c_2, t \oplus (c_1 \cdot c_2)\rangle \quad (2.19)$$

dove c_1 e c_2 sono i qubit di controllo, t è il qubit target, " $\oplus$ " indica l'addizione modulo 2 (o *XOR*) e " $\cdot$ " indica l'*AND*; se t è in $|0\rangle$ la Toffoli equivale a un *AND*, se è in $|1\rangle$ a un *NAND*.

Le porte a tre qubit sono riassunte in Figura 2.4.

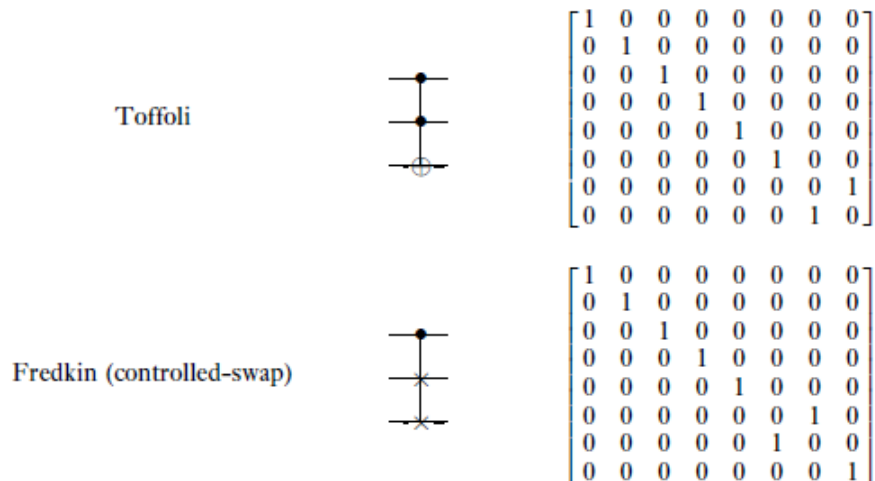


Figura 2.4: Porte a tre qubit.

Per qualunque operatore unitario U , può essere realizzata la versione controllata CU e, in generale, la versione multi-controllata $C^n U$.

2.2.3 Entanglement e phase kickback

La logica multi-qubit rende necessaria l'introduzione del concetto di separabilità dello stato quantistico, dove per separabilità si intende la possibilità di esprimere lo stato come prodotto tensoriale ($\otimes$) di più stati.

Considerato un registro di qubit u e un operatore unitario U , $|u\rangle$ è detto autostato di U se vale la seguente:

$$U |u\rangle = e^{i\phi} |u\rangle \quad (2.20)$$

ovvero se U lascia invariato lo stato $|u\rangle$, a meno di una fase globale (l'autovalore).

Fissato un qubit c in uno stato $|c\rangle = \alpha |0\rangle + \beta |1\rangle$, il sistema che comprende c e u è descritto dallo stato:

$$|\psi_i\rangle = (\alpha |0\rangle + \beta |1\rangle) \otimes |u\rangle \quad (2.21)$$

Definito l'operatore unitario CU con controllo c e target u , lo stato risultante è:

- non separabile, se $|u\rangle$ non è un autostato dell'operatore U , come mostrato dalla formula

$$|\psi_f\rangle = \alpha |0\rangle |u\rangle + \beta |1\rangle |u'\rangle \quad (2.22)$$

in cui l'azione di controllo di CU trasforma lo stato target $|u\rangle$ nello stato $|u'\rangle$, introducendo quindi una correlazione quantistica (*entanglement*);

- separabile, se $|u\rangle$ è un autostato dell'operatore U , come mostrato dalla formula

$$|\psi_f\rangle = (\alpha |0\rangle + \beta e^{i\phi} |1\rangle) \otimes |u\rangle \quad (2.23)$$

in cui, se $\phi \neq 2k\pi$ (con $k \in \mathbb{Z}$), l'azione di controllo di CU "rimbalza" sul controllo stesso, che assorbe la fase globale $e^{i\phi}$ applicata allo stato target $|u\rangle$ nella propria fase relativa (*phase kickback*).

Va sottolineato di conseguenza che, in generale, la fase globale introdotta da una porta U non è trascurabile se la porta è condizionata.

Si consideri, come esempio, l'azione di controllo del circuito mostrato in Figura 2.5:

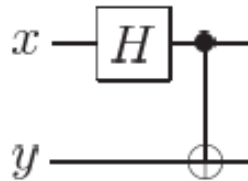


Figura 2.5: Azione di controllo.

Se x e y sono entrambi in uno stato della base computazionale, si ottengono dei noti esempi di stati in entanglement, detti stati di Bell, la cui tabella di verità è mostrata in Figura 2.6.

Se y si trova in uno stato della base della fase (autostato di X), ovvero in $|+\rangle$ o $|-\rangle$, lo stato risultante è separabile; per $y = |-\rangle$, in particolare, si ottiene un esempio di phase kickback, con $Xy = X|-\rangle = -|-\rangle$, la cui fase globale, -1 , è assorbita dalla fase relativa del controllo.

In	Out
$ 00\rangle$	$(00\rangle + 11\rangle)/\sqrt{2} \equiv \beta_{00}\rangle$
$ 01\rangle$	$(01\rangle + 10\rangle)/\sqrt{2} \equiv \beta_{01}\rangle$
$ 10\rangle$	$(00\rangle - 11\rangle)/\sqrt{2} \equiv \beta_{10}\rangle$
$ 11\rangle$	$(01\rangle - 10\rangle)/\sqrt{2} \equiv \beta_{11}\rangle$

Figura 2.6: Stati di Bell.

2.3 L'algoritmo di Grover

2.3.1 Tassonomia di riferimento

Prima di approfondire l'algoritmo di ricerca di Grover, risulta utile presentare la panoramica degli algoritmi fondamentali in cui si colloca e delle relative applicazioni (Figura 2.7).

In letteratura, sulla base del metodo di elaborazione impiegato, gli algoritmi fondamentali sono tradizionalmente classificati nel modo seguente:

- algoritmi basati sulla trasformata di Fourier quantistica (*QFT*), che offrono un guadagno esponenziale per specifici problemi strutturati, di cui un noto esempio è la fattorizzazione in numeri primi, che viene risolta dall'algoritmo di Shor, compromettendo la sicurezza della crittografia asimmetrica, in particolare di RSA;
- algoritmi di ricerca quantistica (*quantum search*), di cui quello di Grover è capostipite, basati su amplificazione di ampiezza, che offrono un guadagno quadratico su una vasta gamma di applicazioni come:
 - alcuni problemi della classe NP (*Nondeterministic Polynomial time*), per i quali la verifica di una soluzione può essere effettuata in tempo polinomiale;
 - approssimazione statistica;
 - violazione di algoritmi crittografici a chiave simmetrica e di funzioni hash.

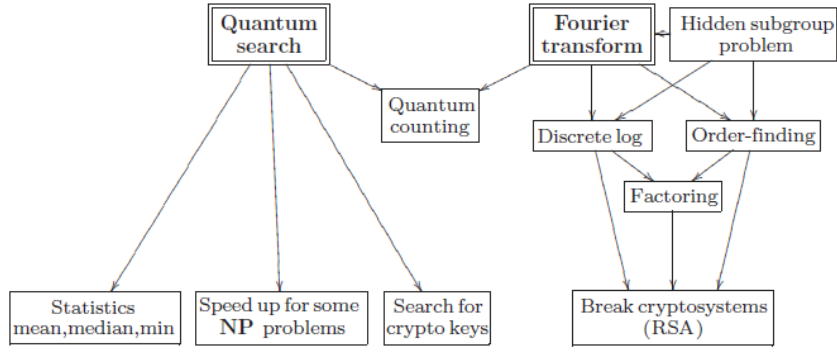


Figura 2.7: Algoritmi fondamentali e applicazioni.

Per completezza, a questi algoritmi si affiancano i seguenti:

- algoritmi di simulazione quantistica (*quantum simulation*), basati sull'operatore hamiltoniano, che descrivono lo stato e l'evoluzione di sistemi fisici, non trattabili dal calcolo classico per la crescita esponenziale dello spazio di Hilbert;
- algoritmi basati sui *quantum walks*, l'analogo quantistico dei *random walks*, che sono impiegati per l'esplorazione e l'analisi di grafi.

Si consulti [4] per una discussione sulle applicazioni delle classi algoritmiche presentate.

2.3.2 Formulazione del problema

L'algoritmo di Grover, introdotto da Lov Grover in [5], ha l'obiettivo di ottimizzare la risoluzione di problemi di ricerca in spazi non strutturati, ovvero privi di ordine e relazioni logiche (ricerca esaustiva o a forza bruta).

Si assume uno spazio non strutturato di $N = 2^n$ elementi, in modo che sia possibile effettuarne il mapping in uno spazio di Hilbert mediante un registro di n qubit; ogni elemento corrisponde a uno stato della base computazionale $|x\rangle$ con $x \in \{0, 1\}^n$.

Si assume, inoltre, di conoscere il numero di soluzioni $M \in [1, N]$.

Si suppone di disporre di un oracolo quantistico O , ovvero di un operatore unitario black-box capace di marcare le soluzioni del problema; l'oracolo O è associato a una funzione booleana $f(x)$, che riconosce le soluzioni sulla base della logica del problema.

L'azione dell'oracolo può essere scritta nel modo seguente:

$$|x\rangle \xrightarrow{O} (-1)^{f(x)} |x\rangle \quad (2.24)$$

e in particolare

- $f(x) = 1 \implies |x\rangle \mapsto -|x\rangle$
- $f(x) = 0 \implies |x\rangle \mapsto |x\rangle$

ovvero, se x è soluzione, l'oracolo O marca lo stato $|x\rangle$ con un'inversione di fase (diretta o tramite phase kickback con qubit extra in $|-\rangle$).

È importante considerare che, dal punto di vista logico-computazionale, riconoscere e conoscere una soluzione assumono due significati differenti; il meccanismo interno dell'oracolo è in grado di verificare che uno stato sia soluzione, ma non di determinarla.

L'idea alla base dell'algoritmo di Grover è di estrarre gli stati soluzione marcati dall'oracolo, minimizzando il costo computazionale; il procedimento è discusso nel prossimo paragrafo.

2.3.3 Procedimento

Lo schema circuitale dell'algoritmo di Grover è presentato in Figura 2.8.

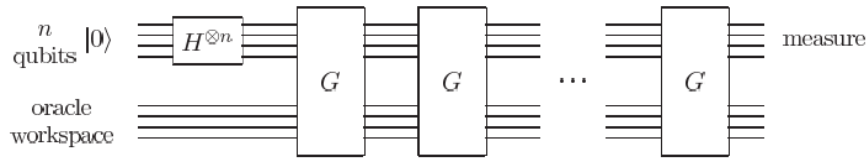


Figura 2.8: Algoritmo di Grover.

L'algoritmo prevede la seguente inizializzazione in sovrapposizione uniforme del registro di n qubit:

$$|\psi\rangle = H^{\otimes n} |0\rangle^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{x=0}^{N-1} |x\rangle \quad (2.25)$$

Per alleggerire la notazione, lo stato $|0\rangle^{\otimes n}$ sarà in seguito semplificato con $|0\rangle$.

La logica interna dell'oracolo e gli eventuali qubit extra ad essa necessari non hanno rilevanza ai fini dell'algoritmo.

Il procedimento consiste nell'applicazione iterativa dell'operatore di Grover G , che può essere scomposto nei seguenti operatori:

1. L'oracolo O ;
2. La trasformata $H^{\otimes n}$;
3. Il phase-flip condizionale, la cui azione su uno stato $|x\rangle$ è descritta da:

$$|x\rangle \mapsto -(-1)^{\delta_{x0}} |x\rangle = (2|0\rangle\langle 0| - I) |x\rangle \quad (2.26)$$

dove δ_{x0} è la delta di Kronecker tra x e 0, che condiziona l'inversione di fase di $|x\rangle$ in modo che sia applicata solo se $x = 0$ (inversione rispetto a $|0\rangle$);

4. La trasformata $H^{\otimes n}$.

Lo schema circuitale dell'operatore G è riportato in Figura 2.9.

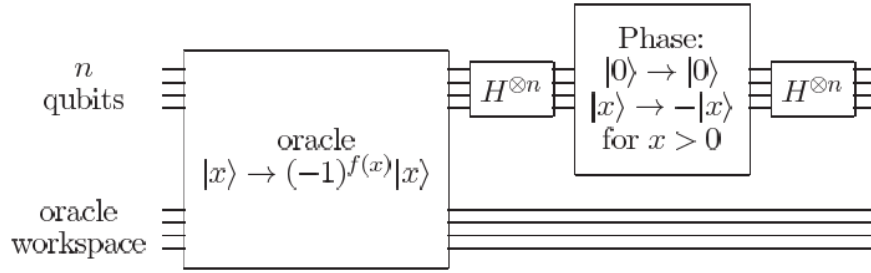


Figura 2.9: Operatore di Grover.

I passi 2, 3 e 4 compongono l'operatore di diffusione D :

$$D = H^{\otimes n} (2 |0\rangle \langle 0| - I) H^{\otimes n} = 2 |\psi\rangle \langle \psi| - I \quad (2.27)$$

Si assume che $M \leq N/2$ affinché D sia efficace (se $M > N/2$, è necessario estendere lo spazio a $2N$ elementi mediante un registro di $n + 1$ qubit).

L'operatore D è anche detto di inversione rispetto alla media, in quanto l'azione che applica sulla generica sovrapposizione è descritta da:

$$\sum_{x=0}^{N-1} \alpha_x |x\rangle \xrightarrow{D} \sum_{x=0}^{N-1} \left[-\alpha_x + 2\langle \alpha \rangle \right] |x\rangle \quad (2.28)$$

dove α_x è l'ampiezza di probabilità associata a uno stato $|x\rangle$ e $\langle \alpha \rangle$ è la media aritmetica di tutte le ampiezze di probabilità. Le ampiezze di probabilità negative (associate a stati soluzione) assumono valori positivi significativamente oltre la media, mentre quelle positive (associate a stati non soluzione) vengono attenuate al di sotto della media.

L'operatore G può quindi essere espresso come:

$$G = (2 |\psi\rangle \langle \psi| - I) O \quad (2.29)$$

che viene ripetuto fino al raggiungimento della probabilità ottimale di successo di misurare uno degli stati soluzione.

La trattazione prosegue con un'analisi geometrica che consente una rappresentazione semplificata dell'evoluzione dello stato, l'individuazione del numero ottimale di iterazioni e la valutazione del costo computazionale.

2.3.4 Analisi geometrica

Si indichi con $\sum'_x$ la sommatoria per tutti gli x soluzione e con $\sum''_x$ la sommatoria per tutti gli x non soluzione. Si definiscono i seguenti stati $|\alpha\rangle$ e $|\beta\rangle$:

$$|\alpha\rangle = \frac{1}{\sqrt{N-M}} \sum''_x |x\rangle, \quad (2.30)$$

$$|\beta\rangle = \frac{1}{\sqrt{M}} \sum'_x |x\rangle \quad (2.31)$$

che sono le sovrapposizioni uniformi rispettivamente di stati non soluzione e soluzione. Lo stato iniziale di sovrapposizione uniforme $|\psi\rangle$ può essere riformulato come segue:

$$|\psi\rangle = \sqrt{\frac{N-M}{N}} |\alpha\rangle + \sqrt{\frac{M}{N}} |\beta\rangle \quad (2.32)$$

ovvero $|\psi\rangle$ si trova nel sottospazio bidimensionale di Hilbert generato dagli stati $|\alpha\rangle$ e $|\beta\rangle$.

L'oracolo O effettua una riflessione rispetto allo stato $|\alpha\rangle$ in questo sottospazio.

Il diffusore D effettua una riflessione rispetto allo stato $|\psi\rangle$ nello stesso sottospazio.

L'operatore di Grover G applica, quindi, la composizione di due riflessioni, equivalente ad una rotazione, nel sottospazio generato da $|\alpha\rangle$ e $|\beta\rangle$.

Ponendo $\cos \frac{\theta}{2} = \sqrt{\frac{N-M}{N}}$, si ottiene $|\psi\rangle = \cos \frac{\theta}{2} |\alpha\rangle + \sin \frac{\theta}{2} |\beta\rangle$, con $\theta \in (0, \pi/2]$.

L'azione dell'operatore G sullo stato $|\psi\rangle$ è la seguente:

$$G|\psi\rangle = \cos \frac{3\theta}{2} |\alpha\rangle + \sin \frac{3\theta}{2} |\beta\rangle \quad (2.33)$$

ovvero una rotazione di θ dello stato $|\psi\rangle$, come mostrato in Figura 2.10.

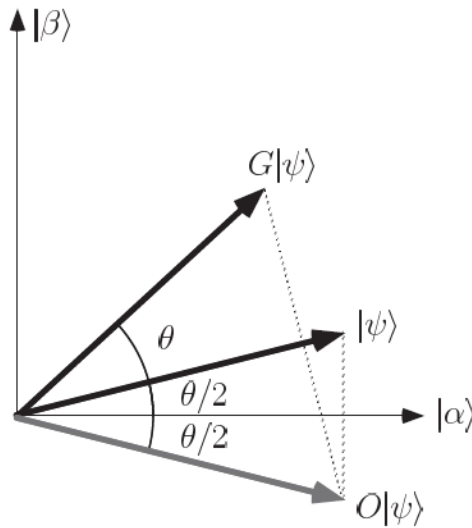


Figura 2.10: Rotazione indotta dall'operatore di Grover.

Analogamente, per k iterazioni dell'operatore G si ottiene:

$$G^k |\psi\rangle = \cos \left(\frac{2k+1}{2} \theta \right) |\alpha\rangle + \sin \left(\frac{2k+1}{2} \theta \right) |\beta\rangle. \quad (2.34)$$

La probabilità di misurare una delle soluzioni dopo k iterazioni è quindi data da:

$$P_{success}(k) = \sin^2 \left(\frac{2k+1}{2} \theta \right). \quad (2.35)$$

L'angolo iniziale $\theta/2$, e quindi quello di rotazione θ , si determinano dalla seguente relazione:

$$\theta/2 = \arcsin \left(\sqrt{M/N} \right) \quad (2.36)$$

e in particolare

$$M \ll N \implies \theta/2 \approx \sin(\theta/2) = \sqrt{M/N}. \quad (2.37)$$

Il numero ottimale di iterazioni si individua nel modo seguente:

$$R = \text{round} \left(\frac{\pi/2 - \theta/2}{\theta} \right) \quad (2.38)$$

e in particolare

$$M \ll N \implies R = \text{round} \left(\frac{\pi}{4} \sqrt{\frac{N}{M}} - \frac{1}{2} \right). \quad (2.39)$$

$(\pi/2 - \theta/2)$ è l'angolo tra lo stato iniziale $|\psi\rangle$ e lo stato $|\beta\rangle$; per la natura discreta delle iterazioni, la rotazione indotta da G^R porta a un angolo finale $\theta_R \in [\pi/4, 3\pi/4]$ e $P_{\text{success}}(R) \geq 1/2$. In particolare, per $M \ll N$, $\theta_R \in [\pi/2 - \epsilon, \pi/2 + \epsilon]$ e $P_{\text{success}}(R) = 1 - \epsilon^2$, con $\epsilon \approx \sqrt{M/N}$.

Il costo computazionale si valuta tramite il limite superiore $R \leq \left\lceil \frac{\pi}{4} \sqrt{N/M} \right\rceil$ e corrisponde quindi a $O\left(\sqrt{N/M}\right)$, ovvero a un guadagno quadratico rispetto a $O(N/M)$ del caso medio classico; tale vantaggio diventa significativo per $M \ll N$, scenario operativo standard dell'algoritmo di Grover.

Inoltre, si può osservare che, se l'obiettivo è trovare ogni soluzione, l'algoritmo va eseguito mediamente $(M \ln M)$ volte, con costo computazionale complessivo di $O\left(\sqrt{N \cdot M} \ln M\right)$ rispetto a $O(N)$ del caso classico; per tale scenario, a causa dei duplicati delle soluzioni misurate, il vantaggio si ottiene strettamente per $M \ll N$.

2.3.5 Limiti e sviluppi successivi

L'algoritmo di Grover si scontra con un notevole divario tra astrazione teorica e applicabilità reale. Di seguito vengono riportati i due principali limiti dell'algoritmo con i relativi sviluppi successivi che hanno consentito di colmare tale divario e di giungere a tutti gli effetti a un paradigma applicativo:

- L'approccio black-box: l'algoritmo è "cieco" rispetto all'oracolo e quindi ne ignora anche il costo computazionale e la relativa complessità circuitale.

Tuttavia, nella maggioranza dei casi pratici, l'oracolo costituisce il collo di bottiglia dell'algoritmo e, in particolare, per gli attuali hardware quantistici NISQ (*Noisy Intermediate-Scale Quantum*), la chiamata iterativa dell'oracolo comporta un accumulo di rumore tale da causare il fallimento dell'algoritmo.

L'approccio black-box, imponendo la sovrapposizione uniforme iniziale, impedisce semplificazioni dell'oracolo fondate su una preparazione differente dello stato.

La struttura rigida descritta viene superata grazie alla generalizzazione dell'algoritmo,

chiamata *quantum amplitude amplification* (QAA) e discussa in [6], che, rendendo modulare il metodo di amplificazione dell'ampiezza, consente un'accelerazione quadratica applicabile a una vasta gamma di algoritmi quantistici, con circa $1/\sqrt{p}$ iterazioni, dove p è la probabilità di successo iniziale del problema. In particolare, QAA rende la progettazione dell'oracolo strettamente connessa al contesto applicativo grazie a una preparazione iniziale arbitraria A e a un diffusore generalizzato, che applica un'inversione rispetto alla media ponderata associata ad A .

- La conoscenza a priori del numero di soluzioni: l'algoritmo non è pensato per i casi applicativi in cui non è noto il numero di soluzioni e, in generale, non convergerebbe a causa della sovrarotazione dello stato.

Tale problema è stato superato attraverso tre approcci differenti: BBHT, *quantum counting* e *fixed-point quantum search* (FPQS), discussi rispettivamente in [7], [8] e [9].

BBHT accetta il rischio di sovrarotazione, eseguendo più volte l'algoritmo con numero crescente di iterazioni finché la misura non corrisponde a una soluzione mediante verifica classica. Quantum counting previene il rischio di sovrarotazione, calcolando esplicitamente il numero di soluzioni. FPQS elimina il rischio di sovrarotazione, adottando una specifica sequenza di angoli di rotazione.

Questi approcci possono essere integrati nel paradigma QAA, a seconda delle specifiche esigenze algoritmiche e circuitali. Di particolare interesse per l'approssimazione statistica è la generalizzazione del quantum counting con QAA, chiamata *quantum amplitude estimation* (QAE) e discussa in [6], che consente di stimare il modulo dell'ampiezza della probabilità di successo iniziale di un problema ($\sqrt{p}$).

Capitolo 3

Il framework QLPP

3.1 Fondamenti formali

Per la trattazione della programmazione logica quantistica e del relativo framework QLPP, è necessario introdurre i fondamenti formali che descrivono la logica quantistica, discussa dettagliatamente in [3] e ripresa in [1] per la progettazione del framework QLPP.

Di seguito, vengono introdotte le strutture logico-algebriche classiche, per poi illustrare quelle quantistiche, evidenziandone similitudini e differenze:

- Logica classica: basata sull'algebra booleana, in cui ciascuna proposizione corrisponde a un sottoinsieme di un insieme di possibilità Ω e, per ciascun elemento di Ω , è univocamente vera se l'elemento appartiene a tale sottoinsieme o falsa se non vi appartiene (principio di bivalenza).

Dati due insiemi X e Y , le operazioni di negazione, congiunzione e disgiunzione sono definite come segue:

- negazione: complemento di X in Y

$$Y \setminus X = Y - X, \quad X \subseteq Y; \quad (3.1)$$

- congiunzione: intersezione di X e Y

$$X \wedge Y = X \cap Y; \quad (3.2)$$

- disgiunzione: unione di X e Y

$$X \vee Y = X \cup Y. \quad (3.3)$$

Come diretta implicazione delle operazioni di intersezione e unione, l'algebra booleana presenta la proprietà distributiva. La realizzazione fisica delle operazioni di negazione, congiunzione e disgiunzione avviene rispettivamente con le porte *NOT*, *AND* e *OR*.

La semantica classica è composizionale (dalle parti al tutto), ovvero il significato di

ogni parte di una formula è autonomo dal contesto in cui è inserita e il significato globale è ricostruito dalle sue parti.

- Logica quantistica: basata sull'algebra ortomodulare, in cui ciascuna proposizione corrisponde a un sottospazio chiuso di uno spazio di Hilbert H ed è, in generale, indeterminata rispetto a ogni elemento di tale sottospazio (principio di sovrapposizione). Dati due sottospazi chiusi X e Y , è possibile definire le operazioni di negazione, congiunzione e disgiunzione come segue:

- negazione: complemento ortogonale (o ortocomplemento) di X , ovvero il sottospazio X' di tutti i vettori ortogonali a X

$$\neg X = X', \quad X' = \{x' \in H : \langle x', x \rangle = 0, \forall x \in X\}; \quad (3.4)$$

- congiunzione: intersezione di X e Y , analogamente alla logica classica,

$$X \wedge Y = X \sqcap Y = X \cap Y; \quad (3.5)$$

- disgiunzione: sottospazio chiuso più piccolo contenente tutte le combinazioni lineari di X e Y , definito, per la legge di De Morgan, come l'ortocomplemento dell'intersezione di $\neg X$ e $\neg Y$

$$X \vee Y = X \sqcup Y = \neg(\neg X \sqcap \neg Y). \quad (3.6)$$

L'insieme di tutti i sottospazi chiusi di H con tali operazioni prende il nome di reticolo ortomodulare; tale struttura algebrica non presenta la proprietà distributiva, ma una condizione più debole, detta proprietà ortomodulare, definita da

$$Y = X \sqcup (Y \sqcap \neg X), \quad X \sqsubseteq Y, \quad (3.7)$$

in base alla quale Y può essere scomposto in X e l'intersezione di Y e $\neg X$.

La realizzazione fisica corrispondente alle operazioni definite è la seguente:

- $\neg$:= porta *NOT*
- $\alpha \wedge \beta$:= $\top(\alpha, \beta, f)$
- $\alpha \vee \beta$:= $\neg(\neg\alpha \wedge \neg\beta)$
- $\oplus$:= porta *CNOT*

dove α, β sono due formule generiche, $\top$ indica la porta Toffoli e f indica il valore logico *false*, corrispondente ad un qubit in stato $|0\rangle$.

La semantica quantistica è olistica (dal tutto alle parti), ovvero il significato globale di una formula è dato dallo stato associato alla formula stessa e il significato di una parte

può dipendere dal contesto in cui è inserita. La struttura di una generica formula α può essere espressa mediante un albero sintattico definito come

$$STree^\alpha = \left(Level_1^\alpha, Level_2^\alpha, \dots, Level_h^\alpha \right), \quad (3.8)$$

$$Level_i^\alpha = (\beta_{i1}, \beta_{i2}, \dots, \beta_{ir_i}), \quad \forall i \in \{1, 2, \dots, h\}, \quad (3.9)$$

ovvero una sequenza di h livelli in cui il generico elemento $Level_i^\alpha$ indica una sequenza di r_i sottoformule $(\beta_{i1}, \beta_{i2}, \dots, \beta_{ir_i})$. Il livello successivo $Level_{i+1}^\alpha$ si ottiene rimuovendo il connettivo logico principale di ciascuna sottoformula; il livello più basso $Level_1^\alpha$ corrisponde ad (α) , il livello più alto $Level_h^\alpha$ alla sequenza degli atomi di α ($q_1, q_2, \dots, q_k$). Come esempio, l'albero sintattico per la formula $\top(\neg q_1, \neg q_2, f)$ è dato dalla sequenza di livelli $(\top(\neg q_1, \neg q_2, f)) \rightarrow (\neg q_1, \neg q_2, f) \rightarrow (q_1, q_2, f)$.

Si definisce una mappa olistica Hol per una formula α l'associazione di un significato $Hol(Level_i^\alpha)$ per ogni $Level_i^\alpha$ e precisamente

$$Hol\left(Level_i^\alpha\right) = \rho_i, \quad (3.10)$$

dove ρ_i è un operatore di densità che rappresenta lo stato globale per il livello $Level_i^\alpha$. La mappa olistica consente di determinare il significato contestuale di una sottoformula β_{ij} nel modo seguente:

$$Hol^\alpha\left(\beta_{ij}\right) = Red_j(\rho_i), \quad (3.11)$$

ovvero la riduzione dello stato ρ_i rispetto al suo j -esimo sottosistema.

La riduzione $Red_j(\rho_i)$ è ottenuta da

$$Red_j(\rho_i) = PTr(\rho_{iS_1} \otimes \rho_{iS_2} \otimes \dots \otimes \rho_{iS_{r_i}}) = \rho_{iS_j} \prod_{\substack{k=1 \\ k \neq j}}^{r_i} Tr(\rho_{iS_k}), \quad (3.12)$$

in cui PTr effettua la traccia parziale dei sottosistemi S_k , con $k \neq j$, corrispondente a una marginalizzazione a S_j ; il generico $Tr(\rho_{iS_k})$ restituisce la traccia del sottosistema S_k , ovvero uno scalare che ne rappresenta il peso statistico ($0 \leq Tr(\rho_{iS_k}) \leq 1$).

In generale, date due formule α e γ e una sottoformula β , vale che

$$Hol^\alpha(\beta) \neq Hol^\gamma(\beta), \quad (3.13)$$

ovvero il significato della sottoformula β è dipendente dal contesto in cui è inserita.

3.2 La programmazione logica quantistica

La progettazione dei circuiti quantistici avviene tipicamente in ambienti di sviluppo, come Qiskit, in cui viene effettuata una descrizione procedurale a livello delle porte circuitali. Questo approccio, tuttavia, presenta i suoi limiti al crescere della complessità dei circuiti.

Per tale ragione, la ricerca si è orientata verso strumenti di alto livello che consentano l'astrazione dei circuiti quantistici e di conseguenza una gestione più efficace degli stessi.

In tale contesto, si inserisce la programmazione logica, che descrive i problemi a partire da una base di conoscenza costituita da dati, relazioni e regole (logica del primo ordine), che un motore di inferenza utilizza per la risoluzione automatica del problema. Questo paradigma nell'ambito del calcolo quantistico prende il nome di programmazione logica quantistica (*Quantum Logic Programming*, QLP); grazie all'approccio QLP, si passa dunque da una progettazione bottom-up a una top-down dei circuiti quantistici.

La metodologia adottata nel presente lavoro di tesi si inserisce in tale paradigma mediante il framework di programmazione logica quantistica in Prolog (*Quantum Logic Programming in Prolog*, QLPP), introdotto nel lavoro discusso in [1], per la sintesi automatica di circuiti quantistici a partire dal formalismo della logica quantistica e dalla relativa semantica olistica, descritti nel paragrafo precedente.

Il framework QLPP è illustrato nelle successive sezioni, distinguendo tra l'architettura di base, relativa al lavoro preesistente, e l'estensione progettuale, relativa al contributo sperimentale; il file di riferimento per l'architettura di base è `quantumV6.pl`, mentre il file aggiornato con l'estensione progettuale è `quantumV7.pl`.

3.3 Architettura di base

Il framework QLPP si sviluppa su una base di conoscenza mediante la quale il motore logico è capace di elaborare una query e generare il circuito desiderato. In particolare, come mostrato in Figura 3.1, la base di conoscenza si articola nei seguenti componenti fondamentali:

1. Definizione degli operatori: configurazione dei simboli, delle regole sintattiche e del significato fisico degli operatori logici quantistici.
2. Astrazione dell'hardware: istanza virtuale dei registri di qubit e bit del circuito.
3. Compilazione: processo di traduzione degli operatori logici nella corrispondente implementazione circuitale. Possono essere distinte due tipologie di input a cui conseguono scenari differenti di compilazione:
 - a) Espressioni: costrutti composti da uno o più operatori algebrici che agiscono su uno stesso target del registro quantistico; per un dato set di espressioni, viene effettuata la compilazione secondo la seguente pipeline standard:
 - (i) Sintesi logica: consiste nella generazione degli alberi sintattici, ciascuno dei quali associato a un'espressione, con mapping degli operatori logici nelle corrispondenti porte fisiche.
 - (ii) Layering: consiste nella generazione di un layer circuitale con scheduling di esecuzione delle porte, tramite linearizzazione degli alberi sintattici.

- b) Operatori speciali: costrutti puramente atomici che agiscono su più di un target del registro quantistico o che coinvolgono il registro classico (misurazione); per un dato operatore speciale, viene effettuata una compilazione per espansione nelle corrispondenti porte fisiche, che vengono iniettate in un nuovo layer circuitale.

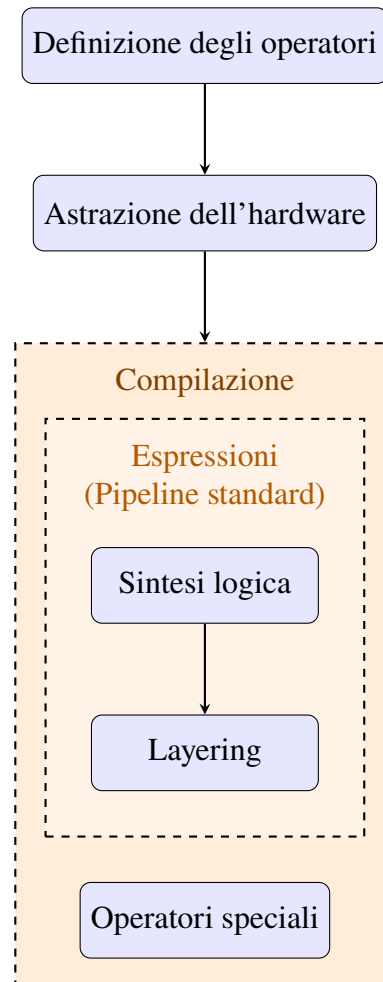


Figura 3.1: Architettura di base del framework QLPP.

3.3.1 Definizione degli operatori

Per adattare la programmazione logica al calcolo quantistico, è necessaria la definizione di un linguaggio di dominio specifico (*Domain-Specific Language*, DSL) per gli operatori logici quantistici. Prolog consente nativamente di estendere il proprio parser a un DSL mediante la direttiva `op/3`, definita come:

$$\text{op}(\text{Precedenza}, \text{Tipo}, \text{Nome})$$

con

- **Precedenza:** valore intero nell'intervallo $[0, 1200]$ che stabilisce la priorità algebrica dell'operatore (più basso il valore, più alta la precedenza);

- Tipo: letterale che stabilisce le proprietà dell'operatore, ovvero l'arità (unario o binario), la notazione posizionale (prefisso, infisso o postfisso) e l'associatività (sinistra o destra), e in particolare:
 - il simbolo *f* indica la posizione dell'operatore rispetto agli operandi;
 - i simboli *x* e *y* indicano un operando che, rispettivamente, esclude o ammette la presenza di operatori con la stessa priorità (valore di precedenza);
- Nome: uno o più letterali che identificano gli operatori con precedenza e tipo specificati.

Come mostrato nel Codice 3.1, gli operatori definiti si distinguono in unari prefissi (*fy*) e binari infissi con associatività sinistra (*yfx*); gli operatori unari hanno priorità maggiore rispetto ai binari e tra gli operatori binari il *qAND* ha priorità maggiore rispetto agli altri.

```
:- op(100, fy, [qH, qNOT, qRNOT, qZ, qY, qS, qT]).
:- op(120, yfx, qAND).
:- op(150, yfx, [qOR, qXOR, qCZ, qCY, qCS]).
```

Codice 3.1: Estensione del parser Prolog.

Gli operatori quantistici definiti sono algebrici, nonché processabili dal parser, e sono gli elementi costitutivi delle espressioni della pipeline standard di compilazione.

Per la validazione degli operatori si definisce la seguente clausola *q_operator/1* che utilizza il predicato nativo *member/2*:

```
q_operator(Operatore) :- member(Operatore, Operatori_Validi).
```

Come mostrato nel Codice 3.2, la lista degli operatori validi include anche quelli non processabili dal parser, ovvero gli operatori speciali quantistici *qSWAP* e *qCSWAP*.

```
q_operator(X) :- member(X, [qH, qNOT, qRNOT, qZ, qY, qS, qT, qXOR, qCZ, qCY, qCS,
                             qSWAP, qAND, qCSWAP]).
```

Codice 3.2: Validazione degli operatori.

Infine, viene associata una semantica a ciascuno degli operatori, ovvero una porta fisica con specifica del numero di qubit richiesti; a tal fine, si utilizza la clausola *gate/3* seguente:

```
gate(Operatore, Porta, Numero_Qubit).
```

Il Codice 3.3 stabilisce il mapping tra gli operatori e le porte fisiche.

```
gate(qH, h, 1).
gate(qNOT, not, 1).
gate(qRNOT, rnot, 1).
```

```

gate(qZ, pauli_Z, 1).
gate(qY, pauli_Y, 1).
gate(qS, phase, 1).
gate(qT, pi_8, 1).
gate(qXOR, cnot, 2).
gate(qCZ, c_pauli_Z, 2).
gate(qCY, c_pauli_Y, 2).
gate(qCS, c_phase, 2).
gate(qSWAP, swap, 2).
gate(qAND, tof, 3).
gate(qCSWAP, fredkin, 3).

```

Codice 3.3: Mapping tra operatori e porte fisiche.

3.3.2 Astrazione dell'hardware

Per la rappresentazione di alto livello del registro quantistico, viene impiegato un contatore di qubit gestito mediante l'insieme di clausole seguenti, riportate nel Codice 3.4:

- `asserta/1`: predicato nativo che consente la definizione dinamica di `qnn/1` che conserva una costante numerica, ovvero il valore del contatore;
- `retract/1`: predicato nativo che consente la rimozione dinamica di `qnn/1`;
- `qbit_number/1`: incremento del contatore e indicizzazione del qubit aggiunto;
- `q_value/1`: come prima, con verifica che l'argomento sia una variabile non inizializzata (variabile libera), e ritorno del valore default di fail in caso di errore;
- `reset_qbit_number/1`: reset del contatore di qubit.

```

% init
:- asserta(qnn(0)).

% incremento e indicizzazione
qbit_number(N) :-
    retract(qnn(N)),
    Np1 is N + 1, !,
    asserta(qnn(Np1)), !.
qvalue(N) :- var(N), qbit_number(N), !.
qvalue(_).

% reset
reset_qbit_number :-
    retract(qnn(_)),
    asserta(qnn(0)).

```

Codice 3.4: Contatore di qubit.

L'istanza del registro di qubit viene effettuata mediante la seguente clausola `make_qbits/2`:

`make_qbits(Dimensione, Registro_Qubit)`

con

- `Dimensione`: argomento per l'input del numero di qubit del registro;
- `Registro_Qubit`: argomento per l'output del registro di qubit, rappresentato come una lista di coppie chiave-valore, in cui la chiave è un qubit indicizzato `q/1` e il valore, inizializzato di default al qubit stesso, è un'espressione.

Il Codice 3.5 mostra l'implementazione di `make_qbits/2`, che impiega il contatore di qubit precedentemente illustrato.

```
make_qbits(0, []).
make_qbits(N, [q(M):q(M) | RRest]) :-
    N > 0,
    Nm1 is N - 1,
    qbit_number(M),
    make_qbits(Nm1, RRest).
```

Codice 3.5: Istanza del registro di qubit.

Per la rappresentazione di alto livello del registro classico, sono utilizzate le clausole `cbit_number/1`, `c_value/1`, `reset_cbit_number/1`, `make_cbits/2` e `c/1`, in modo del tutto analogo a quanto visto per la gestione del registro quantistico.

3.3.3 Compilazione

Il processo di compilazione prevede il caricamento dell'input (lista di coppie chiave-valore) nel registro di qubit/bit mediante la seguente clausola:

`fill_register(Registro_Iniziale, Input, Registro_Finale).`

Come mostrato nel Codice 3.6, `fill_register/3` verifica la validità delle chiavi (`q/1` o `c/1`) e inserisce nel registro i valori associati ad esse, lasciando invariati gli altri; nel caso degli operatori speciali, come vedremo in seguito, il caricamento è preceduto dall'espansione dell'operatore e corrisponde già all'output di compilazione.

```
fill_register(Reg, [], Reg).
fill_register([], _, []).
fill_register([Bit:_ | Rin], [Bit:Exp | Rest], [Bit:Exp | Rout]) :-
    member(Bit, [q(_), c(_)]), !,
    fill_register(Rin, Rest, Rout).
fill_register([BitK:Val | Rin], [Bit:Exp | Rest], [BitK:Val | Rout]) :-
```

```

BitK \= Bit,
fill_register(Rin,[Bit:Exp|Rest],Rout).

```

Codice 3.6: Caricamento nel registro.

La pipeline standard di compilazione (sintesi logica e layering) viene avviata globalmente dalla seguente clausola:

```

transform(Registro, Circuito).

```

Dato in input un registro al quale sono state associate delle espressioni con `fill_register/3`, `transform/2` restituisce in output il circuito corrispondente. Al suo interno, per ciascun qubit del registro, viene chiamata la seguente clausola:

```

qcircuit(Espressione, Qubit_Target, Sottocircuito).

```

Data un'espressione e il qubit ad essa associato, `qcircuit/3` si occupa della sintesi logica e del layering, chiamando rispettivamente le clausole `circuit_tree/3` e `layers/2`, e restituisce il sottocircuito risultante.

Il Codice 3.7 mostra l'implementazione di `transform/2` e `qcircuit/3`.

```

transform([],[]).
transform([q(K):Exp|Rin],[q(K):Circ|Rout]) :- !,
    qcircuit(Exp,q(K),Circ),
    transform(Rin,Rout).
qcircuit(Formula,Qbit,Layers) :-
    circuit_tree(Formula,Qbit,Tree),
    layers(Tree,Layers).

```

Codice 3.7: Pipeline standard di compilazione.

La clausola per la fase di sintesi logica è la seguente:

```

circuit_tree(Espressione, Qubit_Target, Albero).

```

A partire da un'espressione associata a un qubit, `circuit_tree/3` costruisce il corrispondente albero sintattico. In particolare, al suo interno vengono chiamate `normalize/2` e `translate/3`.

`normalize/2` si presenta nella seguente forma:

```

normalize(Espressione, Espressione_Std).

```

Tale clausola consiste nel pattern matching di un'espressione, con scomposizione e validazione secondo la struttura delle operazioni unarie e binarie e la legge di De Morgan.

translate/2 si presenta nella seguente forma:

```
translate([Espressione_Std, Qubit_Target, Qubit_Richiesti], Albero).
```

Tale clausola consiste nella traduzione dell'espressione normalizzata in un albero sintattico (rappresentato mediante liste annidate), che, data la lista di qubit richiesti nelle operazioni, stabilisce le dipendenze logiche per un qubit target, con mapping di ciascun operatore alla porta corrispondente; in particolare, la traduzione delle operazioni logiche viene distinta per porte a singolo qubit, a due qubit e Toffoli.

Il Codice 3.8 mostra l'implementazione della fase di sintesi logica.

```
% ***** sintesi logica *****
%
%
circuit_tree(Formula,Qbit,Circuit_tree) :-
    normalize(Formula,Norm_formula), !,
    translate([Norm_formula,Qbit,_],Circuit_tree).
%
%
% *** normalizzazione ***
%
% caso base
normalize(q(N),q(N)) :- !.
%
% legge di De Morgan
normalize(Arg1_in qOR Arg2_in,qNOT(qNOT Arg1_out qAND qNOT Arg2_out))
:- !,
    normalize(Arg1_in,Arg1_out),
    normalize(Arg2_in,Arg2_out).
%
% operazioni unarie
normalize(Formula_in,Formula_out):-
    Formula_in =.. [Op,Arg_in],
    normalize(Arg_in,Arg_out),
    Formula_out =.. [Op,Arg_out].
%
% operazioni binarie
normalize(Formula_in,Formula_out) :-
    Formula_in =.. [Op,Arg1_in,Arg2_in],
    normalize(Arg1_in,Arg1_out),
    normalize(Arg2_in,Arg2_out),
    Formula_out =.. [Op,Arg1_out,Arg2_out].
%
% *** traduzione ***
%
% caso base
translate([q(N),q(N),[q(N)|_]], [q(N)]).
```

```

%
% porte a un qubit
translate([Formula,q(N),Qbits],[Layer_gate,Left]) :-
    Formula =.. [Op,LF],
    last(Qbits,q(N)),
    q_operator(Op),
    gate(Op,Gate,1),
    Layer_gate =.. [Gate,q(N1)],
    translate([LF,q(N1),Qbits],Left).

%
% porta Toffoli
translate([LF qAND RF,q(N3),[q(N1),q(N2),q(N3)|_]],
    [tof(q(N1),q(N2),q(N3)),Left,Right]) :-
    translate([LF,q(N1),[q(N1)|_]],Left),
    translate([RF,q(N2),[q(N2)|_]],Right).

%
% porte a due qubit
translate([Formula,q(N2),[q(N1),q(N2)|_]], [Layer_gate,Left,Right]) :-
    Formula =.. [Op,LF,RF],
    q_operator(Op),
    gate(Op,Gate,2),
    translate([LF,q(N1),QbitsL],Left),
    translate([RF,q(N2),QbitsR],Right),
    append(_,[q(N1)],QbitsL),
    append(_,[q(N2)],QbitsR),
    Layer_gate =.. [Gate,q(N1),q(N2)].

```

Codice 3.8: Fase di sintesi logica.

La clausola per la fase di layering è la seguente:

```
layers(Albero, Layers) :- all_layers([Albero], Layers).
```

`layers/2` linearizza l'albero sintattico in un insieme di layer (rappresentato mediante lista di liste), che stabilisce lo scheduling di esecuzione circuitale relativo a un qubit target; in particolare, all'interno di `all_layers/2` vengono chiamate `level_gates/2` e `subtrees/2`. `level_gates/2` si presenta nella seguente forma:

```
level_gates(Livello, Porte).
```

Tale clausola estrae ogni porta da un determinato livello dell'albero sintattico. `subtrees/2` si presenta nella seguente forma:

```
subtrees(Livello, Sottoalberi).
```

Tale clausola estrae ogni sottoalbero da un determinato livello dell'albero sintattico per l'elaborazione del livello successivo.

Il Codice 3.9 mostra l'implementazione della fase di layering.

```
% ***** layering *****
%
%
layers(Tree,Levels) :- all_layers([Tree],Levels).
all_layers([],[]).
all_layers(Tree_list,[Gates|Other_levels]) :-
    level_gates(Tree_list,Gates),
    subtrees(Tree_list,Subtrees),
    all_layers(Subtrees,Other_levels).
%
%
% *** estrazione delle porte ***
%
level_gates([],[]).
level_gates([[Gate|_] | Rest_trees],[Gate|Rest_gates]) :-
    level_gates(Rest_trees,Rest_gates).
%
% *** estrazione dei sottoalberi ***
%
subtrees([],[]).
subtrees([[_|Subs] | Rest_trees],Subtrees) :-
    subtrees(Rest_trees,Other_subs),
    append(Subs,Other_subs,Subtrees).
```

Codice 3.9: Fase di layering.

È importante evidenziare che il layering non è da intendersi come fisico, ma temporale. L'output di compilazione è un singolo layer circuitale, le cui porte sono disposte secondo l'ordine stabilito nella fase di layering per ciascun qubit target del registro. Infine, il processo di compilazione prevede una semplificazione post-processing mediante la seguente clausola:

filter(Circuito, Circuito_Filtrato).

Come mostrato nel Codice 3.10, *filter/2* rimuove l'inizializzazione circuitale fittizia con valori di identità e i wrapper ridondanti; la compilazione di operatori speciali non prevede questo passaggio per l'assenza di possibili semplificazioni.

```
filter([],[]).
filter([q(N):q(N) | Rest],[q(N):q(N) | FRest]) :- !,
    filter(Rest,FRest).
filter([q(N):[[q(N)]] | Rest],[q(N):q(N) | FRest]) :- !,
    filter(Rest,FRest).
filter([q(N):Circ | Rest],[q(N):FCirc | FRest]) :-
    reverse(Circ,[_ | FCirc]),
```

```
filter(Rest, FRest).
```

Codice 3.10: Semplificazione post-processing.

La compilazione di operatori speciali viene avviata globalmente dalla clausola `apply_operator/4` per gli operatori quantistici (come *SWAP* e Fredkin) o dalla clausola `apply_operator/5` per l'operatore di misurazione; tali clausole si presentano rispettivamente nelle seguenti forme:

```
apply_operator(Operatore, Qubit_Richiesti,
               Registro_Qubit_Iniziale,
               Registro_Qubit_Finale),

apply_operator(measurement, [Qubit, Bit],
               Registro_Qubit_Iniziale,
               Registro_Bit_Iniziale,
               Registro_Unificato_Finale).
```

`measurement` è l'identificativo dell'operatore speciale di misurazione.

La compilazione di operatori speciali effettua un bypass del processo mediante un'espansione nella corrispondente lista di qubit e porte ad essi associate; dunque, il layer circuitale di output corrisponde all'input caricato nel registro.

All'interno di queste clausole, l'espansione avviene mediante la chiamata di `make_exps/2`, che si presenta nella forma seguente:

```
make_exps(Porta, Espansione).
```

Il Codice 3.11 mostra l'implementazione della compilazione di operatori speciali.

```
% ***** operatori speciali *****
%
%
apply_operator(QOP2, [Qbit1, Qbit2], RegIn, RegOut) :-
    q_operator(QOP2), gate(QOP2, Gate, 2),
    make_exps(Gate, [Qbit1:Exp1, Qbit2:Exp2]),
    fill_register(RegIn, [Qbit1:Exp1, Qbit2:Exp2], RegOut).
apply_operator(QOP3, [Qbit1, Qbit2, Qbit3], RegIn, RegOut) :-
    q_operator(QOP3), gate(QOP3, Gate, 3),
    make_exps(Gate, [Qbit1:Exp1, Qbit2:Exp2, Qbit3:Exp3]),
    fill_register(RegIn, [Qbit1:Exp1, Qbit2:Exp2, Qbit3:Exp3], RegOut).
apply_operator(measurement, [Qbit, Cbit], QRegister, CRegister, RegOut) :-
    append(QRegister, CRegister, RegIn),
    make_exps(measurement, [Qbit:Exp1, Cbit:Exp2]),
    fill_register(RegIn, [Qbit:Exp1, Cbit:Exp2], RegOut).
%
%
% *** espansione ***
```

```
%
make_exps(swap,[Qbit1:[swap(Qbit2,Qbit1)],Qbit2:[swap(Qbit1,Qbit2)]]).
make_exps(fredkin,[Qbit1:Qbit1,Qbit2:[cswap(Qbit1,Qbit3,Qbit2)],
        Qbit3:[cswap(Qbit1,Qbit2,Qbit3)]]).
make_exps(measurement,[Qbit:[measured(Cbit,Qbit)],
        Cbit:[measure(Qbit,Cbit)]]).
```

Codice 3.11: Operatori speciali.

Per la creazione del primo layer circuitale, si utilizzano le clausole `build_circuit/3` e `build_circuit/4`, rispettivamente per pipeline standard e operatore speciale quantistico:

```
build_circuit(Registro_Qubit, exp(Espressioni), Circuito),

build_circuit(Registro_Qubit, operator(Operatore),
        Qubit_Richiesti, Circuito).
```

`exp/1` e `operator/1` sono gli identificativi rispettivamente per l'input di espressioni e di operatore speciale quantistico.

Per l'aggiunta di un layer circuitale ad un circuito preesistente, si utilizzano le clausole `complete_circuit/4`, `complete_circuit/5` e `complete_circuit/6`, rispettivamente per pipeline standard, operatore speciale quantistico e operatore speciale di misurazione:

```
complete_circuit(Registro_Qubit, Circuito_Precedente,
        exp(Espressioni), Circuito),

complete_circuit(Registro_Qubit, Circuito_Precedente,
        operator(Operatore),
        Qubit_Richiesti, Circuito),

complete_circuit(Registro_Qubit, Registro_Bit,
        Circuito_Precedente, measurement,
        [Qubit, Bit], Circuito).
```

Il Codice 3.12 mostra l'implementazione delle clausole descritte per la creazione ed estensione di un circuito e il Codice 3.13 riporta esempi di query per i due scenari di compilazione. La `query1/1` realizza un circuito con 3 qubit e 2 layer utilizzando la pipeline standard di compilazione per gli operatori qNOT e qXOR, mentre la `query2/1` realizza un circuito con 2 qubit, 2 bit e 3 layer utilizzando la compilazione per espansione dell'operatore speciale quantistico qSWAP e dell'operatore speciale di misurazione `measurement`.

```
% ***** creazione ed estensione di un circuito *****
%
%
% *** costruzione del primo layer ***
%
```

```

% pipeline standard
build_circuit(QRegister,exp(Exp),[FCircuit]) :-
    fill_register(QRegister,Exp,QRfilled),
    transform(QRfilled,Circuit),
    filter(Circuit,FCircuit).

%
% operatore speciale quantistico
build_circuit(QRegister,operator(QOP),Qbits,[Circuit]) :-
    apply_operator(QOP,Qbits,QRegister,Circuit).

%
% *** aggiunta di un layer ***
%
% pipeline standard
complete_circuit(QRegister,PCircuit,exp(Exp),NCircuit) :-
    fill_register(QRegister,Exp,QRfilled),
    transform(QRfilled,Add_Circuit),
    filter(Add_Circuit,FAddCircuit),
    append(PCircuit,[FAddCircuit],NCircuit).

%
% operatore speciale quantistico
complete_circuit(QRegister,PCircuit,operator(QOP),Qbits,NCircuit) :-
    apply_operator(QOP,Qbits,QRegister,QRout),
    append(PCircuit,[QRout],NCircuit).

%
% operatore speciale di misurazione
complete_circuit(QRegister,CRegister,PCircuit,measurement,[Qbit,Cbit],
    NCircuit) :-
    apply_operator(measurement,[Qbit,Cbit],QRegister,CRegister,
        OutCircuit),
    append(PCircuit,[OutCircuit],NCircuit).

```

Codice 3.12: Creazione ed estensione di un circuito.

```

% ***** creazione ed estensione di un circuito - query *****
%
%
% pipeline standard
query1(Circuit1) :-
    reset_qbit_number,
    make_qbits(3,QRin),
    build_circuit(QRin,exp([q(0): qNOT(q(2) qXOR q(0))]),QRout),
    complete_circuit(QRin,QRout,exp([q(1): q(2) qXOR q(1)]),Circuit1).

%
% Circuit1 = [[q(0):[[cnot(q(2), q(0))], [not(q(0))]],
%              q(1):q(1),
%              q(2):q(2)],
%              [q(0):q(0),
%              q(1):[[cnot(q(2), q(1))]],

```

```

%           q(2):q(2)]]
%
%
% operatori speciali
query2(Circuit2) :-
    reset_qbit_number,
    reset_cbit_number,
    make_qbits(2,QRin),
    make_cbits(2,CRin),
    build_circuit(QRin,operator(qSWAP),[q(0),q(1)],QRout),
    complete_circuit(QRin,CRin,QRout,measurement,[q(0),c(0)],
                    PCircuit),
    complete_circuit(QRin,CRin,PCircuit,measurement,[q(1),c(1)],
                    Circuit2).

%
% Circuit2 = [[q(0):[swap(q(1),q(0))],
%               q(1):[swap(q(0),q(1))]],
%             [q(0):[measured(c(0),q(0))],
%               q(1):q(1),
%               c(0):[measure(q(0),c(0))],
%               c(1):c(1)],
%             [q(0):q(0),
%               q(1):[measured(c(1),q(1))],
%               c(0):c(0),
%               c(1):[measure(q(1),c(1))]]]

```

Codice 3.13: Creazione ed estensione di un circuito - Query.

3.4 Estensione progettuale

Le aggiunte e modifiche apportate a QLPP rispondono alla necessità di rendere il framework più flessibile e usabile e di risolvere alcuni limiti strutturali, come la mancata gestione di ancilla qubit per la catena di porte Toffoli. Si è quindi lavorato nella direzione di implementare clausole che avessero un livello di astrazione maggiore, non solo a livello di compilazione, ma anche e soprattutto a livello circuitale; in tal senso, l'aggiunta di operatori complessi, realizzati su più layer circuitali, ha reso il framework un motore logico maggiormente potente.

In particolare, l'estensione del framework include i seguenti punti:

- catena di porte Toffoli;
- cascata di porte a due qubit;
- funzionalità di supporto;
- altri operatori;
- interfaccia Python/Qiskit.

3.4.1 Catena di porte Toffoli

Considerando che una catena di porte Toffoli necessita di qubit ancilla come target dei risultati intermedi, la logica necessaria a implementarla non può avvenire a livello di compilazione, bensì a livello circuitale, costruendo un layer circuitale per ogni porta Toffoli della catena; a tal fine, sono state realizzate altre clausole `build_circuit` e `complete_circuit`, con identificativo `n-tof`, distinguendo una fase di inizializzazione e una di concatenazione. Inoltre, sono state previste due modalità operative: costruzione della coniugazione `compute-action-uncompute` con identificativo `conj` (default), e costruzione senza `uncompute` con identificativo `chain`.

Nella fase di inizializzazione, vengono specificati la lista dei qubit di controllo, una lista di qubit per ancilla e target finale e, opzionalmente, la modalità operativa (se non specificata, viene applicato il default); la dimensione delle liste di qubit è, inoltre, validata in modo che sia possibile la realizzazione della catena. Infine, viene costruita la prima porta Toffoli e viene isolato il sottocircuito inizializzato dal resto del circuito.

Il Codice 3.14 mostra l'implementazione della fase di inizializzazione.

```
% ***** catena di porte Toffoli - inizializzazione *****
%
%
build_circuit(QRegister,n-tof,CQbits,ATQbits,NCircuit) :-
    % default
    build_circuit(QRegister,n-tof,CQbits,ATQbits,conj,NCircuit).
%
build_circuit(_,n-tof,[q(_)],[],_,[]) :- !.
%
build_circuit(QRegister,n-tof,CQbits,ATQbits,Mode,NCircuit) :-
    % validazione
    length(CQbits,CQ_len), CQ_len > 1,
    length(ATQbits,ATQ_len), ATQ_len is CQ_len - 1,

    % costruzione porta Toffoli iniziale
    CQbits = [q(C1)|CQRest1],
    CQRest1 = [q(C2)|CQRest],
    ATQbits = [q(T)|_],
    build_circuit(QRegister,exp([q(T): q(C1) qAND q(C2)]),QRout),

    % fase di concatenazione
    complete_circuit(QRegister,[],QRout,n-tof,CQRest,ATQbits,Mode,
        NCircuit).
%
%
complete_circuit(QRegister,PCircuit,n-tof,CQbits,ATQbits,NCircuit) :-
    % default
    complete_circuit(QRegister,PCircuit,n-tof,CQbits,ATQbits,conj,
        NCircuit).
```

```

%
complete_circuit(_,PCircuit,n-tof,[q(_)],[],_,PCircuit) :- !.
%
complete_circuit(QRegister,PCircuit,n-tof,CQbits,ATQbits,Mode,
                  NCircuit) :-
    % validazione
    length(CQbits,CQ_len), CQ_len > 1,
    length(ATQbits,ATQ_len), ATQ_len is CQ_len - 1,

    % costruzione porta Toffoli iniziale
    CQbits = [q(C1)|CQRest1],
    CQRest1 = [q(C2)|CQRest],
    ATQbits = [q(T)|_],
    complete_circuit(QRegister,PCircuit,exp([q(T): q(C1) qAND q(C2)]),
                    NCircuit1),

    % isolamento sottocircuito
    append(PCircuit,TOF_PCircuit,NCircuit1),

    % fase di concatenazione
    complete_circuit(QRegister,PCircuit,TOF_PCircuit,n-tof,CQRest,
                    ATQbits,Mode,NCircuit).

```

Codice 3.14: Catena di porte Toffoli - Inizializzazione.

Nella fase di concatenazione, vengono costruite le successive porte Toffoli finché la lista dei qubit di controllo non è vuota. Nella modalità *conj* viene poi invertita la catena, eccetto la porta Toffoli sul target finale, ed effettuata la coniugazione *compute-action-uncompute*, aggiunta poi in coda al circuito. Nella modalità *chain* viene soltanto effettuata l'aggiunta della catena in coda al circuito.

Il Codice 3.15 mostra l'implementazione della fase di concatenazione.

```

% ***** catena di porte Toffoli - concatenazione *****
%
%
complete_circuit(_,PCircuit,TOF_PCircuit,n-tof,[],_,conj,NCircuit) :-
    % uncompute
    reverse(TOF_PCircuit,TOF_InvCircuit1),
    TOF_InvCircuit1 = [_|TOF_InvCircuit],

    % coniugazione compute-action-uncompute
    append(TOF_PCircuit,TOF_InvCircuit,TOF_Circuit),

    % aggiunta in coda al circuito
    append(PCircuit,TOF_Circuit,NCircuit).
%
complete_circuit(_,PCircuit,TOF_PCircuit,n-tof,[],_,chain,NCircuit) :-
    % aggiunta in coda al circuito

```

```

    append(PCircuit,TOF_PCircuit,NCircuit).
%
complete_circuit(QRegister,PCircuit,TOF_PCircuit,n-tof,CQbits,ATQbits,
    Mode,TOF_NCircuit) :-
    % costruzione porta Toffoli
    CQbits = [q(C)|CQRest],
    ATQbits = [q(A)|ATQRest],
    ATQRest = [q(T)|_],
    complete_circuit(QRegister,TOF_PCircuit,
        exp([q(T): q(C) qAND q(A)]),TOF_NCircuit1),

    complete_circuit(QRegister,PCircuit,TOF_NCircuit1,n-tof,CQRest,
        ATQRest,Mode,TOF_NCircuit).

```

Codice 3.15: Catena di porte Toffoli - Concatenazione.

Il Codice 3.16 riporta la query4/1 di esempio, che realizza il circuito di una catena Toffoli con 7 qubit.

```

% ***** catena di porte Toffoli - query *****
%
%
query3(Circuit3) :-
    reset_qbit_number,
    make_qbits(7,QRin),
    build_circuit(QRin,n-tof,[q(0),q(1),q(2),q(3)], [q(4),q(5),q(6)],
        Circuit3).
%
% Circuit3 = [[q(0):q(0),
%             q(1):q(1),
%             q(2):q(2),
%             q(3):q(3),
%             q(4):[[tof(q(0),q(1),q(4))]],
%             q(5):q(5),
%             q(6):q(6)],
%             [q(0):q(0),
%             q(1):q(1),
%             q(2):q(2),
%             q(3):q(3),
%             q(4):q(4),
%             q(5):[[tof(q(2),q(4),q(5))]],
%             q(6):q(6)],
%             [q(0):q(0),
%             q(1):q(1),
%             q(2):q(2),
%             q(3):q(3),
%             q(4):q(4),
%             q(5):q(5),
%             q(6):[[tof(q(3),q(5),q(6))]]],

```

```

%      [q(0):q(0),
%      q(1):q(1),
%      q(2):q(2),
%      q(3):q(3),
%      q(4):q(4),
%      q(5):[[tof(q(2),q(4),q(5))]],
%      q(6):q(6)],
%      [q(0):q(0),
%      q(1):q(1),
%      q(2):q(2),
%      q(3):q(3),
%      q(4):[[tof(q(0),q(1),q(4))]],
%      q(5):q(5),
%      q(6):q(6)]]

```

Codice 3.16: Catena di porte Toffoli - Query.

3.4.2 Cascata di porte a due qubit

Partendo dall'osservazione che le porte controllate a due qubit presentano il target come argomento destro, per simmetria si sono resi associativi destri gli operatori binari a cui corrispondono, modificando la relativa direttiva `op/3`; in tal modo, può essere tradotta un'espressione che contiene una cascata di operazioni binarie aventi lo stesso target. Inoltre, per il cambio di associatività, è stato necessario modificare la clausola `subtrees/2`, in modo che consideri a sinistra il nodo foglia nel caso di porte a due qubit interne all'albero.

Il Codice 3.17 mostra l'implementazione della cascata di porte a due qubit e il Codice 3.18 riporta la query3/1 di esempio, che realizza 1 layer circuitale con 4 qubit per la cascata di 3 operatori qXOR.

```

% ***** cascata di porte a due qubit *****
%
%
% *** estensione del parser Prolog ***
%
% operatori binari associativi a sinistra
:- op(120, yfx, qAND).
%
% operatori binari associativi a destra
:- op(150, xfy, [qXOR, qCZ, qCY, qCS]).
%
% operatori unari
:- op(100, fy, [qH, qNOT, qRNOT, qZ, qY, qS, qT]).
%
% *** estrazione dei sottoalberi ***
%
% caso base

```

```

subtrees([], []).
%
% porte a un qubit e due qubit
subtrees([Subs|Rest_trees], Subtrees) :-
    length(Subs, Subs_len), Subs_len < 3,
    Subs = [_|Rest_subs],
    subtrees(Rest_trees, Other_subs),
    append(Rest_subs, Other_subs, Subtrees).
%
% porta Toffoli
subtrees([Subs|Rest_trees], Subtrees) :-
    length(Subs, Subs_len), Subs_len = 3,
    Subs = [_|[_|Rest_subs]],
    subtrees(Rest_trees, Other_subs),
    append(Rest_subs, Other_subs, Subtrees).

```

Codice 3.17: Cascata di porte a due qubit.

```

% ***** cascata di porte a due qubit - query *****
%
%
query4(Circuit4) :-
    reset_qbit_number,
    make_qbits(4, QRin),
    build_circuit(QRin, exp([q(0): q(3) qXOR q(2) qXOR q(1)
                             qXOR q(0)]), Circuit4).
%
% Circuit4 = [[q(0):[cnot(q(1), q(0))],
%                [cnot(q(2), q(0))],
%                [cnot(q(3), q(0))]],
%             q(1):q(1),
%             q(2):q(2),
%             q(3):q(3)]

```

Codice 3.18: Cascata di porte a due qubit - Query.

3.4.3 Funzionalità di supporto

Le funzionalità di supporto integrate nel framework possono essere distinte in base a due obiettivi principali:

- usabilità;
- ispezione della sintesi logica.

L'usabilità è stata migliorata grazie a clausole per la selezione rapida di intervalli validi di qubit/bit, per la generazione di espressioni complesse e per la chiarezza dell'output.

Le clausole sono le seguenti:

- `qidx_check/1`: verifica l'esistenza di un qubit, controllando che il suo indice sia inferiore al valore attuale del contatore `qnn/1`;
- `cidx_check/1`: come sopra, ma per un bit rispetto al contatore `cnn/1`;
- `qrangle/3`, `qrangle/2`: estrazione di un intervallo di qubit, validato con `qidx_check/1` (se l'indice iniziale non è specificato, viene considerato 0 di default);
- `crangle/3`, `crangle/2`: come sopra, ma per un intervallo di bit, validato con `cidx_check/1`;
- `superpose/2`: superposizione (con `qH`) di una lista arbitraria di qubit;
- `negate/2`: negazione (con `qNOT`) di una lista arbitraria di qubit;
- `bw_qXOR/3`: bitwise `qXOR` (in-place) di due liste arbitrarie di qubit;
- `qXOR_1n/3`: `qXOR` "uno a molti" (un qubit di controllo, n qubit target);
- `qXOR_n1/2`: `qXOR` "molti a uno" (n qubit di controllo, un qubit target);
- `compress/2`: compressione dell'output con la rimozione delle identità.

Il Codice 3.19 mostra l'implementazione delle clausole per l'usabilità e il Codice 3.20 riporta la `query5/1` di esempio, che realizza 1 layer circuitale con 7 qubit, di cui i primi 4 vengono estratti con `qrangle/2` e messi in superposizione con `superpose/2` e gli altri 3 vengono trascurati per l'uso di `compress/2`.

```
% controllo dell'indice
%
qidx_check(QIDX) :- qnn(MAX), QIDX < MAX.
%
cidx_check(CIDX) :- cnn(MAX), CIDX < MAX.

% estrazione dell'intervallo
%
qrangle(N1,N2,Range) :- R is N2-1, qidx_check(R), findall(q(N),between(
    N1,R,N),Range).
qrangle(N2,Range) :- R is N2-1, qidx_check(R), findall(q(N),between(0,R
    ,N),Range).
%
crangle(N1,N2,Range) :- R is N2-1, cidx_check(R), findall(c(N),between(
    N1,R,N),Range).
crangle(N2,Range) :- R is N2-1, cidx_check(R), findall(c(N),between(0,R
    ,N),Range).

% superposizione di una lista di qubit
superpose([],[]) :- !.
superpose([q(N)|RestIn],[q(N):qH q(N)|RestOut]) :-
    superpose(RestIn,RestOut).
```

```

% negazione di una lista di qubit
negate([],[]) :- !.
negate([q(N)|RestIn],[q(N):qNOT q(N)|RestOut]) :-
    negate(RestIn,RestOut).

% bitwise qXOR
bw_qXOR([],[],[]) :- !.
bw_qXOR([Qbit1|QRest1],[Qbit2|QRest2],[Qbit2: Qbit1 qXOR Qbit2|RestOut
]) :-
    bw_qXOR(QRest1,QRest2,RestOut).

% qXOR 1:n
qXOR_1n(_,[],[]) :- !.
qXOR_1n(q(C),[q(T)|RestIn],[q(T): q(C) qXOR q(T)|RestOut]) :-
    qXOR_1n(q(C),RestIn,RestOut).

% qXOR n:1
qXOR_n1([X],X) :- !.
qXOR_n1([X|RestIn],X qXOR RestOut) :-
    qXOR_n1(RestIn,RestOut).

% compressione dell'output
compress([],[]) :- !.
compress([[SubExp|SubRest]|Rest],[FSubRest|FRest]) :- !,
    compress([SubExp|SubRest],FSubRest),
    compress(Rest,FRest).
compress([q(N):q(N)|Rest],FRest) :- !,
    compress(Rest,FRest).
compress([q(N):X|Rest],[q(N):X|FRest]) :-
    compress(Rest,FRest).

```

Codice 3.19: Strumenti per l'usabilità.

```

query5(Circuit5) :-
    reset_qbit_number,
    make_qbits(7,QRin),
    qrange(4,TQbits),
    superpose(TQbits,Exp),
    build_circuit(QRin,exp(Exp),QRout),
    compress(QRout,Circuit5).

% Circuit5 = [[q(0):[[h(q(0))]],
%             q(1):[[h(q(1))]],
%             q(2):[[h(q(2))]],
%             q(3):[[h(q(3))]]]]

```

Codice 3.20: Strumenti per l'usabilità - Query.

L'ispezione della sintesi logica è stata realizzata includendo la libreria debug e inserendo in `q_circuit/3` la clausola `circuit_tree_log/3`, che effettua il log delle traduzioni non banali da espressione ad albero sintattico; per abilitare il log si utilizza il comando "`debug(circuit_tree)`", per disabilitarlo "`nodebug(circuit_tree)`". Inoltre, la leggibilità di stampa degli operatori logici è stata migliorata mediante la clausola integrata in runtime `portray/1`. Il Codice 3.21 mostra l'implementazione delle clausole per l'ispezione della sintesi logica e il Codice 3.22 riporta la `query6/1` di esempio, che realizza 1 layer circuitale con 3 qubit, per la cascata di 1 qNOT e 2 qXOR, e ne stampa il log delle traduzioni.

```
% modulo debug
:- use_module(library(debug)).

% stampa del log
circuit_tree_log(Formula,Qbit,Tree) :-
    (Qbit = q(0) ->
        (write('Non-trivial formula -> tree translations:'),
         nl, write('{')) ;
        true),
    (Formula \= q(_) ->
        format('~n Target: ~w~n Formula: ~p~n Tree: ~w~n',
               [Qbit,Formula,Tree]) ;
        true),
    ((qnn(N), N1 is N-1, Qbit = q(N1)) ->
        (write('}'), nl, nl) ;
        true).

% integrazione del log alla pipeline
qcircuit(Formula,Qbit,Layers) :-
    circuit_tree(Formula,Qbit,Tree),
    debug(circuit_tree,'~@',[circuit_tree_log(Formula,Qbit,Tree)]),
    layers(Tree,Layers).

% formattazione degli operatori logici
portray(Formula) :-
    compound(Formula),
    Formula =.. [Op, Left, Right],
    current_op(_, Type, Op),
    member(Type, [xfx, xfy, yfx]), !,
    format('~p ~w ~p', [Left, Op, Right]).
portray(Formula) :-
    compound(Formula),
    Formula =.. [Op, Right],
    current_op(_, fy, Op), !,
    (Right \= q(_) ->
        (format('~w (~p)', [Op, Right])) ;
        format('~w ~p', [Op, Right])).
```

Codice 3.21: Log delle traduzioni.

```

query6(Circuit6) :-
    debug(circuit_tree),
    reset_qbit_number,
    make_qbits(3,QRin),
    build_circuit(QRin,exp([q(0): qNOT(q(2) qXOR q(1) qXOR q(0))]),
        Circuit6).

% % Non-trivial formula -> tree translations:
% {
%   Target:  q(0)
%   Formula: qNOT (q(2) qXOR q(1) qXOR q(0))
%   Tree:    [not(q(0)),
%             [cnot(q(2),q(0)),
%             [q(2)], [cnot(q(1),q(0)),
%             [q(1)], [q(0)]]]]]
%
% %
% % }
%
%
% Circuit6 = [[q(0):[[cnot(q(1),q(0))],[cnot(q(2),q(0))],[not(q(0))]],
%               q(1):q(1),
%               q(2):q(2)]]

```

Codice 3.22: Log delle traduzioni - Query.

Inoltre, è stato realizzato il modulo Python `qtree.py` (mostrato nel Codice 3.23), che effettua il parsing dell'albero sintattico QLPP e lo visualizza mediante l'utilizzo della libreria `Graphviz`, con possibilità di *relabeling* dei qubit.

```

# *** qtree.py ***
#
# imports
import graphviz
import uuid
import re
import ast
from IPython.display import HTML
#
# relabeling
def map_qtree(str,map):
    for old_label,new_label in map.items():
        str = str.replace(old_label,new_label)
    return str
#
# parsing
def parse_qtree(str):
    pattern = r"([a-zA-Z_]\w*\((?:\[^\(\)]|\([^\(\)]*\))*)\)"

```

```

        quoted_str = re.sub(pattern, r"'\\1'", str)
        return ast.literal_eval(quoted_str)

#
# rendering
def draw_qtree(tree, title='Quantum Tree'):
    dot = graphviz.Digraph()
    dot.attr(rankdir='TB')
    dot.attr('node', shape='box', style='rounded, filled',
             fillcolor='lightgrey')
    dot.attr(labelloc='t')
    dot.attr(fontname='Helvetica')
    dot.attr(label=f"""<
        <FONT POINT-SIZE="20"><B>{title}</B></FONT>
        <BR/>
        <FONT POINT-SIZE="14" COLOR="grey">{tree}</FONT>
        <BR/>
        >""")

def draw_qsubtree(subtree, parent_id=None):
    if not subtree:
        return

    if isinstance(subtree, list):
        current_val = subtree[0]
        children = subtree[1:]
    else:
        current_val = subtree
        children = []

    node_id = str(uuid.uuid4())
    label = str(current_val)

    if re.match(r"^[a-zA-Z_]+\(\d+\)$", label):
        color = 'white'
    else:
        color = 'lightblue'

    dot.node(node_id, label=label, fillcolor=color)

    if parent_id:
        dot.edge(parent_id, node_id)

    for child in children:
        draw_qsubtree(child, parent_id=node_id)

draw_qsubtree(tree)

svg_dot = dot.pipe(format='svg').decode('utf-8')

```

```
return HTML(f'<div style="width: 100%; height: 100%;  
display: flex; justify-content: center;"  
>{svg_dot}</div>')
```

Codice 3.23: Modulo di rendering dell'albero sintattico.

La Figura 3.2 mostra il diagramma Graphviz dell'albero sintattico della query6/1.

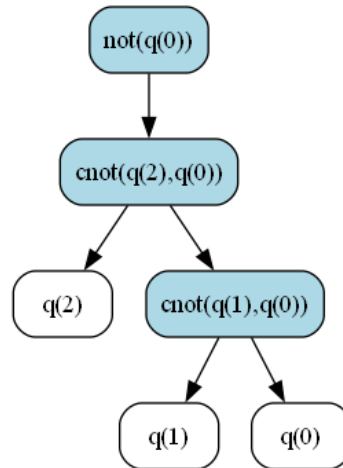


Figura 3.2: Diagramma Graphviz dell'albero sintattico.

3.4.4 Altri operatori

Utilizzando altre clausole `build_circuit` e `complete_circuit`, similmente a quanto visto per la catena di porte Toffoli, è possibile implementare operatori arbitrari, realizzati su più layer circuitali. In particolare, sono stati aggiunti gli operatori con i seguenti identificativi:

- **or**: operatore *OR* scomposto secondo la legge di De Morgan in negazione dei controlli, porta Toffoli e negazione sul target (Codice 3.24);
- **n-or**: generalizzazione dell'operatore *OR*, con l'utilizzo di `negate/2` e dell'operatore `n-tof` (Codice 3.25);
- **parity**: parity check di un numero arbitrario di qubit su ancilla con l'utilizzo di `qXOR_n1/2`, copia dell'ancilla su target e `uncompute` (Codice 3.26);
- **ghz**: entanglement GHZ di un numero arbitrario di qubit con l'utilizzo di `qXOR_1n/3` (Codice 3.27);
- **incr/decr**: incremento/decremento di un numero arbitrario di qubit con l'utilizzo dell'operatore `n-tof` per i flip a più di due controlli (Codice 3.28).

Inoltre, è stata integrata la misurazione multipla nell'operatore speciale `measurement`; in particolare, la compilazione per espansione avviene con l'utilizzo della clausola integrata

maplist/4, che, date una lista di qubit e una di bit, applica sulle due liste il mapping a coppie specificato da measured_exp/3 e da measure_exp/3 (Codice 3.29).

```
% ***** operatore OR *****
%
%
% legge di De Morgan
%  $Q1 \text{ OR } Q2 = \text{qNOT}(\text{qNOT } Q1 \text{ qAND } \text{qNOT } Q2)$ 
%
build_circuit(QRegister,or,[q(N1),q(N2),q(N3)],NCircuit) :- !,
    % compute di negazione dei controlli
    build_circuit(QRegister,exp([q(N1): qNOT q(N1),
                                q(N2): qNOT q(N2)]),[QRout]),

    % porta Toffoli e negazione sul target
    complete_circuit(QRegister,[QRout],
                    exp([q(N3): qNOT (q(N1) qAND q(N2))]),NCircuit1),

    % uncompute di negazione dei controlli
    reverse(QRout,Inv_QRout),
    append(NCircuit1,[Inv_QRout],NCircuit).
%
complete_circuit(QRegister,PCircuit,or,[q(N1),q(N2),q(N3)],
                NCircuit) :- !,
    % compute di negazione dei controlli
    build_circuit(QRegister,exp([q(N1): qNOT q(N1),
                                q(N2): qNOT q(N2)]),[QRout]),

    % porta Toffoli e negazione sul target
    complete_circuit(QRegister,[QRout],
                    exp([q(N3): qNOT (q(N1) qAND q(N2))]),NCircuit1),

    % uncompute di negazione dei controlli
    reverse(QRout,Inv_QRout),
    append(NCircuit1,[Inv_QRout],NCircuit2),

    % aggiunta in coda al circuito
    append(PCircuit,NCircuit2,NCircuit).
```

Codice 3.24: Operatore OR.

```
% ***** generalizzazione dell'operatore OR *****
%
%
% legge di De Morgan
%  $Q1 \text{ OR } Q2 \dots \text{OR } Qn = \text{qNOT}(\text{qNOT } Q1 \text{ qAND } \text{qNOT } Q2 \dots \text{qAND } \text{qNOT } Qn)$ 
%
build_circuit(QRegister,n-or,CQbits,ATQbits,NCircuit) :-
```

```

    % compute di negazione dei controlli
    negate(CQbits,Neg),
    build_circuit(QRegister,exp(Neg),[QRout]), !,

    % catena di porte Toffoli
    complete_circuit(QRegister,[QRout],n-tof,CQbits,ATQbits,conj,
                    NCircuit1),

    % negazione sul target
    last(ATQbits,q(T)),
    complete_circuit(QRegister,NCircuit1,exp([q(T): qNOT q(T)]),
                    NCircuit2),

    % uncompute di negazione dei controlli
    reverse(QRout,Inv_QRout),
    append(NCircuit2,[Inv_QRout],NCircuit).
%
complete_circuit(QRegister,PCircuit,n-or,CQbits,ATQbits,NCircuit) :-
    % compute di negazione dei controlli
    negate(CQbits,Neg),
    build_circuit(QRegister,exp(Neg),[QRout]), !,

    % catena di porte Toffoli
    complete_circuit(QRegister,[QRout],n-tof,CQbits,ATQbits,conj,
                    NCircuit1),

    % negazione sul target
    last(ATQbits,q(T)),
    complete_circuit(QRegister,NCircuit1,exp([q(T): qNOT q(T)]),
                    NCircuit2),

    % uncompute di negazione dei controlli
    reverse(QRout,Inv_QRout),
    append(NCircuit2,[Inv_QRout],NCircuit3),

    % aggiunta in coda al circuito
    append(PCircuit,NCircuit3,NCircuit).

```

Codice 3.25: Generalizzazione dell'operatore OR.

```

build_circuit(QRegister,parity,CQbits,[q(A),q(T)],NCircuit) :-
    % validazione
    length(CQbits,CQ_len), CQ_len > 1,

    % compute del parity check
    reverse(CQbits,Inv_CQbits),
    append(Inv_CQbits,[q(A)],Inv_CAQbits),
    qXOR_n1(Inv_CAQbits,Parity),

```

```

build_circuit(QRegister,exp([q(A): Parity]),PC_QRout),

% copia dell'ancilla su target
complete_circuit(QRegister,PC_QRout,exp([q(T): q(A) qXOR q(T)]),
    NCircuit1),

% uncompute del parity check
append(CQbits,[q(A)],CAQbits),
qXOR_n1(CAQbits,Inv_Parity),
complete_circuit(QRegister,NCircuit1,exp([q(A): Inv_Parity]),
    NCircuit).

%
complete_circuit(QRegister,PCircuit,parity,CQbits,[q(A),q(T)],
    NCircuit) :-

% validazione
length(CQbits,CQ_len), CQ_len > 1,

% compute del parity check in coda al circuito
reverse(CQbits,Inv_CQbits),
append(Inv_CQbits,[q(A)],Inv_CAQbits),
qXOR_n1(Inv_CAQbits,Parity),
complete_circuit(QRegister,PCircuit,exp([q(A): Parity]),
    NCircuit1),

% copia dell'ancilla su target
complete_circuit(QRegister,NCircuit1,exp([q(T): q(A) qXOR q(T)]),
    NCircuit2),

% uncompute del parity check in coda al circuito
append(CQbits,[q(A)],CAQbits),
qXOR_n1(CAQbits,Inv_Parity),
complete_circuit(QRegister,NCircuit2,exp([q(A): Inv_Parity]),
    NCircuit).

```

Codice 3.26: Parity check.

```

build_circuit(QRegister,ghz,Qbits,NCircuit) :-
% validazione
length(Qbits,Q_len), Q_len > 1,

% costruzione dell'entanglement GHZ
append([q(C)],TQbits,Qbits),
build_circuit(QRegister,exp([q(C): qH q(C)]),QRout),
qXOR_1n(q(C),TQbits,GHZ),
complete_circuit(QRegister,QRout,exp(GHZ),NCircuit).

%
complete_circuit(QRegister,PCircuit,ghz,Qbits,NCircuit) :-
% validazione

```

```

length(Qbits,Q_len), Q_len > 1,

% costruzione dell'entanglement GHZ in coda al circuito
append([q(C)],TQbits,Qbits),
complete_circuit(QRegister,PCircuit,exp([q(C): qH q(C)]),
NCircuit1),
qXOR_1n(q(C),TQbits,GHZ),
complete_circuit(QRegister,NCircuit1,exp(GHZ),NCircuit).

```

Codice 3.27: Entanglement GHZ.

```

build_circuit(QRegister,incr,q(C),[],q(T),NCircuit) :- !,
% flip a un controllo
build_circuit(QRegister,exp([q(T): q(C) qXOR q(T)]),NCircuit1),
complete_circuit(QRegister,NCircuit1,exp([q(C): qNOT q(C)]),
NCircuit).

%
build_circuit(QRegister,incr,[q(C1),q(C2)],[],q(T),NCircuit) :- !,
% flip a due controlli
build_circuit(QRegister,exp([q(T): q(C1) qAND q(C2)]),NCircuit1),

complete_circuit(QRegister,NCircuit1,incr,q(C1),_,q(C2),NCircuit).

%
build_circuit(QRegister,incr,CQbits,AQbits,q(T),NCircuit) :-
% flip multi-controllo
append(AQbits,[q(T)],ATQbits),
build_circuit(QRegister,n-tof,CQbits,ATQbits,NCircuit1),
append(CQRest,[q(T1)],CQbits),
append(AQRest,[q(_)],AQbits),

complete_circuit(QRegister,NCircuit1,incr,CQRest,AQRest,q(T1),
NCircuit).

%
build_circuit(QRegister,decr,CQbits,AQbits,TQbit,NCircuit) :-
% inversione della sequenza di flip
build_circuit(QRegister,incr,CQbits,AQbits,TQbit,NCircuit1),
reverse(NCircuit1,NCircuit).

%
%
complete_circuit(QRegister,PCircuit,incr,q(C),[],q(T),NCircuit) :- !,
% flip a un controllo in coda al circuito
complete_circuit(QRegister,PCircuit,exp([q(T): q(C) qXOR q(T)]),
NCircuit1),
complete_circuit(QRegister,NCircuit1,exp([q(C): qNOT q(C)]),
NCircuit).

%
complete_circuit(QRegister,PCircuit,incr,[q(C1),q(C2)],[],q(T),
NCircuit) :- !,

```

```

    % flip a due controlli in coda al circuito
    complete_circuit(QRegister,PCircuit,exp([q(T): q(C1) qAND q(C2)]),
        NCircuit1),
    complete_circuit(QRegister,NCircuit1,incr,q(C1),_,q(C2),NCircuit).
%
complete_circuit(QRegister,PCircuit,incr,CQbits,AQbits,q(T),
    NCircuit) :-
    % flip multi-controllo in coda al circuito
    append(AQbits,[q(T)],ATQbits),
    complete_circuit(QRegister,PCircuit,n-tof,CQbits,ATQbits,
        NCircuit1),
    append(CQRest,[q(T1)],CQbits),
    append(AQRest,[q(_)],AQbits),
    complete_circuit(QRegister,NCircuit1,incr,CQRest,AQRest,q(T1),
        NCircuit).
%
complete_circuit(QRegister,PCircuit,decr,CQbits,AQbits,TQbit,
    NCircuit) :-
    % inversione della sequenza di flip
    build_circuit(QRegister,incr,CQbits,AQbits,TQbit,NCircuit1),
    reverse(NCircuit1,NCircuit2),

    % aggiunta in coda al circuito
    append(PCircuit,NCircuit2,NCircuit).

```

Codice 3.28: Incremento e decremento.

```

apply_operator(measurement,[Qbits,Cbits],QRegister,CRegister,
    RegOut) :-
    append(QRegister,CRegister,RegIn),
    make_exps(measurement,[Qbits,Cbits],Exp),
    fill_register(RegIn,Exp,RegOut).
%
measured_exp(Qbit,Cbit,Qbit:measured(Cbit,Qbit)).
measure_exp(Qbit,Cbit,Cbit:measure(Qbit,Cbit)).
make_exps(measurement,[Qbits,Cbits],Exp) :-
    maplist(measured_exp,Qbits,Cbits,QExp),
    maplist(measure_exp,Qbits,Cbits,CExp),
    append(QExp,CExp,Exp).
%
complete_circuit(QRegister,CRegister,PCircuit,measurement,
    [Qbits,Cbits],NCircuit) :-
    length(Qbits,Q_len), length(Cbits,C_len), Q_len is C_len,
    apply_operator(measurement,[Qbits,Cbits],QRegister,CRegister,
        OutCircuit),
    append(PCircuit,[OutCircuit],NCircuit).

```

Codice 3.29: Misurazione multipla.

3.4.5 Interfaccia Python/Qiskit

L'interfaccia Python/Qiskit consiste delle seguenti fasi:

- query al motore logico QLPP con il modulo di terze parti PySwip;
- traduzione del circuito QLPP nel formato standard OpenQASM 3.0 con il modulo personalizzato `qlpp_to_qasm.py` (mostrato nel Codice 3.30);
- conversione del circuito OpenQASM 3.0 in oggetto `QuantumCircuit` con il modulo `qasm3` di Qiskit.

```
# *** qlpp_to_qasm.py ***
#
import re
#
def get_string(qlpp_circuit, barriers=False):

    # pattern regex di un layer QLPP
    qlpp_pattern = r'(h|not|rnot|pauli_Y|pauli_Z|phase|pi_8|cnot|
                    c_pauli_Y|c_pauli_Z|swap|tof|cswap|measure)
                    \s*(\s*(q\(\d+\)(?:\s*,\s*q\(\d+\))*)\s*)'

    # mapping delle porte QLPP -> QASM
    mapping = {

        # porte a singolo qubit
        'h': 'h',
        'not': 'x',
        'rnot': 'sx',
        'pauli_Y': 'y',
        'pauli_Z': 'z',
        'phase': 's',
        'pi_8': 't',

        # porte a 2 qubit
        'cnot': 'cx',
        'c_pauli_Y': 'cy',
        'c_pauli_Z': 'cz',
        'swap': 'swap',

        # porte a 3 qubit
        'tof': 'ccx', # rccx
        'cswap': 'cswap',

        # misurazione
        'measure': 'measure'

    }
```

```

# set di qubit e bit di QLPP
qlpp_reg_matches = set()

# lista di stringhe del corpo del codice QASM
qasm_body_str = []

# parsing dei layer QLPP
for qlpp_layer in qlpp_circuit:

    # stringa del layer QLPP
    qlpp_layer_str = str(qlpp_layer)

    # aggiornamento del set di qubit e bit QLPP
    qlpp_reg_matches.update(re.findall(r'([qc])\((\d+)\)',
                                       qlpp_layer_str))

    # porte QLPP
    qlpp_gates = re.findall(qlpp_pattern, qlpp_layer_str)

    # traduzione QLPP -> QASM
    if qlpp_gates:
        for qlpp_gate, qlpp_gate_args in qlpp_gates:
            qasm_gate = mapping.get(qlpp_gate)
            if qasm_gate:
                qlpp_gate_args_idx = re.findall(
                    r'\d+', qlpp_gate_args)
                if qasm_gate != 'measure':
                    qasm_gate_args = ", ".join(
                        [f"q[{i}]" for i in qlpp_gate_args_idx])
                else:
                    qasm_gate_args = f"q[{qlpp_gate_args_idx[0]}]
                    -> c[{qlpp_gate_args_idx[1]}]"
                qasm_body_str.append(f"{qasm_gate}
                                    {qasm_gate_args};")

            if barriers:
                qasm_body_str.append("barrier q;")
                qasm_body_str.append("")

# dimensione del registro di qubit
qubit_dim = max([int(qidx) for label, qidx in qlpp_reg_matches
                  if label == 'q'], default=-1) + 1

# dimensione del registro di bit
bit_dim = max([int(cidx) for label, cidx in qlpp_reg_matches
               if label == 'c'], default=-1) + 1

# caso nullo
if qubit_dim == 0:

```

```

    return ""

    # stringa del corpo di codice QASM
    qasm_body_str = "\n".join(qasm_body_str)

    # lista di stringhe per l'intestazione del codice QASM
    qasm_header_str = [
        "OPENQASM 3.0;",
        'include "stdgates.inc";',
        # "gate rccx ctrl1, ctrl2, tgt {
        #     u2(0, pi) tgt;u1(pi/4) tgt;cx ctrl2, tgt;u1(-pi/4) tgt;
        #     cx ctrl1, tgt;u1(pi/4) tgt;cx ctrl2, tgt;u1(-pi/4) tgt;
        #     u2(0, pi) tgt;}",
        f"qubit[{qubit_dim}] q;",
        ""
    ]
    if bit_dim != 0:
        qasm_header_str.append(f"bit[{bit_dim}] c;")
        qasm_header_str.append("")

    # stringa dell'intestazione del codice QASM
    qasm_header_str = "\n".join(qasm_header_str) + "\n"

    # stringa di codice QASM
    qasm_code = qasm_header_str + qasm_body_str

    return qasm_code

```

Codice 3.30: Modulo di traduzione da QLPP a QASM.

Il Codice 3.31 mostra l'implementazione dell'interfaccia Python/Qiskit e il Codice 3.32 riporta la `parity_query/1` di esempio, i cui corrispondenti circuiti QASM e Qiskit sono mostrati rispettivamente nel Codice 3.33 e in Figura 3.3.

```

# *** interfaccia python/qiskit ***
#
# moduli
import pyswip
import qlpp_to_qasm
from qiskit import qasm3
#
# query Prolog
def prolog_query (filename, query):
    prolog = pyswip.Prolog()
    prolog.consult(filename)
    solution = next(prolog.query(query), None)

```

```

    if solution is not None:
        return solution

    else:
        print("Nessuna soluzione trovata")
        return None

#
# risultato query QLPP
solution = prolog_query(filename="quantumV7.pl", query="parity_query(C
    )")
#
# circuito QLPP
qlpp_circuit = solution["C"]
#
# traduzione in formato OpenQASM 3.0
qasm_code = qlpp_to_qasm.get_string(qlpp_circuit=qlpp_circuit,
                                    barriers=True)
#
# conversione in oggetto QuantumCircuit
qc = qasm3.loads(qasm_code)
#
# salvataggio file qasm
with open("parity_check.qasm", "w", encoding="utf-8") as f:
    f.write(qasm_code)
#
# caricamento file qasm
# qc = qasm3.load("parity_check.qasm")

```

Codice 3.31: Interfaccia Python/Qiskit.

```

# *** interfaccia python/qiskit - query ***
#
# parity check query
parity_query(C) :-
    reset_qbit_number,
    make_qbits(6, QRin),
    qrange(4, CQbits), qrange(4, 6, ATQbits),
    build_circuit(QRin, parity, CQbits, ATQbits, C).
#
# C = [[q(0):q(0),
#       q(1):q(1),
#       q(2):q(2),
#       q(3):q(3),
#       q(4):[[cnot(q(0),q(4))],[cnot(q(1),q(4))],[cnot(q(2),q(4))],
#             [cnot(q(3),q(4))]],
#       q(5):q(5)],
#       [q(0):q(0),
#       q(1):q(1),

```

```

#      q(2):q(2),
#      q(3):q(3),
#      q(4):q(4),
#      q(5):[[cnot(q(4),q(5))]],
#      [q(0):q(0),
#      q(1):q(1),
#      q(2):q(2),
#      q(3):q(3),
#      q(4):[[cnot(q(3),q(4))],[cnot(q(2),q(4))],[cnot(q(1),q(4))],
#      [cnot(q(0),q(4))]],
#      q(5):q(5)]]

```

Codice 3.32: Interfaccia Python/Qiskit - Query.

```

# *** interfaccia python/qiskit - circuito qasm ***
#
# parity_check.qasm
#
OPENQASM 3.0;
include "stdgates.inc";
qubit[6] q;
#
cx q[0], q[4];
cx q[1], q[4];
cx q[2], q[4];
cx q[3], q[4];
barrier q;
#
cx q[4], q[5];
barrier q;
#
cx q[3], q[4];
cx q[2], q[4];
cx q[1], q[4];
cx q[0], q[4];
barrier q;

```

Codice 3.33: Interfaccia Python/Qiskit - Circuito QASM.

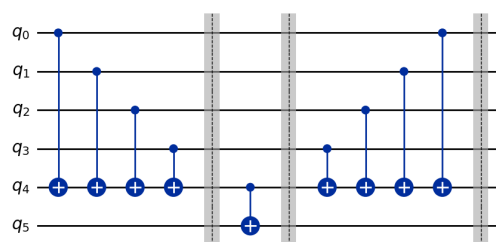


Figura 3.3: Interfaccia Python/Qiskit - Circuito Qiskit.

Capitolo 4

Scenario applicativo: il problema delle n regine

4.1 Panoramica progettuale

Il problema delle n regine (*n-queens problem*) è uno dei benchmark più noti nell'ambito dei problemi di ottimizzazione combinatoria e, in particolare, di soddisfacimento dei vincoli (*Constraint Satisfaction Problems*, CSP). Estensione del puzzle delle 8 regine, il problema consiste nel disporre n regine su una scacchiera $n \times n$ in modo che nessuna di esse possa attaccarne un'altra, ovvero ne condivida riga, colonna o diagonale.

Formalmente, il problema può essere modellato tramite un vettore di variabili discrete $X = (x_0, x_1, \dots, x_{n-1})$, in cui l'indice $i \in \{0, \dots, n-1\}$ fissa in modo implicito e univoco la riga occupata, mentre il valore della variabile $x_i \in \{0, \dots, n-1\}$ specifica la colonna assegnata alla regina posta sulla riga i ; il vincolo di riga è, quindi, soddisfatto per costruzione.

Lo spazio delle configurazioni ammissibili è delimitato da due vincoli fondamentali:

- Vincolo di colonna:

$$x_i \neq x_j, \quad \forall i, j \in \{0, \dots, n-1\}, i \neq j; \quad (4.1)$$

- Vincolo di diagonale:

$$|x_i - x_j| \neq |i - j|, \quad \forall i, j \in \{0, \dots, n-1\}, i \neq j, \quad (4.2)$$

ovvero

$$x_i - x_j \neq i - j, \quad \forall i, j \in \{0, \dots, n-1\}, i \neq j \quad (4.3)$$

per le diagonali principali (o differenza) e

$$x_i + x_j \neq i + j, \quad \forall i, j \in \{0, \dots, n-1\}, i \neq j \quad (4.4)$$

per le diagonali secondarie (o somma).

Mentre il vincolo di colonna restringe lo spazio di ricerca dalle n^n configurazioni possibili al sottoinsieme delle permutazioni $n!$, il vincolo di diagonale è il cuore della complessità combinatoria del problema. La ricerca esaustiva e il conteggio di tutte le soluzioni distinte sono soggetti a una crescita combinatoria esponenziale intrattabile al crescere di n , rendendo il problema delle n regine ideale per il confronto tra paradigma classico e quantistico:

- Paradigma classico - Bitwise backtracking: algoritmo di backtracking caratterizzato da una rappresentazione binaria della scacchiera (*bitboard*), da operazioni bitwise per la verifica dei vincoli e dal *pruning* dinamico, che interrompe la ricerca su sottospazi in cui è stato riscontrato un conflitto; questo approccio costituisce lo stato dell'arte per la risoluzione del problema delle n regine.
- Paradigma quantistico - Quantum search: algoritmo di Grover e sviluppi successivi, discussi nel Paragrafo 2.3, che offrono un guadagno quadratico teorico per la ricerca esaustiva, ma rendono necessaria la progettazione di hardware *fault-tolerant* per essere competitivi. Il problema delle n regine rappresenta un benchmark di analisi e ottimizzazione di oracoli quantistici per CSP, a partire dal quale possono essere implementati problemi con vincoli meno strutturati e più densi, come il completamento delle n regine (*n-queens completion problem*), per i quali il quantum search può offrire, in ottica *fault-tolerant*, un vantaggio concreto rispetto all'approccio classico.

Nel presente lavoro di tesi, il problema delle n regine è stato impiegato come scenario applicativo per il framework QLPP. A partire da un'implementazione *gate-based* personalizzata in Qiskit del caso 4×4 (`4queens_problem.ipynb`), il più semplice con soluzioni, si sono astratte delle regole generali per la configurazione e la logica circuitale (approccio *bottom-up*), grazie alle quali si è realizzato un motore logico QLPP (`nqueens_problem.pl`), che consente una qualunque istanza dell'oracolo quantistico per il problema delle n regine (approccio *top-down*). Successivamente, è stato realizzato il flusso operativo su Python/Qiskit (`nqueens_problem.ipynb`), in cui viene utilizzata la relativa interfaccia verso QLPP, descritta nel Paragrafo 3.4.5, al fine di integrare l'istanza dell'oracolo QLPP nell'algoritmo di Grover. Infine, si analizzano e discutono i risultati ottenuti dalle simulazioni e si presentano delle possibili ottimizzazioni per l'esecuzione su hardware reale NISQ.

In particolare, la progettazione si declina come segue:

- motore logico QLPP, con
 - inizializzazione;
 - vincolo di colonna;
 - vincolo di diagonale;
 - test di soddisfacimento;
 - composizione dell'oracolo;
- flusso operativo in Python/Qiskit, con

- configurazione;
 - composizione del circuito;
 - esecuzione su simulatore;
 - filtro di ammissibilità;
 - valutazione con SNR;
 - ispezione degli output;
- risultati;
 - possibili ottimizzazioni.

4.2 Motore logico QLPP

Il motore logico QLPP consiste nell'insieme di regole che consentono la realizzazione di una qualunque istanza dell'oracolo quantistico per il problema delle n regine; in particolare, la logica comprende l'inizializzazione, il vincolo di colonna, il vincolo di diagonale, il test di soddisfacimento e la composizione dell'oracolo.

4.2.1 Inizializzazione

L'inizializzazione viene effettuata dalla clausola `init/16`, che, per un dato N in input, si occupa della configurazione dei parametri per la specifica istanza dell'oracolo.

In particolare, l'inizializzazione comprende le fasi di dimensionamento del registro, di partizionamento del registro, di partizionamento dello stato e di costruzione di moduli per la verifica dei vincoli di colonna e di diagonale e per l'offset di colonna applicato prima di ciascuna verifica di vincolo diagonale; inoltre, viene estratta la coppia di qubit di controllo per l'azione target di verifica, distinguendo il caso di una catena Toffoli, in cui un qubit è ausiliario, dal caso di una porta Toffoli (non presente per il vincolo diagonale dato che i controlli sono almeno tre perché comprensivi del qubit di overflow).

Il Codice 4.1 mostra l'implementazione dell'inizializzazione.

```
init(N,

    % parametri base
    QRin,Queens,XOR_Qbits,Check_Qbits,Aux_Qbits,

    % parametri per il vincolo di colonna
    CEQ_Check_SubCircuit,Inv_CEQ_Check_SubCircuit,
    XOR_FQbit,CEQ_Check_FQbit,

    % parametri per il vincolo di diagonale
    DEQ_Incr_SubCircuit,DEQ_Decr_SubCircuit,
```

```

DEQ_Check_SubCircuit, Inv_DEQ_Check_SubCircuit,
Overflow_Qbit, DEQ_Check_FQbit

) :-

% *** parametri base ***
%
% dimensionamento
get_dims(N, QueenDIM, StateDIM, ChecksDIM, AuxDIM, DIM),
reset_qbit_number,
make_qbits(DIM, QRin),
%
% partizionamento
%
% lista delle liste di qubit per ogni regina
qrangle(0, StateDIM, State_Qbits),
queens_split(State_Qbits, QueenDIM, Queens),
%
% lista di qubit per lo XOR tra due regine
Aux_start is StateDIM + QueenDIM,
qrangle(StateDIM, Aux_start, XOR_Qbits),
%
% lista di qubit ausiliari
Aux_end is Aux_start + AuxDIM,
qrangle(Aux_start, Aux_end, Aux_Qbits),
%
% qubit per l'overflow
Overflow_Qbit = q(Aux_end),
%
% lista di qubit per le verifiche di vincolo
Checks_start is Aux_end + 1,
Checks_end is Checks_start + ChecksDIM,
qrangle(Checks_start, Checks_end, Check_Qbits),

% *** parametri per il vincolo di colonna ***
%
% modulo di verifica e del suo inverso
XOR_end is Aux_start - 1,
qrangle(StateDIM, XOR_end, XOR_PQbits),
CEQ_Check_Aux_end is Aux_start + QueenDIM - 2,
qrangle(Aux_start, CEQ_Check_Aux_end, CEQ_Check_Aux_Qbits),
check_subcircuit(QRin, XOR_PQbits, CEQ_Check_Aux_Qbits,
                  CEQ_Check_SubCircuit, Inv_CEQ_Check_SubCircuit),
%
% coppia di qubit di controllo per l'azione target di verifica
( QueenDIM > 2 ->

    % caso 1: catena Toffoli
    XOR_FQbit = q(XOR_end),

```

```

CEQ_Check_FQbit_idx is CEQ_Check_Aux_end - 1,
CEQ_Check_FQbit = q(CEQ_Check_FQbit_idx)
;
% caso 2: porta Toffoli
XOR_FQbit = q(StateDIM),
CEQ_Check_FQbit = q(XOR_end)

),

% *** parametri per il vincolo di diagonale ***
%
% modulo di incremento e di decremento
DEQ_Incr_Aux_end is Aux_start + QueenDIM - 2,
qrangle(Aux_start, DEQ_Incr_Aux_end, DEQ_Incr_Aux_Qbits),
incr_subcircuit(QRin, XOR_Qbits, DEQ_Incr_Aux_Qbits, Overflow_Qbit,
    DEQ_Incr_SubCircuit, DEQ_Decr_SubCircuit),

%
% modulo di verifica e del suo inverso
DEQ_Check_Aux_end is Aux_start + QueenDIM - 1,
qrangle(Aux_start, DEQ_Check_Aux_end, DEQ_Check_Aux_Qbits),
check_subcircuit(QRin, XOR_Qbits, DEQ_Check_Aux_Qbits,
    DEQ_Check_SubCircuit, Inv_DEQ_Check_SubCircuit),

%
% coppia di qubit di controllo per l'azione target di verifica
DEQ_Check_FQbit_idx is DEQ_Check_Aux_end - 1,
DEQ_Check_FQbit = q(DEQ_Check_FQbit_idx).

```

Codice 4.1: Inizializzazione.

Il dimensionamento del registro è gestito dalle seguenti clausole:

- $queenDIM/2$: calcolo della dimensione necessaria al singolo stato, ovvero alla codifica del valore di colonna di una regina, e, per costruzione, all'operazione di *XOR* bitwise tra una coppia di regine ($QueenDIM = \lceil \log_2(N) \rceil$);
- $stateDIM/3$: calcolo della dimensione necessaria allo stato di tutte le regine ($StateDIM = N \cdot QueenDIM$);
- $checksDIM/2$: calcolo della dimensione necessaria alle verifiche di vincolo tra tutte le possibili coppie di regine ($ChecksDIM = N \cdot (N - 1)/2$);
- $auxDIM/2$: calcolo della dimensione necessaria alle operazioni ausiliarie delle catene Toffoli ($AuxDIM = Cmax - 2$, con $Cmax = ChecksDIM - 1$ numero massimo di controlli);
- $dim/5$: calcolo della dimensione totale del registro, considerando un qubit aggiuntivo per l'overflow ($DIM = QueenDIM + StateDIM + ChecksDIM + AuxDIM + 1$);

- `get_dims/6`: chiamata di tutti i passi precedenti per un dato N e ritorno dei parametri di dimensionamento a `init/16`.

Viene quindi istanziato in `init/16` il registro `QRin` di dimensione `DIM`; più precisamente, `QRin` è un’astrazione del registro che sarà poi propriamente istanziato in `Qiskit`.

Il Codice 4.2 mostra l’implementazione del dimensionamento del registro.

```
% dimensione per una regina
queenDIM(N, QueenDIM) :-
    QueenDIM is integer(ceiling(log(N)/log(2))).

% dimensione per lo stato
stateDIM(N, QueenDIM, StateDIM) :-
    StateDIM is N * QueenDIM.

% dimensione per le verifiche di vincolo
checksDIM(N, ChecksDIM) :-
    ChecksDIM is (N*(N-1)/2).

% dimensione per le operazioni intermedie delle catene Toffoli
auxDIM(ChecksDIM, AuxDIM) :-
    AuxDIM is ChecksDIM - 3.

% dimensione totale
dim(QueenDIM, StateDIM, ChecksDIM, AuxDIM, DIM) :-
    DIM is QueenDIM + StateDIM + ChecksDIM + AuxDIM + 1.

% calcolo dei parametri di dimensionamento per un dato N
get_dims(N, QueenDIM, StateDIM, ChecksDIM, AuxDIM, DIM) :-
    queenDIM(N, QueenDIM),
    stateDIM(N, QueenDIM, StateDIM),
    checksDIM(N, ChecksDIM),
    auxDIM(ChecksDIM, AuxDIM),
    dim(QueenDIM, StateDIM, ChecksDIM, AuxDIM, DIM).
```

Codice 4.2: Dimensionamento del registro.

Il partizionamento del registro, effettuato in `init/16`, utilizzando in ordine le dimensioni parziali `StateDIM`, `QueenDIM`, `AuxDIM`, 1 (overflow) e `ChecksDIM`, è il seguente:

```
QRin = {State_Qbits,
        XOR_Qbits,
        Aux_Qbits,
        Ovfl_Qbit,
        Check_Qbits}
```

La preparazione iniziale dei qubit è delegata a Qiskit e consiste nella sovrapposizione uniforme con Hadamard dei State_Qbits e nell'impostazione allo stato $|1\rangle$ con Pauli-X dei XOR_Qbits e di Overflow_Qbit (i rimanenti qubit si trovano di default allo stato $|0\rangle$).

Inoltre, viene realizzata un'ulteriore partizione mediante la chiamata in `init/16` della clausola `queens_split/3`, che trasforma State_Qbits in Queens, una lista di liste, ciascuna delle quali contiene i qubit relativi allo stato di una regina, in modo da semplificare la selezione delle coppie di regine per le verifiche di vincolo.

Il Codice 4.3 mostra l'implementazione del partizionamento dello stato.

```
queens_split([],_,[]) :- !.
queens_split(State_Qbits,QueenDIM,[Queen|Rest]) :-
    length(Queen,QueenDIM),
    append(Queen,State_Rest,State_Qbits), !,
    queens_split(State_Rest,QueenDIM,Rest).
```

Codice 4.3: Partizionamento dello stato.

I moduli sono parametri che rappresentano dei sottocircuiti riutilizzabili per l'implementazione della logica dei vincoli.

È presente la costruzione di due moduli e i rispettivi inversi mediante le seguenti clausole:

- `check_subcircuit/5`: costruzione del modulo di verifica (e del suo inverso), che consiste in una o più porte Toffoli con qubit di controllo e ausiliari noti a priori, esclusa l'azione sul target, diverso per ogni verifica di un vincolo;
- `incr_subcircuit/6`: costruzione del modulo di incremento (e di decremento), con qubit di controllo, ausiliari e target noti a priori, per l'applicazione di un determinato offset di colonna prima di ciascuna verifica di vincolo diagonale.

Il Codice 4.4 mostra l'implementazione dei moduli per la verifica e per l'offset.

```
% modulo di verifica e del suo inverso
check_subcircuit(QRin,CQbits,ATQbits,
    Check_SubCircuit,Inv_Check_SubCircuit) :-
    build_circuit(QRin,n-tof,CQbits,ATQbits,chain,
        Check_SubCircuit),
    reverse(Check_SubCircuit,Inv_Check_SubCircuit).

% modulo di incremento e di decremento
incr_subcircuit(QRin,CQbits,AQbits,Overflow_Qbit,
    Incr_SubCircuit,Decr_SubCircuit) :-
    build_circuit(QRin,incr,CQbits,AQbits,Overflow_Qbit,
        Incr_SubCircuit),
    reverse(Incr_SubCircuit,Decr_SubCircuit).
```

Codice 4.4: Moduli per la verifica e per l'offset.

4.2.2 Vincolo di colonna

Il vincolo di colonna effettua tante verifiche di uguaglianza quante sono le possibili coppie di regine ($N \cdot (N - 1)/2$); le verifiche avvengono confrontando in ordine ciascuna regina, eccetto l'ultima, con tutte le successive.

Il vincolo di colonna viene realizzato dalle seguenti clausole:

- `col_eq/11`: costruzione del circuito parziale del vincolo di colonna, relativamente ai confronti di una data regina con tutte le successive;
- `col_eq_compute/11`: chiamata del passo precedente per ogni regina, eccetto l'ultima.

Il Codice 4.5 mostra l'implementazione del vincolo di colonna.

```
col_eq_compute(_, CEQ_Circuit, [_], _, _, _, _, _, _, CEQ_Circuit) :- !.  
%  
col_eq_compute(QRin, CEQ_PCircuit, [Queen|Rest], XOR_Qbits,  
               CEQ_Check_SubCircuit, Inv_CEQ_Check_SubCircuit,  
               XOR_FQbit, CEQ_Check_FQbit, Check_Qbits, CheckRest,  
               CEQ_Circuit) :-  
  
    % fanout regina  
    bw_qXOR(Queen, XOR_Qbits, XOR_Exp),  
    build_circuit(QRin, exp(XOR_Exp), [Fanout_Circuit]),  
  
    % sottocircuito di uguaglianza per una regina  
    col_eq(QRin, [], Rest, XOR_Qbits, CEQ_Check_SubCircuit,  
          Inv_CEQ_Check_SubCircuit, XOR_FQbit, CEQ_Check_FQbit,  
          CheckRest, CheckRest1, CEQ_Checks_PCircuit),  
  
    % sottocircuito del vincolo di colonna per una regina  
    CEQ_Checks_PCircuit1 = [Fanout_Circuit|CEQ_Checks_PCircuit],  
    reverse(Fanout_Circuit, Inv_Fanout_Circuit),  
    append(CEQ_Checks_PCircuit1, [Inv_Fanout_Circuit],  
          CEQ_Checks_Circuit),  
  
    % aggiunta in coda al sottocircuito del vincolo di colonna  
    append(CEQ_PCircuit, CEQ_Checks_Circuit, CEQ_PCircuit1),  
  
    col_eq_compute(QRin, CEQ_PCircuit1, Rest, XOR_Qbits,  
                  CEQ_Check_SubCircuit, Inv_CEQ_Check_SubCircuit,  
                  XOR_FQbit, CEQ_Check_FQbit, Check_Qbits, CheckRest1,  
                  CEQ_Circuit).
```

```

col_eq(_, CEQ_PCircuit, [], _, _, _, _, PCheckRest, PCheckRest,
      CEQ_PCircuit) :- !.
%
col_eq(QRin, CEQ_PCircuit, [Queen|Rest], XOR_Qbits, CEQ_Check_SubCircuit,
      Inv_CEQ_Check_SubCircuit, XOR_FQbit, CEQ_Check_FQbit,
      [Check_Qbit|PCheckRest], CheckRest, CEQ_Circuit) :-

    % sottocircuito di XOR bitwise
    bw_qXOR(Queen, XOR_Qbits, XOR_Exp),
    build_circuit(QRin, exp(XOR_Exp), [XOR_Circuit]),

    % aggiunta in testa al modulo di verifica
    CEQ_Check_Compute = [XOR_Circuit|CEQ_Check_SubCircuit],

    % azione sul target della verifica
    build_circuit(QRin,
                  exp([Check_Qbit: XOR_FQbit qAND CEQ_Check_FQbit]),
                  [CEQ_Check_Action]),

    % uncompute del sottocircuito di XOR bitwise
    reverse(XOR_Circuit, Inv_XOR_Circuit),

    % aggiunta in coda al modulo inverso di verifica
    append(Inv_CEQ_Check_SubCircuit, [Inv_XOR_Circuit],
           CEQ_Check_Uncompute),

    % sottocircuito di singola uguaglianza
    CEQ_Check_Action_Uncompute = [CEQ_Check_Action|
                                   CEQ_Check_Uncompute],
    append(CEQ_Check_Compute, CEQ_Check_Action_Uncompute,
           CEQ_Check_Circuit),

    % aggiunta in coda
    % al sottocircuito di uguaglianza per una regina
    append(CEQ_PCircuit, CEQ_Check_Circuit, CEQ_PCircuit1),

    col_eq(QRin, CEQ_PCircuit1, Rest, XOR_Qbits, CEQ_Check_SubCircuit,
          Inv_CEQ_Check_SubCircuit, XOR_FQbit, CEQ_Check_FQbit,
          PCheckRest, CheckRest, CEQ_Circuit).

```

Codice 4.5: Vincolo di colonna.

A fini esplicativi, si consideri il vincolo di colonna per la prima regina (Q0) nel caso 4×4 ; per una maggiore chiarezza, l'implementazione Qiskit di tale esempio e dei successivi è quella di `4queens_problem.ipynb`, realizzata bottom-up senza qubit ausiliari mediante l'utilizzo

della scomposizione di Barenco del default *noancilla* per le porte a più di due controlli.
Il mapping QLPP $\leftrightarrow$ Qiskit tra i qubit è il seguente:

State_Qbits : $q(0) - q(7) \longleftrightarrow Q0_0 - Q3_1$

XOR_Qbits : $q(8), q(9) \longleftrightarrow xor_0, xor_1$

Aux_Qbits : $q(10) - q(12) \longleftrightarrow \emptyset$

Overflow_Qbit : $q(13) \longleftrightarrow ovfl$

Check_Qbits : $q(14) - q(19) \longleftrightarrow tgt_0 - tgt_5$

Il flusso di operazioni per la regina Q0 è il seguente:

1. fanout, con porte *CNOT*, di Q0 su XOR_Qbits;
2. *XOR* bitwise in-place, con porte *CNOT*, tra Q0 e Q1 su XOR_Qbits;
3. *AND* reduction, con porta Toffoli, dei XOR_Qbits con target sul primo qubit non utilizzato dei Check_Qbits;
4. uncompute dello *XOR* bitwise;
5. ripetere le operazioni dal passo 2 per i confronti successivi (Q0-Q2 e Q0-Q3);
6. uncompute del fanout di Q0.

I passi 2, 3 e 4 compongono l'operatore di uguaglianza *EQ*. Il flusso descritto si ripete poi analogamente per Q1 (confronti Q1-Q2 e Q1-Q3) e per Q2 (confronto Q2-Q3), a completamento dell'implementazione del vincolo di colonna.

Di seguito sono riportati in ordine il diagramma parziale Qiskit (Figura 4.1), il circuito parziale Qiskit (Figura 4.2) e il circuito parziale QLPP (Codice 4.6), corrispondenti al flusso descritto per la regina Q0.

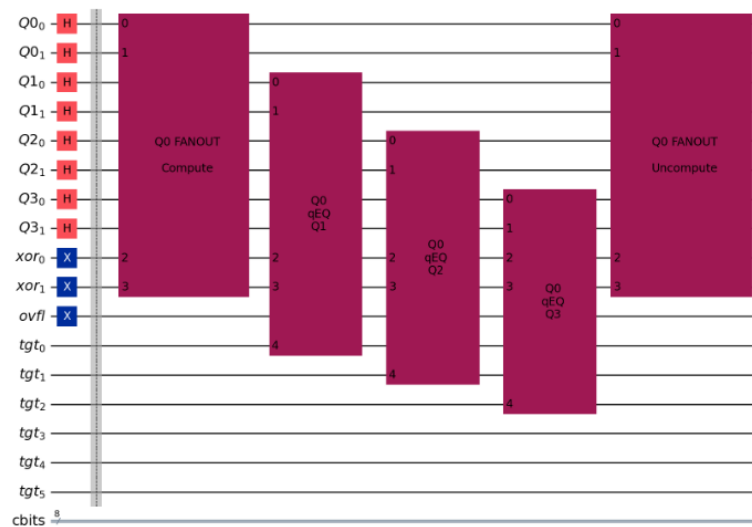


Figura 4.1: Vincolo di colonna - Diagramma parziale Qiskit.

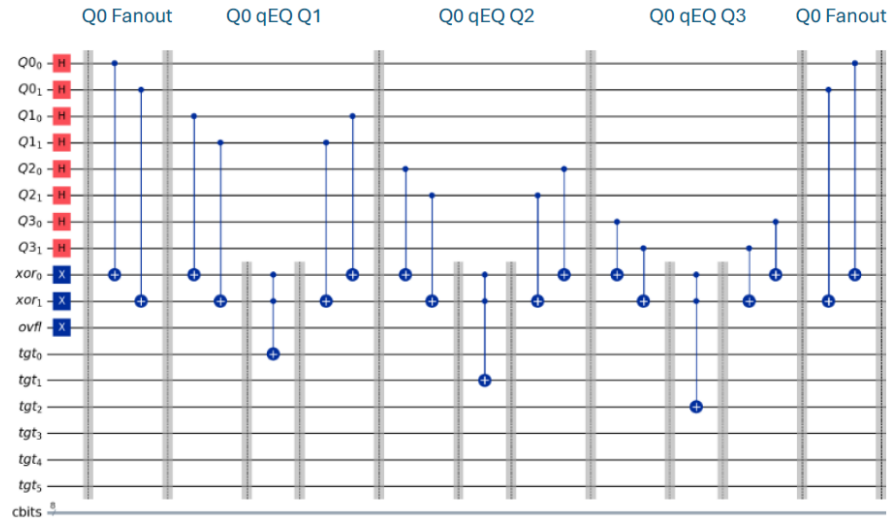


Figura 4.2: Vincolo di colonna - Circuito parziale Qiskit.

```
% *** Q0 fanout compute ***
[q(8):[[cnot(q(0),q(8))]],q(9):[[cnot(q(1),q(9))]]],

% *** Q0 qEQ Q1 ***
%
% Q0 qXOR Q1 compute
[q(8):[[cnot(q(2),q(8))]],q(9):[[cnot(q(3),q(9))]]],
%
% qAND reduction
[q(14):[[tof(q(8),q(9),q(14))]]],
%
% Q0 qXOR Q1 uncompute
[q(9):[[cnot(q(3),q(9))]],q(8):[[cnot(q(2),q(8))]]],

% *** Q0 qEQ Q2 ***
%
% Q0 qXOR Q2 compute
[q(8):[[cnot(q(4),q(8))]],q(9):[[cnot(q(5),q(9))]]],
%
% qAND reduction
[q(15):[[tof(q(8),q(9),q(15))]]],
%
% Q0 qXOR Q2 uncompute
[q(9):[[cnot(q(5),q(9))]],q(8):[[cnot(q(4),q(8))]]],

% *** Q0 qEQ Q3 ***
%
% Q0 qXOR Q3 compute
[q(8):[[cnot(q(6),q(8))]],q(9):[[cnot(q(7),q(9))]]],
%
% qAND reduction
[q(16):[[tof(q(8),q(9),q(16))]]],
%
```

```

% Q0 qXOR Q3 uncompute
[q(9) : [[cnot(q(7), q(9))]], q(8) : [[cnot(q(6), q(8))]]],

% *** Q0 fanout uncompute ***
[q(9) : [[cnot(q(1), q(9))]], q(9) : [[cnot(q(0), q(8))]]],

% ...

```

Codice 4.6: Vincolo di colonna - Circuito parziale QLPP.

4.2.3 Vincolo di diagonale

Il vincolo di diagonale effettua le verifiche di uguaglianza in modo simile a quanto visto per il vincolo di colonna, ma con alcune differenze. Prima di ogni verifica per una data regina, viene applicato per tale regina un offset di colonna cumulativo con qubit per l'overflow. I qubit di controllo per ciascuna verifica includono il qubit di overflow, che, se a $|0\rangle$ (logica inversa), invalida la verifica. Al termine delle verifiche per una data regina, l'offset accumulato viene resettato. Il vincolo di diagonale si scompone nelle verifiche per le diagonali principali (o differenza), con offset positivo, e in quelle per le diagonali secondarie (o somma), con offset negativo ($N \cdot (N - 1)$ confronti complessivi).

Il vincolo di diagonale è realizzato mediante le seguenti clausole:

- `diag_eq/14`: costruzione del circuito parziale di una delle due componenti del vincolo di diagonale, relativamente ai confronti di una data regina con tutte le successive;
- `diag_eq_compute/13`: chiamata del passo precedente per ogni regina, eccetto l'ultima.

Per la costruzione del vincolo di diagonale sono necessarie due chiamate a `diag_eq_compute/13`, una per la componente relativa alle diagonali principali e una per la componente relativa alle diagonali secondarie; questo avviene scambiando di ruolo i moduli di incremento e di decremento nei parametri passati a tale clausola.

Il Codice 4.7 mostra l'implementazione del vincolo di diagonale.

```

diag_eq_compute(_, DEQ_Circuit, [_], _, _, _, _, _, _, _, DEQ_Circuit) :- !.
%
diag_eq_compute(QRin, DEQ_PCircuit, [Queen|Rest], XOR_Qbits,
                DEQ_Ctr_SubCircuit, Inv_DEQ_Ctr_SubCircuit,
                DEQ_Check_SubCircuit, Inv_DEQ_Check_SubCircuit,
                Overflow_Qbit, DEQ_Check_FQbit, Check_Qbits,
                CheckRest, DEQ_Circuit) :-

    % fanout regina
    bw_qXOR(Queen, XOR_Qbits, XOR_Exp),
    build_circuit(QRin, exp(XOR_Exp), [Fanout_Circuit]),

```

```

% sottocircuito di uguaglianza per una regina
diag_eq(QRin, [], Rest, XOR_Qbits,
        DEQ_Ctr_SubCircuit, Inv_DEQ_Ctr_SubCircuit,
        DEQ_Check_SubCircuit, Inv_DEQ_Check_SubCircuit,
        Overflow_Qbit, DEQ_Check_FQbit,
        CheckRest, CheckRest1, [], DEQ_Checks_PCircuit),

% sottocircuito del vincolo di diagonale per una regina
DEQ_Checks_PCircuit1 = [Fanout_Circuit|DEQ_Checks_PCircuit],
reverse(Fanout_Circuit, Inv_Fanout_Circuit),
append(DEQ_Checks_PCircuit1, [Inv_Fanout_Circuit],
        DEQ_Checks_Circuit),

% aggiunta in coda al sottocircuito del vincolo di diagonale
append(DEQ_PCircuit, DEQ_Checks_Circuit, DEQ_PCircuit1),

diag_eq_compute(QRin, DEQ_PCircuit1, Rest, XOR_Qbits,
                DEQ_Ctr_SubCircuit, Inv_DEQ_Ctr_SubCircuit,
                DEQ_Check_SubCircuit, Inv_DEQ_Check_SubCircuit,
                Overflow_Qbit, DEQ_Check_FQbit,
                Check_Qbits, CheckRest1, DEQ_Circuit).

%
%
diag_eq(_, DEQ_PCircuit, [], _, _, _, _, _, _, PCheckRest, PCheckRest,
        Inv_DEQ_Ctr_Chain, DEQ_PCircuit1) :- !,

% aggiunta in coda del sottocircuito di reset dell'offset
append(DEQ_PCircuit, Inv_DEQ_Ctr_Chain, DEQ_PCircuit1).

%
diag_eq(QRin, DEQ_PCircuit, [Queen|Rest], XOR_Qbits,
        DEQ_Ctr_SubCircuit, Inv_DEQ_Ctr_SubCircuit,
        DEQ_Check_SubCircuit, Inv_DEQ_Check_SubCircuit,
        Overflow_Qbit, DEQ_Check_FQbit, [Check_Qbit|PCheckRest],
        CheckRest, Inv_DEQ_Ctr_PChain, DEQ_Circuit) :-

% sottocircuito di XOR bitwise
bw_qXOR(Queen, XOR_Qbits, XOR_Exp),
build_circuit(QRin, exp(XOR_Exp), [XOR_Circuit]),

% aggiunta in testa al modulo di verifica
DEQ_Check_Compute = [XOR_Circuit|DEQ_Check_SubCircuit],

% azione sul target della verifica
build_circuit(QRin, exp([Check_Qbit:

```

```

                                Overflow_Qbit qAND DEQ_Check_FQbit]),
                                [DEQ_Check_Action]),

% uncompute del sottocircuito di XOR bitwise
reverse(XOR_Circuit, Inv_XOR_Circuit),

% aggiunta in coda al modulo inverso di verifica
append(Inv_DEQ_Check_SubCircuit, [Inv_XOR_Circuit],
        DEQ_Check_Uncompute),

% sottocircuito di singola uguaglianza
DEQ_Check_Action_Uncompute = [DEQ_Check_Action|
                                DEQ_Check_Uncompute],
append(DEQ_Check_Compute, DEQ_Check_Action_Uncompute,
        DEQ_Check_Circuit),

% aggiunta in coda al modulo per l'offset
append(DEQ_Ctr_SubCircuit, DEQ_Check_Circuit, DEQ_PCircuit1),

% aggiunta in coda
% al sottocircuito di uguaglianza per una regina
append(DEQ_PCircuit, DEQ_PCircuit1, DEQ_PCircuit2),

% accumulazione del sottocircuito di reset dell'offset
append(Inv_DEQ_Ctr_PChain, Inv_DEQ_Ctr_SubCircuit,
        Inv_DEQ_Ctr_PChain1),

diag_eq(QRin, DEQ_PCircuit2, Rest, XOR_Qbits,
        DEQ_Ctr_SubCircuit, Inv_DEQ_Ctr_SubCircuit,
        DEQ_Check_SubCircuit, Inv_DEQ_Check_SubCircuit,
        Overflow_Qbit, DEQ_Check_FQbit,
        PCheckRest, CheckRest, Inv_DEQ_Ctr_PChain1, DEQ_Circuit).

```

Codice 4.7: Vincolo di diagonale.

Analogamente a quanto fatto per il vincolo di colonna, si consideri il vincolo di diagonale per la regina Q_0 nel caso 4×4 .

Per comodità, si riporta nuovamente il mapping QLPP $\leftrightarrow$ Qiskit tra i qubit:

State_Qbits : $q(0) - q(7) \longleftrightarrow Q_{0_0} - Q_{3_1}$

XOR_Qbits : $q(8), q(9) \longleftrightarrow xor_0, xor_1$

Aux_Qbits : $q(10) - q(12) \longleftrightarrow \emptyset$

Overflow_Qbit : $q(13) \longleftrightarrow ovfl$

$$\text{Check_Qbits} : q(14) - q(19) \longleftrightarrow \text{tgt}_0 - \text{tgt}_5$$

Il flusso operativo per la regina Q0 nel caso delle diagonali principali è il seguente:

1. fanout, con porte *CNOT*, di Q0 su XOR_Qbits;
2. incremento di Q0 su XOR_Qbits e Overflow_Qbit;
3. *XOR* bitwise in-place, con porte *CNOT*, tra Q0 e Q1 su XOR_Qbits;
4. *AND* reduction, con catena di porte Toffoli, dei XOR_Qbits e di Overflow_Qbit, con primo qubit ausiliario dei Aux_Qbits (e relativo uncompute) e con target sul primo qubit non utilizzato dei Check_Qbits;
5. uncompute dello *XOR* bitwise;
6. ripetere le operazioni dal passo 2 per i confronti successivi (Q0-Q2 e Q0-Q3);
7. reset dell'offset positivo accumulato;
8. uncompute del fanout di Q0.

I passi 3, 4 e 5 compongono l'operatore di uguaglianza *EQ*. Il flusso descritto si ripete poi analogamente per Q1 (confronti Q1-Q2 e Q1-Q3) e per Q2 (confronto Q2-Q3), a completamento dell'implementazione della componente per le diagonali principali; la componente per le diagonali secondarie si realizza in modo analogo, con decremento al passo 2 e reset dell'offset negativo accumulato al passo 7.

Di seguito sono riportati in ordine il diagramma parziale Qiskit (Figura 4.3 e Figura 4.4), il circuito parziale Qiskit (Figura 4.5 e Figura 4.6) e il circuito parziale QLPP (Codice 4.8), corrispondenti al flusso descritto per la regina Q0 nel caso delle diagonali principali.

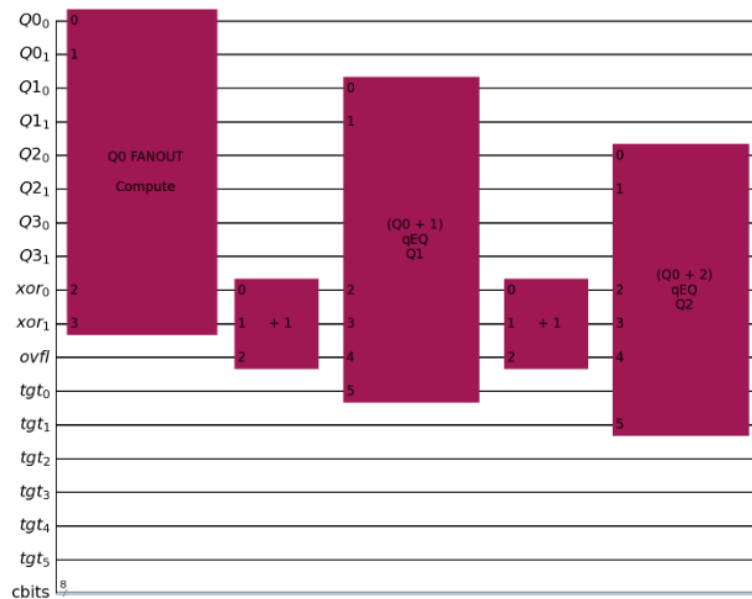


Figura 4.3: Vincolo di diagonale - Diagramma parziale Qiskit (1).

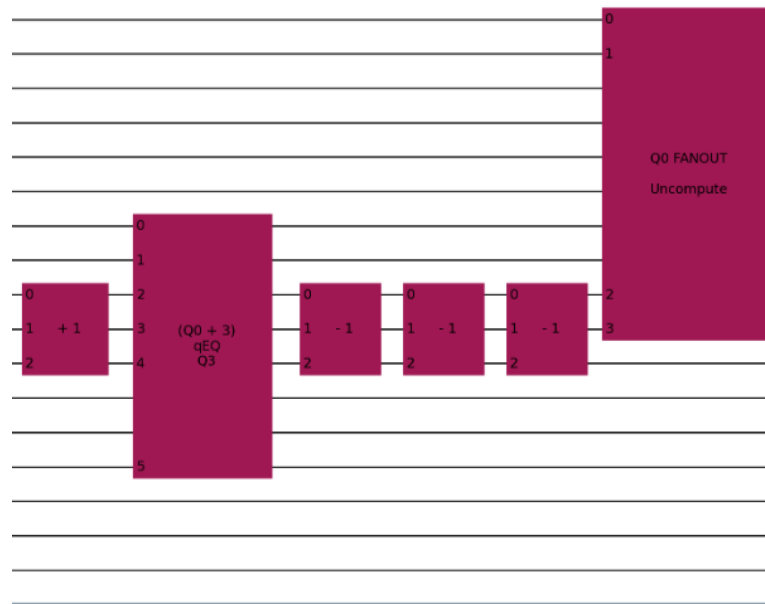


Figura 4.4: Vincolo di diagonale - Diagramma parziale Qiskit (2).

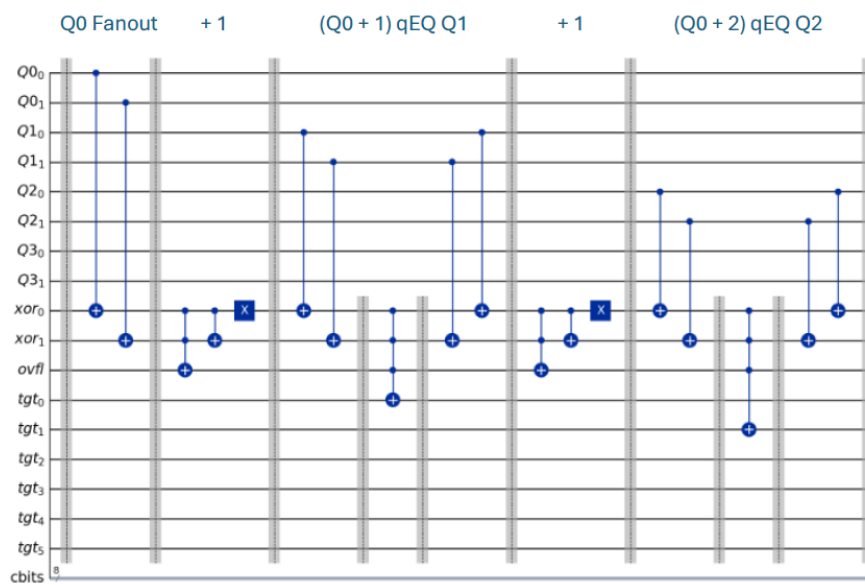


Figura 4.5: Vincolo di diagonale - Circuito parziale Qiskit (1).

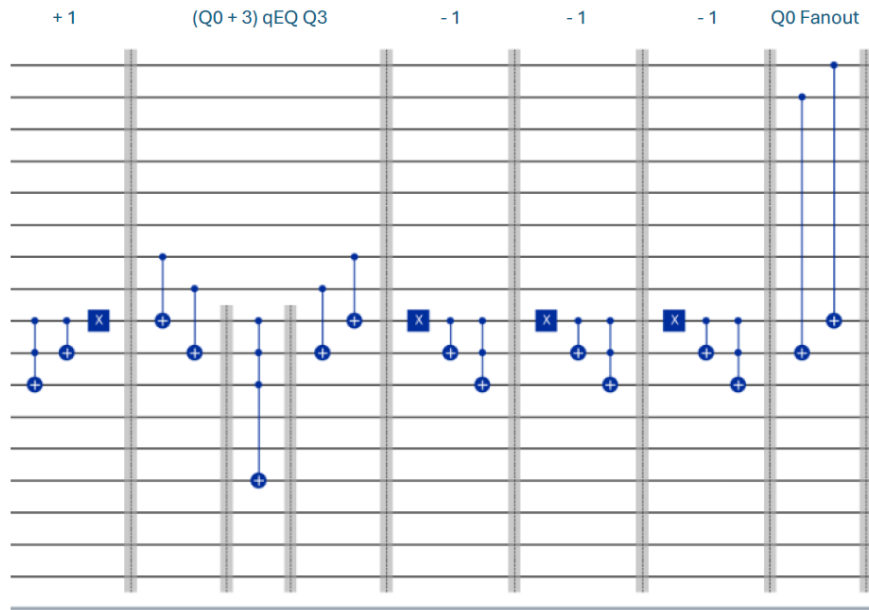


Figura 4.6: Vincolo di diagonale - Circuito parziale Qiskit (2).

```
% *** Q0 fanout compute ***
[q(8):[[cnot(q(0),q(8))]],q(9):[[cnot(q(1),q(9))]]],

% *** Q0 + 1 ***
[q(13):[[tof(q(8),q(9),q(13))]]],
[q(9):[[cnot(q(8),q(9))]]],
[q(8):[[not(q(8))]]],

% *** (Q0 + 1) qEQ Q1 ***
%
% (Q0 + 1) qXOR Q1 compute
[q(8):[[cnot(q(2),q(8))]],q(9):[[cnot(q(3),q(9))]]],
%
% qAND reduction
[q(10):[[tof(q(8),q(9),q(10))]]],
[q(14):[[tof(q(13),q(10),q(14))]]],
[q(10):[[tof(q(8),q(9),q(10))]]],
%
% (Q0 + 1) qXOR Q1 uncompute
[q(9):[[cnot(q(3),q(9))]],q(8):[[cnot(q(2),q(8))]]],

% *** Q0 + 2 ***
[q(13):[[tof(q(8),q(9),q(13))]]],
[q(9):[[cnot(q(8),q(9))]]],
[q(8):[[not(q(8))]]],

% *** (Q0 + 2) qEQ Q2 ***
%
% (Q0 + 2) qXOR Q2 compute
[q(8):[[cnot(q(4),q(8))]],q(9):[[cnot(q(5),q(9))]]],
```

```

%
% qAND reduction
[q(10):[[tof(q(8),q(9),q(10))]]],
[q(15):[[tof(q(13),q(10),q(15))]]],
[q(10):[[tof(q(8),q(9),q(10))]]],
%
% (Q0 + 2) qXOR Q2 uncompute
[q(9):[[cnot(q(5),q(9))]],q(8):[[cnot(q(4),q(8))]]],

% *** Q0 + 3 ***
[q(13):[[tof(q(8),q(9),q(13))]]],
[q(9):[[cnot(q(8),q(9))]]],
[q(8):[[not(q(8))]]],

% *** (Q0 + 3) qEQ Q3 ***
%
% (Q0 + 3) qXOR Q3 compute
[q(8):[[cnot(q(6),q(8))]],q(9):[[cnot(q(7),q(9))]]],
%
% qAND reduction
[q(10):[[tof(q(8),q(9),q(10))]]],
[q(16):[[tof(q(13),q(10),q(16))]]],
[q(10):[[tof(q(8),q(9),q(10))]]],
%
% (Q0 + 3) qXOR Q3 uncompute
[q(9):[[cnot(q(7),q(9))]],q(8):[[cnot(q(6),q(8))]]],

% *** Q0 offset reset ***
%
[q(8):[[not(q(8))]]],
[q(9):[[cnot(q(8),q(9))]]],
[q(13):[[tof(q(8),q(9),q(13))]]],
%
[q(8):[[not(q(8))]]],
[q(9):[[cnot(q(8),q(9))]]],
[q(13):[[tof(q(8),q(9),q(13))]]],
%
[q(8):[[not(q(8))]]],
[q(9):[[cnot(q(8),q(9))]]],
[q(13):[[tof(q(8),q(9),q(13))]]],

% *** Q0 fanout uncompute ***
[q(9):[[cnot(q(1),q(9))]],q(8):[[cnot(q(0),q(8))]]],

% ...

```

Codice 4.8: Vincolo di diagonale - Circuito parziale QLPP.

4.2.4 Test di soddisfacimento

Il test di soddisfacimento dei vincoli, realizzato dalla clausola `test/4`, verifica che sia il vincolo di colonna sia il vincolo di diagonale (nelle sue due componenti) siano rispettati e, in caso affermativo, effettua l'inversione di fase degli stati soluzione, come previsto dall'algoritmo di Grover. In particolare, il test consiste nell'implementazione dell'operatore di *NOR* reduction in base *X*, composto da negazione, *AND* reduction in base *X* e uncompute della negazione dei `Check_Qbits`; il passaggio dalla base computazionale *Z* alla base della fase *X* avviene tramite coniugazione di porte Hadamard. L'operatore di *NOR* reduction in base *X* inverte la fase se non è stata rilevata alcuna uguaglianza, ovvero se tutti i qubit target delle verifiche di vincolo (i `Check_Qbits`) si trovano nello stato $|0\rangle$; l'operatore impiega, inoltre, i qubit ausiliari `Aux_Qbits` (con relativo uncompute) e ha come destinazione finale l'ultimo qubit dei `Check_Qbits`, sebbene anche questo svolga la funzione di controllo per l'inversione di fase.

Il Codice 4.9 mostra l'implementazione del test di soddisfacimento.

```
test(QRin, Aux_Qbits, Check_Qbits, Test_Circuit) :-  
  
    % suddivisione dei qubit target delle verifiche di vincolo in:  
    % - lista dei qubit di controllo per il test, eccetto l'ultimo  
    % - qubit finale di controllo per il test  
    % - qubit target del test  
    append(Check_PQbits, [Check_FQbit, Test_Qbit], Check_Qbits),  
  
    % inversione dell'ordine dei qubit ausiliari  
    reverse(Aux_Qbits, Test_Aux_Qbits),  
  
    % qubit ausiliario finale di controllo per il test  
    last(Test_Aux_Qbits, Test_Aux_FQbit),  
  
    % componenti circuitali  
    %  
    % sottocircuito di negazione  
    negate(Check_Qbits, Neg_Exp),  
    build_circuit(QRin, exp(Neg_Exp), [NOT_SubCircuit]),  
    %  
    % sottocircuito inverso di negazione  
    reverse(NOT_SubCircuit, Inv_NOT_SubCircuit),  
    %  
    % sottocircuito di verifica e del suo inverso  
    check_subcircuit(QRin, Check_PQbits, Test_Aux_Qbits,  
                    Test_Check_SubCircuit, Inv_Test_Check_SubCircuit),  
    %  
    % porta Hadamard sul qubit target  
    build_circuit(QRin, exp([Test_Qbit: qH Test_Qbit]), [H_SubCircuit]),  
    %
```

```

% azione sul target
build_circuit(QRin,
              exp([Test_Qbit: Test_Aux_FQbit qAND Check_FQbit]),
              Test_SubCircuit),

% composizione dell'operatore di qNOR reduction in base X
Test_SubCircuit1 = [NOT_SubCircuit|Test_Check_SubCircuit],
Test_SubCircuit2 = [H_SubCircuit|Test_SubCircuit],
append(Test_SubCircuit1,Test_SubCircuit2,Test_SubCircuit3),
Test_SubCircuit4 = [H_SubCircuit|Inv_Test_Check_SubCircuit],
append(Test_SubCircuit3,Test_SubCircuit4,Test_SubCircuit5),
append(Test_SubCircuit5,[Inv_NOT_SubCircuit],Test_Circuit).

```

Codice 4.9: Test di soddisfacimento.

Di seguito si riportano in ordine il circuito Qiskit senza ancilla (Figura 4.7), il circuito QLPP (Codice 4.10) e il circuito Qiskit con ancilla corrispondente al circuito QLPP per il test di soddisfacimento nel caso 4×4 .

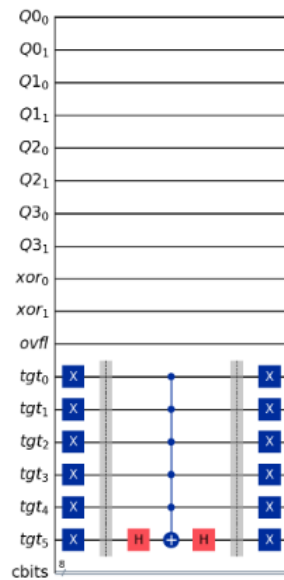


Figura 4.7: Test di soddisfacimento senza ancilla - Circuito Qiskit.

```

% *** qNOR reduction in base X ***
%
% compute dei qNOT
[q(14):[[not(q(14))]],
 q(15):[[not(q(15))]],
 q(16):[[not(q(16))]],
 q(17):[[not(q(17))]],
 q(18):[[not(q(18))]],
 q(19):[[not(q(19))]]],
%
% qAND reduction in base X
[q(12):[[tof(q(14),q(15),q(12))]]],

```

```

[q(11):[[tof(q(16),q(12),q(11))]]],
[q(10):[[tof(q(17),q(11),q(10))]]],
[q(19):[[h(q(19))]]],
[q(19):[[tof(q(10),q(18),q(19))]]],
[q(19):[[h(q(19))]]],
[q(10):[[tof(q(17),q(11),q(10))]]],
[q(11):[[tof(q(16),q(12),q(11))]]],
[q(12):[[tof(q(14),q(15),q(12))]]],
%
% uncompute dei qNOT
[q(19):[[not(q(19))]]],
[q(18):[[not(q(18))]]],
[q(17):[[not(q(17))]]],
[q(16):[[not(q(16))]]],
[q(15):[[not(q(15))]]],
[q(14):[[not(q(14))]]]

```

Codice 4.10: Test di soddisfacimento - Circuito QLPP.

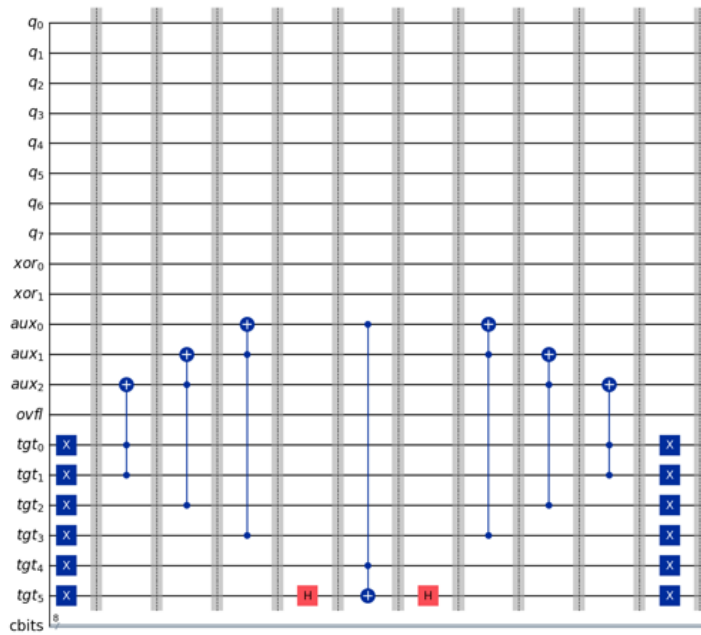


Figura 4.8: Test di soddisfacimento con ancilla - Circuito Qiskit.

4.2.5 Composizione dell'oracolo

La composizione dell'oracolo viene effettuata, per un dato N , dall'insieme di clausole `queens_problem` mostrate nel Codice 4.11, e prevede, nell'ordine, i seguenti passi:

1. `init/16` per la configurazione dei parametri necessari ai componenti circuitali;
2. `col_eq_compute/11` per il circuito del vincolo di colonna;
3. `diag_eq_compute/13` per il circuito del vincolo di diagonale relativo alle diagonali principali;

4. come il passo precedente, ma relativo alle diagonali secondarie (con troncamento, per ridondanza, di XOR bitwise, incremento e fanout finali);
5. test/4 per il circuito del test di soddisfacimento;
6. costruzione del circuito di compute come concatenazione dei circuiti dei vincoli;
7. costruzione del circuito di uncompute dei vincoli;
8. costruzione del circuito dell'oracolo come concatenazione dei circuiti di compute, test e uncompute.

`queens_problem` rappresenta l'interfaccia utente per effettuare query al motore logico QLPP e costruire, quindi, istanze arbitrarie dell'oracolo per il problema delle n regine.

```
queens_problem(N,CEQ_Compute_Circuit,DDEQ_Compute_Circuit,
               SDEQ_Compute_Circuit,Test_Circuit) :-

    % configurazione dei parametri
    init(N,

        % parametri base
        QRin,Queens,XOR_Qbits,Check_Qbits,Aux_Qbits,

        % parametri per il vincolo di colonna
        CEQ_Check_SubCircuit,Inv_CEQ_Check_SubCircuit,
        XOR_FQbit,CEQ_Check_FQbit,

        % parametri per il vincolo di diagonale
        DEQ_Incr_SubCircuit,DEQ_Decr_SubCircuit,
        DEQ_Check_SubCircuit,Inv_DEQ_Check_SubCircuit,
        Overflow_Qbit,DEQ_Check_FQbit

    ),

    % circuito del vincolo di colonna
    %
    col_eq_compute(
        QRin,[],Queens,XOR_Qbits,
        CEQ_Check_SubCircuit,Inv_CEQ_Check_SubCircuit,
        XOR_FQbit,CEQ_Check_FQbit,
        Check_Qbits,Check_Qbits,CEQ_Compute_Circuit1
    ),
    %
    compress(CEQ_Compute_Circuit1,CEQ_Compute_Circuit),

    % circuito del vincolo di diagonale per le diagonali principali
```

```

%
diag_eq_compute(
    QRin,[],Queens,XOR_Qbits,
    DEQ_Incr_SubCircuit,DEQ_Decr_SubCircuit,
    DEQ_Check_SubCircuit,Inv_DEQ_Check_SubCircuit,
    Overflow_Qbit,DEQ_Check_FQbit,
    Check_Qbits,Check_Qbits,DDEQ_Compute_Circuit1
),
%
compress(DDEQ_Compute_Circuit1,DDEQ_Compute_Circuit),

% circuito del vincolo di diagonale per le diagonali secondarie
%
diag_eq_compute(
    QRin,[],Queens,XOR_Qbits,
    DEQ_Decr_SubCircuit,DEQ_Incr_SubCircuit,
    DEQ_Check_SubCircuit,Inv_DEQ_Check_SubCircuit,
    Overflow_Qbit,DEQ_Check_FQbit,
    Check_Qbits,Check_Qbits,SDEQ_Compute_Circuit1
),
%
% troncamento
append(SDEQ_Compute_Circuit2,[],SDEQ_Compute_Circuit1),
append(SDEQ_Compute_Circuit3,DEQ_Incr_SubCircuit,
        SDEQ_Compute_Circuit2),
append(SDEQ_Compute_Circuit4,[],SDEQ_Compute_Circuit3),
%
compress(SDEQ_Compute_Circuit4,SDEQ_Compute_Circuit),

% circuito del test di soddisfacimento
%
test(
    QRin,
    Aux_Qbits,
    Check_Qbits,
    Test_Circuit1
),
%
compress(Test_Circuit1,Test_Circuit).

%
queens_problem(N,Compute_Circuit,Test_Circuit) :-
    queens_problem(N,CEQ_Compute_Circuit,DDEQ_Compute_Circuit,
                    SDEQ_Compute_Circuit,Test_Circuit),

% circuito di compute
append(DDEQ_Compute_Circuit,SDEQ_Compute_Circuit,
        DEQ_Compute_Circuit),

```

```

    append(CEQ_Compute_Circuit, DEQ_Compute_Circuit, Compute_Circuit).
%
queens_problem(N, Circuit) :-
    queens_problem(N, Compute_Circuit, Test_Circuit),

    % circuito di uncompute
    maplist(reverse, Compute_Circuit, Uncompute_Circuit1),
    reverse(Uncompute_Circuit1, Uncompute_Circuit),

    % circuito dell'oracolo
    append(Compute_Circuit, Test_Circuit, Compute_Test_Circuit),
    append(Compute_Test_Circuit, Uncompute_Circuit, Circuit).

```

Codice 4.11: Composizione dell'oracolo.

4.3 Flusso operativo Python/Qiskit

Il flusso operativo Python/Qiskit consiste nell'insieme di funzioni che consentono l'integrazione dell'oracolo restituito dal motore logico QLPP nell'algoritmo di Grover per il problema delle n regine; in particolare, il flusso comprende la configurazione, la composizione del circuito, l'esecuzione su simulatore, il filtro di ammissibilità, la valutazione con SNR e l'ispezione degli output.

4.3.1 Configurazione

L'implementazione dell'algoritmo di Grover prevede una fase di configurazione (Codice 4.12) con:

- import dei moduli necessari;
- impostazione della dimensione della scacchiera (N);
- nome del file (prolog_filename) e query (prolog_query) per l'interfaccia verso QLPP;
- flag di salvataggio (save_to_qasm), flag di caricamento (load_from_qasm) e nome del file QASM (qasm_filename);
- flag di visualizzazione del circuito (draw_circuit);
- calcolo dei parametri di dimensionamento del registro, analogamente a quanto visto per il registro dell'oracolo QLPP;
- calcolo del numero di qubit per il diffusore (diffuser_dim), in base ai qubit di stato e ai qubit ausiliari necessari;

- calcolo della dimensione dello spazio di ricerca (S);
- liste di qubit del registro per il *binding* dell'oracolo (`oracle_list`) e del diffusore (`diffuser_list`);
- numero di soluzioni (M), determinate mediante mapping noto con N;
- calcolo dell'angolo iniziale degli stati soluzione (`teta_i`);
- numero di iterazioni di Grover (`iterations`), impostabili manualmente o in base alla probabilità di successo desiderata (1.0 per il numero ottimale di iterazioni);
- calcolo dell'angolo finale degli stati soluzione (`teta_f`);
- calcolo della probabilità di successo (`p_success`);
- numero di esecuzioni (`shots`), impostabili manualmente o in base a un calcolo empirico del minimo affinché il segnale degli stati soluzione emerga dal rumore di fondo da incertezza statistica;
- flag di post-processing (`post_processing`) per la gestione di valori di N che non sono potenze di 2, per i quali lo spazio di ricerca include valori di colonna non ammissibili.

```
# imports
import math
import pyswip
import qlpp_to_qasm
from qiskit import qasm3
from qiskit import *
from qiskit_aer import AerSimulator
from qiskit.dddsm import DDSIMProvider
from qiskit.circuit.library import MCXVChain
from qiskit.transpiler.passes import RemoveBarriers
from qiskit.visualization import plot_histogram
from IPython.display import display
%matplotlib inline

# dimensione scacchiera
N = 4

# prolog
#
# nome del file
prolog_filename = "nqueens_problem.pl"
#
# query
prolog_query = f"nqueens_problem({N},C)."
```

```

# qasm
#
# flag di salvataggio
save_to_qasm = True
#
# flag di caricamento
load_from_qasm = False
#
# nome del file
qasm_filename = "nqueens_problem.qasm"

# flag di visualizzazione del circuito
draw_circuit = False

# dimensionamento
#
# registro
queen_dim = int(math.ceil(math.log2(N)))
state_dim = N * queen_dim
checks_dim = N * (N - 1) // 2
aux_dim = checks_dim - 3
dim = queen_dim + state_dim + checks_dim + aux_dim + 1
#
# diffusore
diffuser_aux_dim = state_dim - 3
diffuser_dim = state_dim + diffuser_aux_dim
#
# spazio di ricerca
S = 2**state_dim

# binding
#
# oracolo
oracle_list = [*range(dim)]
#
# diffusore
diffuser_list = None
if diffuser_aux_dim <= aux_dim:
    diffuser_list = [
        *range(state_dim),
        *range(state_dim + queen_dim,
            state_dim + queen_dim + diffuser_aux_dim)]
else:
    diffuser_list = [
        *range(state_dim),
        *range(state_dim + queen_dim, state_dim + queen_dim + aux_dim),
        *range(state_dim + queen_dim + aux_dim + 1,
            state_dim + queen_dim + diffuser_aux_dim + 1)]

```

```

# numero di soluzioni
solution_map = {1:1,2:0,3:0,4:2,5:10,6:4,7:40,8:92,9:352,10:724,
                11:2680,12:14200,13:73712,14:365596,15:2279184,
                16:14772512,17:95815104,18:666090624,19:4968057848,
                20:39029188884,21:314666222712,22:2691008701644,
                23:24233937684440,24:227514171977364,
                25:2207893435808352,26:22317699616364044,
                27:234907967154122528}
M = solution_map.get(N,"Mapping not found.")

# angolo iniziale degli stati soluzione
teta_i = math.asin(math.sqrt(M/S))

# iterazioni
k_target = -1 # (-1 per utilizzo di p_target)
p_target = 1.0 # (1.0 per iterazioni ottimali)
iterations = None
if (k_target == -1):
    iterations = round(math.asin(math.sqrt(p_target)) /
                      (2 * teta_i) - 0.5)
else:
    iterations = k_target

# angolo finale degli stati soluzione
teta_f = (2*iterations + 1) * teta_i

# probabilità di successo
p_success = math.sin(teta_f) ** 2
print(f"Success probability: {p_success:.2%}")

# shots
shots_target = 1024 # (-1 per minimo empirico)
shots = None
if shots_target == -1:
    shots = math.ceil(25 * M / p_success)
else:
    shots = shots_target
print("Shots:",shots)

# post-processing (se N non è potenza di 2)
post_processing = False
if not ((N & (N - 1)) == 0):
    post_processing = True

```

Codice 4.12: Configurazione.

4.3.2 Composizione del circuito

La composizione del circuito Qiskit dell'algoritmo di Grover (Codice 4.13) avviene mediante la funzione `nqueens_problem()`, che prevede i seguenti passi:

1. istanza di un circuito Qiskit (`qc`) con registro quantistico dimensionato e partizionato analogamente al registro dell'oracolo QLPP e con registro classico di dimensione pari a `state_dim`;
2. inizializzazione dei qubit di stato in sovrapposizione uniforme (con Hadamard), dei qubit per lo XOR bitwise e del qubit di overflow a $|1\rangle$ (con Pauli-X) e dei restanti qubit a $|0\rangle$ (default);
3. chiamata della funzione `oracle()`, che, mediante gli elementi di interfaccia verso il motore logico QLPP, restituisce il componente circuitale Qiskit dell'oracolo (`oracle_qc`);
4. chiamata della funzione `diffuser(dif_qc)`, che costruisce e restituisce il componente circuitale Qiskit del diffusore (`dif_qc`);
5. composizione del circuito `qc` per le iterazioni di Grover specificate, con binding di `oracle_qc` e `dif_qc` per la prima iterazione e dei rispettivi blocchi unici `oracle_gate` e `diffuser_gate` per le successive;
6. aggiunta in coda a `qc` dell'operatore di misurazione sui qubit di stato.

```
# oracolo
def oracle():

    # caricamento file qasm
    if load_from_qasm:
        oracle_qc = qasm3.load(qasm_filename)

    # generazione codice qasm
    else:

        # risposta della query prolog
        solution = query_prolog()

        # circuito qlpp
        qlpp_circuit = solution["C"]

        # traduzione qlpp -> qasm
        qasm_code = qlpp_to_qasm.get_string(qlpp_circuit=qlpp_circuit,
                                           barriers=True)

    # salvataggio file qasm
```

```

        if save_to_qasm:
            with open(qasm_filename, "w", encoding="utf-8") as f:
                f.write(qasm_code)

        # conversione qasm -> QuantumCircuit
        oracle_qc = qasm3.loads(qasm_code)

    return oracle_qc

# diffusore
def diffuser(dif_qc):
    dif_qc.h(range(state_dim))
    dif_qc.x(range(state_dim))
    #
    dif_qc.barrier(range(state_dim))
    #
    dif_qc.h(state_dim - 1)
    dif_qc.compose(MCXVChain(num_ctrl_qubits=state_dim - 1),
                    range(diffuser_dim), inplace=True)
    dif_qc.h(state_dim - 1)
    #
    dif_qc.barrier(range(state_dim))
    #
    dif_qc.x(range(state_dim))
    dif_qc.h(range(state_dim))

    return dif_qc

# algoritmo di Grover per il problema delle n regine
def nqueens_problem():

    # istanza del circuito
    qc = QuantumCircuit(QuantumRegister(state_dim, 'q'),
                        QuantumRegister(queen_dim, 'xor'),
                        QuantumRegister(aux_dim, 'aux'),
                        QuantumRegister(1, 'ovfl'),
                        QuantumRegister(checks_dim, 'tgt'),
                        ClassicalRegister(state_dim, 'cbits'))

    # Hadamard sui qubit di stato
    qc.h(qc.qregs[0])

    # Pauli-X sui qubit per lo XOR bitwise e sul qubit per l'overflow
    qc.x(qc.qregs[1][:] + qc.qregs[3][:])

    qc.barrier()

```

```

# oracolo
oracle_qc = oracle()
oracle_gate = RemoveBarriers()(oracle_qc).to_gate(
    label='          ORACLE          ')

# diffusore
dif_qc = diffuser(QuantumCircuit(diffuser_dim))
dif_gate = RemoveBarriers()(dif_qc).to_gate(
    label='          DIFFUSER          ')

# composizione del circuito
for i in range(iterations):
    if i == 0:
        qc.compose(oracle_qc, range(dim), inplace=True)
        qc.barrier()
        qc.compose(dif_qc, diffuser_list, inplace=True)
    else:
        qc.compose(oracle_gate, range(dim), inplace=True)
        qc.barrier()
        qc.compose(dif_gate, diffuser_list, inplace=True)

    qc.barrier()

# misurazione
qc.measure(range(state_dim), range(state_dim))

return qc

qc = nqueens_problem()

if draw_circuit:
    display(qc.draw('mpl', style={'fontsize':18, 'subfontsize':13,
                                  'gate_textcolor':'black'},
                    fold=max(70, qc.depth() // 5),
                    scale=max(0.5, 1.0 - qc.depth() / 3000)))

```

Codice 4.13: Composizione del circuito.

4.3.3 Esecuzione su simulatore

La funzione `execution(qc, shots)` (Codice 4.14) effettua in ordine la selezione di un backend di simulazione QASM, la compilazione del circuito per il backend specifico (*transpilation*), l'esecuzione del circuito e l'estrazione delle misurazioni. I backend utilizzati sono AerSimulator di Qiskit, in modalità "statevector", e il DDSIM (*Decision Diagrams Simulator*) "qasm_simulator" di MQT (*Munich Quantum Toolkit*).

Il primo si basa su una rappresentazione esatta del vettore di stato del sistema quantistico,

ovvero sulla memorizzazione di un array di 2^{dim} ampiezze di probabilità; tale backend è stato selezionato solo per $N = 4$, dato che, per valori maggiori di N , la dimensione dim ha superato quella consentita per limiti hardware ($\text{dim} \leq 29$).

Il secondo si basa sulla costruzione di un grafo aciclico diretto (DAG) mediante diagrammi di decisione, grazie al quale circuiti con componenti simmetrici, strutturati o ridondanti possono essere compressi e l'occupazione della memoria può passare, nei casi più favorevoli, da un andamento esponenziale a polinomiale, consentendo la simulazione di circuiti con dim più elevata.

Il metodo `transpile` si occupa di ottimizzare e adattare il circuito al backend selezionato, scomponendo le porte logiche in un set di porte native supportate dal backend; inoltre, viene impostato il `seed_transpiler` per la riproducibilità della compilazione e vengono stampati la profondità del circuito e il conteggio per ciascuna porta, utili anche a valutare la fattibilità di esecuzione su hardware reale NISQ.

Il metodo `run` esegue sul backend il circuito compilato per il numero di volte indicato da `shots`, con l'impostazione del `seed_simulator` per la riproducibilità del risultato dell'esecuzione.

Infine, `result` estrae il risultato dell'esecuzione e `get_counts` ne elabora un dizionario di distribuzione statistica delle misurazioni, in cui la chiave e il valore di ciascuna coppia rappresentano rispettivamente un output e il numero di volte che è stato misurato.

```
def execution(qc, shots=1024):
    backend = None
    qc_compiled = None
    if N == 4:
        backend = AerSimulator()
        qc_compiled = transpile(RemoveBarriers()(qc),
                                optimization_level=3,
                                basis_gates=['u1', 'u2', 'u3', 'x', 'cx',
                                             'h', 's', 'sdg', 't', 'tdg'],
                                seed_transpiler=42)
    else:
        backend = DDSIMProvider().get_backend('qasm_simulator')
        qc_compiled = transpile(RemoveBarriers()(qc), backend=backend,
                                optimization_level=3,
                                seed_transpiler=42)
    print("Depth:", qc_compiled.depth())
    print("Gates:", qc_compiled.count_ops())

    job = backend.run(qc_compiled, shots=shots, seed_simulator=42)
    result = job.result()
    counts = result.get_counts()

    return counts
```

```
counts = execution(qc,shots)
```

Codice 4.14: Esecuzione su simulatore.

4.3.4 Filtro di ammissibilità

Per N non potenza di 2, viene utilizzata la funzione di post-processing `filter(counts)` per il filtraggio degli output non ammissibili, ovvero delle configurazioni che presentano uno o più valori di colonna non validi (Codice 4.15).

```
def filter(counts):
    valid_counts = {}
    is_valid = True

    for bitstring,count in counts.items():
        bitstring = bitstring[::-1]
        columns = []
        for i in range(0,state_dim,queen_dim):
            col = bitstring[i:i+queen_dim]
            if int(col[::-1],2) >= N:
                is_valid = False
                break
            else:
                columns.append(col)
        if is_valid:
            valid_bitstring = ''.join(columns[::-1])
            valid_counts[valid_bitstring] = count
        else:
            is_valid = True

    return valid_counts

if post_processing:
    counts = filter(counts)
```

Codice 4.15: Filtro di ammissibilità.

4.3.5 Valutazione con SNR

A supporto della valutazione delle prestazioni, viene applicata la metrica SNR (*Signal-to-Noise Ratio*) mediante la funzione `snr_evaluation(signal_count,shots,S)`, con `signal_count` il conteggio delle misurazioni che corrispondono a uno degli M stati soluzione (Codice 4.16); la funzione calcola l'SNR come rapporto tra le probabilità a posteriori che una misurazione sia uno stato soluzione (`p_signal_avg`) e che sia uno stato non soluzione

(p_noise_avg). L'SNR restituito dalla funzione viene quindi stampato sia come rapporto puro sia in scala logaritmica (dB), utile a esprimere ampi ordini di grandezza. Nel caso di N non potenza di 2, l'SNR viene calcolato anche dopo il filtraggio, al fine di valutarne il condizionamento dalla porzione di stati non soluzione che sono anche non ammissibili.

```
def snr_evaluation(signal_count, shots, S):
    p_signal = signal_count / shots
    p_signal_avg = p_signal / M
    p_noise = 1 - p_signal
    p_noise_avg = p_noise / (S - M)
    snr = p_signal_avg / (p_noise_avg + 1e-15)

    return snr

# conteggio degli output soluzione
signal_count = sum(sorted(counts.values(), reverse=True)[:M])

# SNR
snr = snr_evaluation(signal_count, shots, S)
snr_db = 10 * math.log10(snr)
print(f"SNR: {snr:.3f} ({snr_db:.3f} dB)")

# SNR post-filtro
if post_processing:
    shots_filt = sum(counts.values())
    S_filt = N ** N
    snr_filt = snr_evaluation(signal_count, shots_filt, S_filt)
    snr_filt_db = 10 * math.log10(snr_filt)
    print(f"SNR post-filter: {snr_filt:.3f} ({snr_filt_db:.3f} dB)")
```

Codice 4.16: Valutazione con SNR.

4.3.6 Ispezione degli output

Infine, l'ispezione degli output (Codice 4.17), eventualmente filtrati, avviene mediante un istogramma con conteggio esplicito degli M stati soluzione (funzione Qiskit `plot_histogram`), e mediante la stampa delle scacchiere soluzione con la funzione `print_boards(counts)`.

```
# istogramma
plot_histogram(counts, number_to_keep=M, title='Outputs')

# stampa delle scacchiere soluzione
def print_boards(counts):
    solutions = sorted(counts, key=counts.get, reverse=True)[:M]

    for i, solution in enumerate(solutions):
```

```

        columns = [int(solution[-state_dim:][j:j+queen_dim],2)
                    for j in range(0,state_dim,queen_dim)][::-1]
    print(f"\nSolution {i+1}: {columns}\n")

    for r in range(N):
        line = " ".join("♚" if columns[r] == c else "."
                        for c in range(N))
        print(f"| {line} |")
    print()

print_boards(counts)

```

Codice 4.17: Ispezione degli output.

4.4 Risultati

I risultati sono distinti per $N = 4$, in cui viene utilizzato AerSimulator in modalità "statevector", e per $N \in \{5, 6, 7, 8\}$, in cui viene utilizzato DDSIM "qasm_simulator".

Per $N = 4$, il circuito, con dimensione del registro $\text{dim} = 20$, è stato testato con un campionamento arbitrario di 32768 shots.

Il Codice 4.18 mostra i dati relativi alla complessità circuitale (profondità e porte) post-compilazione e all'SNR, al crescere delle iterazioni fino all'ottimo ($\text{iterations} = 8$).

```

# *** N = 4 ***
#
# Register size: 20
#
# Shots: 32768
#
# Iterations: 1
# Success probability: 6.89%
Depth: 1474
Gates: OrderedDict([('cx', 1046), ('tdg', 418), ('t', 407),
                    ('u2', 99), ('h', 67), ('u3', 37), ('u1', 32),
                    ('x', 15), ('measure', 8)])
SNR: 9.636 (9.839 dB)
#
# Iterations: 2
# Success probability: 18.34%
Depth: 2946
Gates: OrderedDict([('cx', 2092), ('tdg', 841), ('t', 819),
                    ('u2', 187), ('h', 117), ('u3', 74), ('u1', 64),
                    ('x', 28), ('measure', 8)])
SNR: 29.143 (14.645 dB)
#

```

```

# ...
# ...
# ...
#
# Iterations: 8
# Success probability: 99.56%
Depth: 11778
Gates: OrderedDict([('cx', 8368), ('tdg', 3379), ('t', 3291),
                    ('u2', 715), ('h', 417), ('u3', 296), ('u1', 256),
                    ('x', 106), ('measure', 8)])
SNR: 28182.769 (44.500 dB)

```

Codice 4.18: Simulazione con AerSimulator per $N=4$.

La profondità circuitale e il conteggio di ciascuna porta sono dei parametri di valutazione della complessità circuitale post-compilazione. In particolare, il numero di porte a due qubit (come CX e CZ) è un indicatore rilevante per l'esecuzione su hardware reale NISQ; attualmente, infatti, una porta a due qubit presenta un errore nei circuiti reali tra lo 0.1% e lo 0.5% e l'accumulo di questi errori su ciascun qubit fisico può condurre fino a una probabilità effettiva nulla di successo.

L'andamento dell'SNR dimostra la correttezza dell'implementazione algoritmica, con un segnale più grande del rumore di quasi 1 ordine di grandezza (9.839 dB) con una sola iterazione e di oltre 4 ordini di grandezza (44.500 dB) all'ottimo.

In Figura 4.9 e Figura 4.10 sono mostrati gli istogrammi per $N = 4$, rispettivamente con un'iterazione e con iterazioni ottimali; è possibile notare che, con l'adeguato campionamento adottato, il segnale relativo alle due soluzioni emerge dal rumore anche con una sola iterazione.

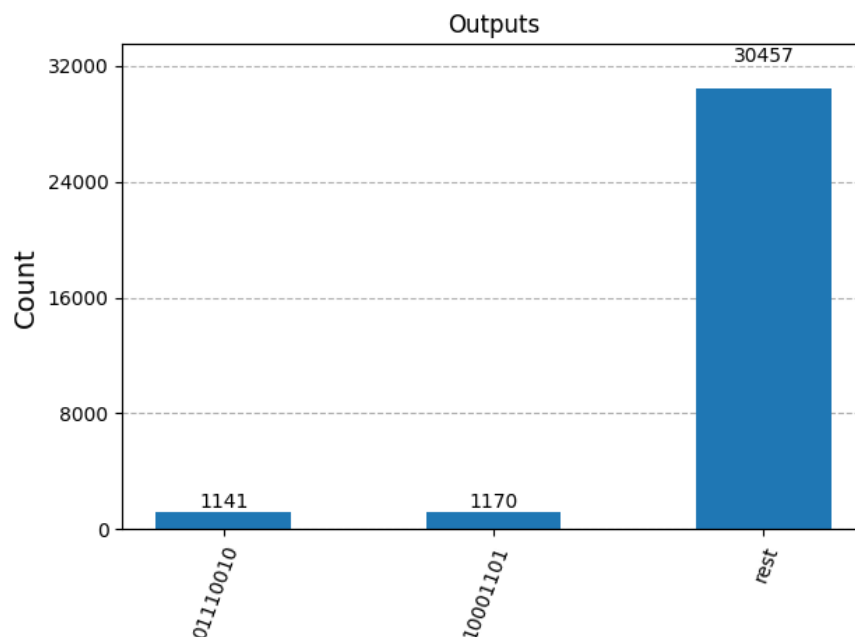


Figura 4.9: Istogramma per $N=4$, con un'iterazione.

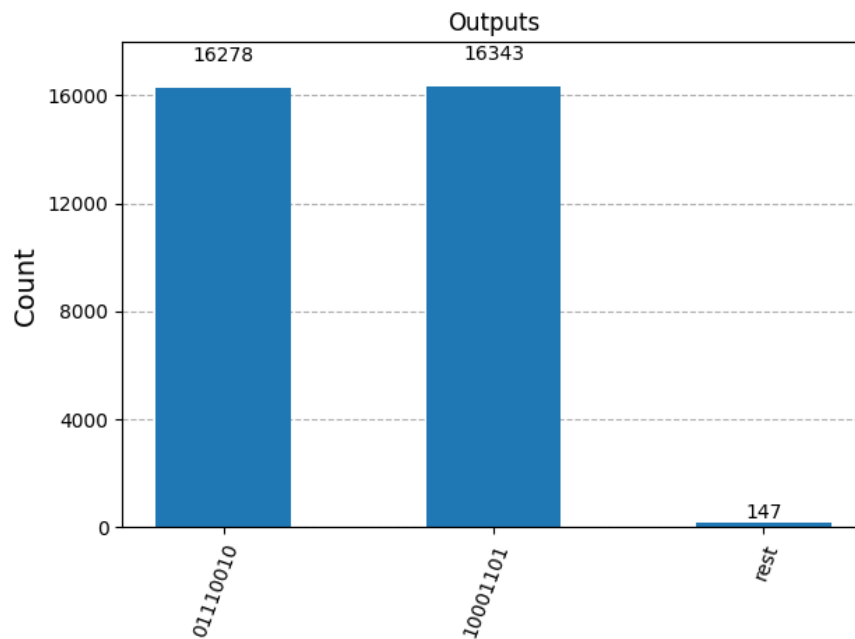


Figura 4.10: Istogramma per $N=4$, con iterazioni ottimali.

Nel Codice 4.19 sono mostrate le scacchiere soluzione per $N = 4$.

Solution 1: [1, 3, 0, 2]

```
| . ♔ . . |
| . . . ♔ |
| ♔ . . . |
| . . ♔ . |
```

Solution 2: [2, 0, 3, 1]

```
| . . ♔ . |
| ♔ . . . |
| . . . ♔ |
| . ♔ . . |
```

Codice 4.19: Scacchiere soluzione per $N=4$.

Per $N \in \{5, 6, 7, 8\}$, i circuiti sono stati testati con un campionamento basato su un minimo empirico di shots tale da far emergere il segnale dal rumore; ciò si è reso necessario per l'onere computazionale di tali simulazioni ed è stato possibile grazie all'assenza di limiti di campionamento nel simulatore DDSIM.

Il Codice 4.20 mostra i dati circuitali per $N = 5$ relativamente alle prime tre iterazioni.

```
# *** N = 5 ***
#
# Register size: 36
```

```

#
# Iterations: 1
# Success probability: 0.27%
# Shots: 91027
Depth: 884
Gates: OrderedDict([('ccx', 587), ('cx', 558), ('x', 88),
                    ('u2', 30), ('h', 15), ('measure', 15),
                    ('u1', 2), ('mcx_vchain', 1)])
SNR: 8.978 (9.532 dB)
SNR post-filter: 10.265 (10.114 dB)
#
# Iterations: 2
# Success probability: 0.76%
# Shots: 32849
Depth: 1766
Gates: OrderedDict([('ccx', 1174), ('cx', 1116), ('x', 172),
                    ('u2', 60), ('h', 15), ('measure', 15),
                    ('u1', 2), ('mcx_vchain', 2)])
SNR: 21.178 (13.259 dB)
SNR post-filter: 32.765 (15.154 dB)
#
# Iterations: 3
# Success probability: 1.49%
# Shots: 16801
Depth: 2648
Gates: OrderedDict([('ccx', 1761), ('cx', 1674), ('x', 256),
                    ('u2', 90), ('h', 15), ('measure', 15),
                    ('mcx_vchain', 3), ('u1', 2)])
SNR: 37.868 (15.783 dB)
SNR post-filter: 102.940 (20.126 dB)
#
# ...
# ...
# ...

```

Codice 4.20: Simulazione con DDSIM per $N=5$.

Va evidenziato che DDSIM mantiene le porte ad un livello di astrazione maggiore, come si può notare dalle CCX e C^nX ("mcx_vchain") che non vengono scomposte.

Anche in questo caso il segnale si presenta di quasi 1 ordine di grandezza più grande del rumore già con un'iterazione (9.532 dB). Inoltre, in questo caso, l'SNR viene calcolato una seconda volta dopo il filtraggio per valutare il condizionamento degli stati non ammissibili; con un'iterazione si rileva già un incremento dell'SNR condizionato (10.114 dB).

In Figura 4.11 è mostrato l'istogramma per $N = 5$, con tre iterazioni.

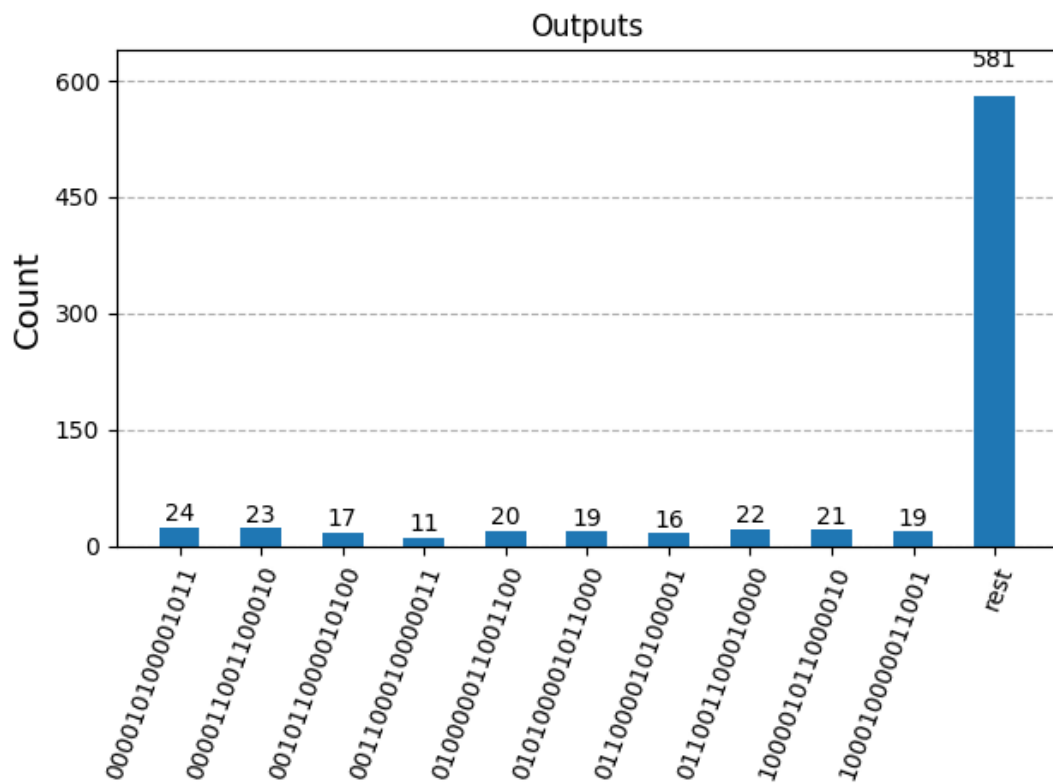


Figura 4.11: Istogramma per $N=5$, con tre iterazioni.

Nel Codice 4.21 sono mostrate le scacchiere soluzione per $N = 5$.

Solution 1: [3, 1, 4, 2, 0]

```
| . . . ♔ . |
| . ♔ . . . |
| . . . . ♔ |
| . . ♔ . . |
| ♔ . . . . |
```

Solution 2: [2, 4, 1, 3, 0]

```
| . . ♔ . . |
| . . . . ♔ |
| . ♔ . . . |
| . . . ♔ . |
| ♔ . . . . |
```

Solution 3: [0, 2, 4, 1, 3]

```
| ♔ . . . . |
| . . ♔ . . |
| . . . . ♔ |
| . ♔ . . . |
| . . . ♔ . |
```

Solution 4: [2, 0, 3, 1, 4]

.	.	♔	.	.
♔	.	.	.	.
.	.	.	♔	.
.	♔	.	.	.
.	.	.	.	♔

Solution 5: [4, 1, 3, 0, 2]

.	.	.	.	♔
.	♔	.	.	.
.	.	.	♔	.
♔	.	.	.	.
.	.	♔	.	.

Solution 6: [0, 3, 1, 4, 2]

♔	.	.	.	.
.	.	.	♔	.
.	♔	.	.	.
.	.	.	.	♔
.	.	♔	.	.

Solution 7: [1, 3, 0, 2, 4]

.	♔	.	.	.
.	.	.	♔	.
♔	.	.	.	.
.	.	♔	.	.
.	.	.	.	♔

Solution 8: [4, 2, 0, 3, 1]

.	.	.	.	♔
.	.	♔	.	.
♔	.	.	.	.
.	.	.	♔	.
.	♔	.	.	.

Solution 9: [1, 4, 2, 0, 3]

.	♔	.	.	.
.	.	.	.	♔
.	.	♔	.	.
♔	.	.	.	.
.	.	.	♔	.

Solution 10: [3, 0, 2, 4, 1]

	.	.	.	♔	.	
	♔	.	.	.	.	
	.	.	♔	.	.	
	.	.	.	.	♔	
	.	♔	.	.	.	

Codice 4.21: Scacchiere soluzione per N=5.

Risultati analoghi si sono ottenuti per $N \in \{6, 7, 8\}$; per $N > 8$ non è stato possibile simulare per l'aumento considerevole del numero di qubit e della complessità circuitale, dovuti al passaggio ad una codifica dei valori di colonna a 4 qubit. Va sottolineato, comunque, che il motore logico QLPP restituisce un circuito, tradotto e salvato in formato QASM, anche per $N > 8$, nonostante non ne sia possibile la simulazione.

In ultima analisi, i risultati ottenuti dimostrano la validità della metodologia proposta.

4.5 Possibili ottimizzazioni

La soluzione descritta può essere raffinata, in prospettiva di esecuzione su hardware reale NISQ, attraverso le seguenti possibili ottimizzazioni della complessità circuitale:

- sostituzione delle porte Toffoli di compute e uncompute con porta *Relative-phase* Toffoli (*RCCX*, *RTOF*), la cui scomposizione richiede la metà di porte *CX* rispetto alla Toffoli, ma che introduce una fase relativa indesiderata, a causa della quale non può essere utilizzata per l'azione target dell'inversione di fase dell'oracolo e in generale per operazioni di cui non è previsto uncompute in circuiti basati sull'interferenza;
- restrizione dello spazio di ricerca da N^N a $N!$ con una preparazione differente dello stato (generalizzazione QAA descritta in 2.3.5) per la generazione della sovrapposizione uniforme delle permutazioni (discussa in [10]) che soddisfano il vincolo di colonna;
- calcolo parallelo dei vincoli (o in generale delle operazioni), aumentando il numero di qubit, al fine di ridurre la profondità circuitale e di conseguenza il tasso di errore per ciascun qubit su hardware reale NISQ.

Capitolo 5

Conclusione

Il presente lavoro di tesi ha affrontato il problema della costruzione di modelli decisionali basati sul calcolo quantistico, incentrandosi sulla progettazione del framework QLPP per la formulazione logica del problema decisionale e dimostrandone le potenzialità sul noto benchmark del problema delle n regine. Un aspetto significativo del lavoro è la separazione tra la logica del problema e la ricerca delle soluzioni. Il motore logico QLPP consente, infatti, di distinguere la progettazione ad alto livello dell'oracolo dal resto del circuito nel contesto dell'algoritmo di ricerca di Grover; l'oracolo, integrato mediante l'interfaccia Python/Qiskit nell'algoritmo di Grover, marca gli stati soluzione con l'inversione di fase, e l'algoritmo, effettuando una ricerca esaustiva, li estrae con il metodo di amplificazione dell'ampiezza. Questo approccio, rispetto a quello manuale in Qiskit, facilita e automatizza la costruzione di circuiti complessi per istanze arbitrarie di un problema decisionale e di conseguenza consente di verificare la correttezza logica generale dell'oracolo tramite testing di più istanze su simulatore o hardware reale. I risultati sperimentali ottenuti per il problema delle n regine hanno evidenziato che il framework presenta gli strumenti e la scalabilità necessari alla realizzazione di circuiti arbitrariamente complessi associati a determinati problemi decisionali. La simulazione, possibile solo per alcune istanze del problema a causa dell'esplosione combinatoria dello stato e della complessità circuitale, ha restituito, grazie a un opportuno campionamento, le soluzioni corrette anche con una o poche iterazioni. La risoluzione del problema delle n regine rappresenta un risultato che evidenzia la corretta funzionalità del framework proposto, ma non un punto di arrivo; il motore logico QLPP può essere, infatti, impiegato per problemi decisionali con vincoli meno strutturati e più densi, nei quali gli algoritmi di ricerca quantistica, in prospettiva futura, possono avere un vantaggio concreto rispetto agli approcci classici. I limiti attuali del lavoro sono rappresentati dalla complessità dei circuiti generati, in cui il collo di bottiglia è rappresentato dall'oracolo; l'ottimizzazione si concentra, in particolare, sulla riduzione della profondità circuitale e del numero di porte a due qubit, al fine di rendere fattibile l'esecuzione dell'algoritmo di Grover, o almeno di un numero ridotto di sue iterazioni, su hardware reale NISQ. L'applicazione di tale approccio e il suo sviluppo futuro sono quindi da associare a due fattori: da un lato l'ottimizzazione dei circuiti, dall'altro il progressivo avvicinarsi ad hardware fault-tolerant.

In conclusione, il contributo innovativo del presente lavoro di tesi è da individuarsi nella definizione di una metodologia operativa che, integrando la programmazione logica con il calcolo quantistico, offre una direzione di interesse nella progettazione di modelli decisionali.

Bibliografia

- [1] Giovanni Pilato et al. «Towards Quantum Logic Programming». In: *2025 IEEE Conference on Pervasive and Intelligent Computing (PICOm)*. IEEE, 2025. DOI: 10 / q9fv.
- [2] M.A. Nielsen e I.L. Chuang. *Quantum Computation and Quantum Information: 10th Anniversary Edition*. Cambridge University Press, 2010. ISBN: 9781139495486.
- [3] M. L. Dalla Chiara e R. Giuntini. «Quantum Logics». In: *arXiv e-prints* (2001). DOI: 10/rnsj. arXiv: quant-ph/0101028 [quant-ph].
- [4] Ashley Montanaro. «Quantum algorithms: an overview». In: *arXiv e-prints* (2015). DOI: 10/f9mxd3. arXiv: 1511.04206 [quant-ph].
- [5] Lov K. Grover. «A fast quantum mechanical algorithm for database search». In: *arXiv e-prints* (1996). DOI: 10/fkzv3d. arXiv: quant-ph/9605043.
- [6] Gilles Brassard et al. «Quantum amplitude amplification and estimation». In: *arXiv e-prints* (2000). DOI: 10/fzjqtg. arXiv: quant-ph/0005055.
- [7] Michel Boyer et al. «Tight bounds on quantum searching». In: *Fortschritte der Physik* 46 (1998), pp. 493–506. DOI: 10/d6gp9h. arXiv: quant-ph/9605034.
- [8] Gilles Brassard, Peter Hoyer e Alain Tapp. «Quantum counting». In: *arXiv e-prints* (1998). DOI: 10/b5pjxv. arXiv: quant-ph/9805082.
- [9] Theodore J. Yoder, Guang Hao Low e Isaac L. Chuang. «Fixed-Point Quantum Search with an Optimal Number of Queries». In: *Physical Review Letters* 113.21 (2014), p. 210501. DOI: 10/w9h. arXiv: 1409.3305 [quant-ph].
- [10] Lennart Binkowski e Marvin Schwiering. «Quantum Fisher-Yates shuffle: Unifying methods for generating uniform superpositions of permutations». In: *arXiv e-prints* (2025). DOI: 10/rnrff. arXiv: 2504.17965 [quant-ph].